

Cracking TOP MNC FULL-STACK DEVELOPER INTERVIEWS

1000 Real Interview Questions & Answers

Practical | Proven | Interview-Focused

Designed For:

Freshers • 1-3 Years • 3-5 Years • 5-8 Years • 8+ Years

Covers Interviews At:

amazon Microsoft accenture Deloitte.

Infosys TCS IBM Cognizant Wipro

Cloud Docker Capgemini

Technology Stack

.NET | C# | React | Angular | SQL | Azure

Docker | CI/CD | System Design |

Microservices | Cloud | Security | HR & Leadership

Sandeep Pal

Senior Full Stack Developer | 10+ Years Experience

.NET Core • C# • Node.js • React.js • Microservices • Azure • SQL

MongoDB • Docker • EF Core • ML.NET

Top-MNC interview preparation handbook

1000 Real Interview Questions & Clear Answers

**.NET | C# | React | Angular | DSA | Azure | Docker | CI/CD
Fresher to Senior (0–8+ Years)**

Preface

Getting selected into a top multinational company is not just about knowing syntax or frameworks. It is about **thinking clearly, explaining confidently, and making the right trade-offs**—the same way experienced engineers do in real projects.

Over the years, I have seen talented developers fail interviews not because they lacked knowledge, but because they **did not know how to present that knowledge** in an interview-ready way.

This book was created to solve exactly that problem.

Inside this book, you will find **1000 carefully structured interview questions**, covering:

- Data Structures & Algorithms
- Object-Oriented Programming
- .NET, C#, React & Angular
- Databases & SQL
- DevOps, Docker & Azure
- Cloud, Security & System Design
- Microservices & Architecture

- Testing & Quality Engineering
- HR, Behavioral & Leadership rounds

Every answer is:

- Written in **simple**
- Explained with **real-time enterprise examples**
- Aligned with **actual MNC interview expectations**
- Suitable for **junior to architect-level roles**

This is not a book you read once and forget.

It is a **reference guide**, a **practice companion**, and a **confidence builder**.

If you prepare honestly using this book, you will not just clear interviews—you will **think like a professional engineer from Day One**.

I wish you clarity, confidence, and success in your interview journey.

CHAPTER-WISE (1000 QUESTIONS)

Chapter 1 – Data Structures & Algorithms (Q1–Q200)

Foundations to Advanced – MNC Interview Level

1. Arrays & Strings (Q1–Q40)
2. Searching & Sorting (Q41–Q60)
3. Stack, Queue & Linked List (Q61–Q80)
4. Hashing & Dictionary (Q81–Q100)

5. Recursion & Backtracking (Q101–Q120)
 6. Trees (Q121–Q140)
 7. Graphs (Q141–Q160)
 8. Advanced DSA Scenarios (Q161–Q200)
-

Chapter 2 – Object-Oriented Programming & Design (Q201–Q300)

OOP Fundamentals to Design Patterns

1. OOP Basics & Principles (Q201–Q220)
 2. Advanced OOP Concepts (Q221–Q240)
 3. SOLID Principles (Q241–Q260)
 4. Design Patterns (Q261–Q300)
-

Chapter 3 – C# & .NET Core Internals (Q301–Q360)

Language, Runtime & Collections

1. C# Fundamentals & Internals
 2. Async / Await & Multithreading
 3. LINQ & Collections
 4. Memory Management & GC
-

Chapter 4 – ASP.NET Core & Web API (Q361–Q420)

Backend Development – Real Projects

1. ASP.NET Core Fundamentals
 2. Middleware & Filters
 3. Dependency Injection
 4. Advanced Web API
 5. Security, Versioning & Exception Handling
-

Chapter 5 – Database & SQL (Q421–Q480)

Performance & Optimization – MNC Level

1. SQL Basics & Joins
 2. Indexing & Execution Plans
 3. Transactions & Locks
 4. Advanced SQL Optimization
-

Chapter 6 – React (Q481–Q540)

Modern Frontend – MNC Focus

1. Core Concepts
 2. Hooks & Lifecycle
 3. Forms & API Integration
 4. Performance Optimization
 5. Authentication & Authorization
-

Chapter 7 – Angular (Q541–Q600)

Enterprise Frontend Framework

1. Angular Fundamentals
 2. Components & Services
 3. Routing & State Management
 4. Reactive Forms
 5. Performance & Security
-

Chapter 8 – CI/CD, Docker & DevOps (Q601–Q660)

Enterprise Deployment & Automation

1. CI/CD Pipelines
 2. Docker & Containerization
 3. Azure DevOps
 4. Release Strategies
-

Chapter 9 – Cloud & Azure (Q661–Q720)

Cloud Architecture & Scalability

1. Azure Fundamentals
 2. App Services & Storage
 3. Monitoring & Logging
 4. Scalability & Cost Optimization
-

Chapter 10 – HR & Scenario-Based Interviews (Q721–Q780)

Behavioral, Ethics & Culture Fit

1. HR Fundamentals
 2. Teamwork & Conflict
 3. Failure & Learning
 4. Leadership & Ethics
-

Chapter 11 – System Design (Q781–Q840)

Scalability & Architecture Thinking

1. System Design Basics
 2. Real-World Design Problems
 3. Advanced Design Scenarios
-

Chapter 12 – Microservices & Architecture (Q841–Q890)

Enterprise & Distributed Systems

1. Microservices Fundamentals
 2. Communication Patterns
 3. Data Consistency
 4. Kubernetes & Service Mesh
-

Chapter 13 – Testing & Quality Engineering (Q891–Q930)

Quality, Automation & Reliability

1. Unit & Integration Testing
 2. Test Automation Strategy
 3. Performance & Security Testing
-

Chapter 14 – Security (Q931–Q970)

Application, API & Cloud Security

1. OWASP Top 10
 2. Secure Coding Practices
 3. Cloud & DevSecOps Security
-

Chapter 15 – Leadership, Architecture & Final MNC Rounds (Q971–Q1000)

Senior, Architect & Bar-Raiser Level

1. Leadership & Ownership
2. Architecture Trade-offs
3. Ethics & Decision Making
4. Final Interview Questions

Author

Sandeep Pal

Senior Full Stack Developer

13K+ LinkedIn Community

10+ Years Experience

.NET Core | C# | React.js | Angular | Microservices | Azure | Docker | SQL | MongoDB | EF Core | ML.NET

Copyright Notice

No part of this book may be reproduced, stored in a retrieval system, or transmitted in any form or by any means—electronic, mechanical, photocopying, recording, or otherwise—without prior written permission from the author, except for brief quotations used in reviews or educational purposes.

Disclaimer

This book is intended for **educational and interview preparation purposes only**.

The author makes no guarantees regarding job placement, interview results, or employment outcomes.

Company names such as **Amazon, Microsoft, Accenture, Deloitte, Capgemini, Cognizant, Infosys, TCS, IBM, Wipro** are mentioned **only for reference and interview preparation context**.

The author is **not affiliated with or endorsed by** these companies.

Trademarks

All trademarks, service marks, logos, and company names mentioned in this book are the property of their respective owners.

Liability Limitation

The author shall not be liable for any direct, indirect, incidental, or consequential damages resulting from the use of the information provided in this book.

Edition Information

- First Edition: **2026**
 - Language: English
 - Format: PDF | Word | Kindle Edition
-

Contact

For permissions, feedback, or collaboration:

LinkedIn: Sandeep Pal

How to Use This Book

This book is designed to be **flexible, practical, and result-oriented**.

You do **not** need to read it like a novel from Page 1 to Page 1000.

Instead, use it as a **personal interview coach** that adapts to:

- Your **experience level**
 - Your **target role**
 - Your **interview timeline**
-

◆ 1. Understand the Purpose of This Book

This is **not a theory textbook**.

This book helps you:

- Think like an **MNC engineer**
- Answer questions the **way interviewers expect**
- Speak with **clarity and confidence**
- Handle **technical, system design, and HR rounds**

Each answer is written to

- **Practical**
 - **Interview-ready**
 - **Based on real enterprise experience**
-

◆ **2. Choose Your Starting Point (Very Important)**

You do **not** need to start from Chapter 1 if you already have experience.

If You Are a Fresher / 0–1 Year

Start with:

- Chapter 1 – DSA
- Chapter 2 – OOP
- Chapter 3 – C# & .NET
- Chapter 10 – HR & Scenario-Based

Focus on **clarity**, not speed.

If You Have 1–3 Years of Experience

Start with:

- Chapter 3 – C# & .NET

- Chapter 4 – ASP.NET Core & Web API
- Chapter 5 – Database & SQL
- Chapter 6 / 7 – React or Angular
- Chapter 10 – HR

Goal: **Clear technical rounds confidently.**

If You Have 3–5 Years of Experience

Start with:

- Chapter 8 – CI/CD, Docker & DevOps
- Chapter 9 – Cloud & Azure
- Chapter 11 – System Design
- Chapter 13 – Testing & Quality
- Chapter 14 – Security

Goal: **Move from “developer” to “senior engineer” mindset.**

If You Have 5+ Years of Experience

Start with:

- Chapter 11 – System Design
- Chapter 12 – Microservices & Architecture
- Chapter 14 – Security
- Chapter 15 – Leadership & Final Rounds

Goal: **Clear architect, lead, and bar-raiser rounds.**

◆ 3. How to Study Each Question (Best Practice)

For every question, follow this simple 4-step approach:

1. **Read the question carefully**
2. **Read the explanation** (understand the “why”)
3. **Understand the real-time example**
4. **Practice saying the answer aloud in your own words**

📌 Tip: Interviewers care more about **how you explain** than how much you know.

◆ 4. Do Not Memorize — Understand

This book is **not meant for memorization**.

✗ Don't try to remember answers word-for-word
Understand concepts and explain them naturally

If you can explain something:

- Simply
- Clearly
- With an example

You are **interview-ready**.

◆ 5. Use the Book for Different Interview Rounds

🔧 Technical Rounds

Use:

- Chapters 1–9
Focus on **problem-solving and clarity**.
-

🏗️ System Design Rounds

Use:

- Chapter 11 & 12
Practice **thinking aloud** and **explaining trade-offs**.
-

HR & Behavioral Rounds

Use:

- Chapter 10
- Chapter 15

Practice answers using the **STAR method**.

◆ **6. Practice Company-Wise Mock Interviews**

At the end of the book, you'll find:

- Amazon mock interviews
- Microsoft mock interviews
- Accenture, Deloitte, Infosys-style interviews

Use these to:

- Simulate real interview flow
 - Improve confidence
 - Reduce interview fear
-

◆ **7. Follow the 30-60-90 Day Study Plan**

If you have:

- **3 months** → Follow the full 30-60-90 plan

- **1 month** → Focus on your target role chapters
- **2 weeks** → Revise weak areas + HR + system design

Consistency matters more than speed.

◆ 8. Use This Book During Interviews

You can safely use this book to:

- Revise before interviews
- Prepare last-minute concepts
- Improve explanation skills
- Mentor others

This book is a **long-term career asset**, not a one-time read.

◆ 9. Final Advice from the Author

You don't need to know everything.

Interviewers are looking for:

- Clear thinking
- Honest answers
- Ownership mindset
- Willingness to learn

If you prepare sincerely using this book, you will not only clear interviews — you will **grow as a professional engineer**.

You Are Ready

Read patiently.
Practice consistently.
Speak confidently.

And remember:

**Interviews don't test perfection.
They test thinking, clarity, and attitude.**

- Questions are **based on real MNC interview patterns** (TCS, Infosys, Accenture, Deloitte, Amazon, Microsoft, etc.)
- Difficulty increases **chapter-wise**
- Each question includes:
 - **Direct Answer**
 - **Real-Time Project Example**
 - **Interviewer Expectation**
 - **Common Mistakes**

(Top MNC Interview Questions – Fresher to Senior Level)

Companies Covered: TCS, Infosys, Accenture, Cognizant, Wipro, Capgemini, Deloitte, Amazon, Microsoft

Difficulty: Easy → Medium → Hard

Focus: Real interview logic, not theory

CHAPTER 1 – DATA STRUCTURES & ALGORITHMS

◆ 1.1 ARRAYS & STRINGS

(Improved, Easy-Readable, Interview-Ready Answers – Q1 to Q20)

Q1. What is an array and why is it used?

Improved Explanation

An array is a **fixed-size collection** used to store multiple values of the **same data type** in **continuous memory**.

Think of it like keeping items in labeled lockers placed side-by-side — every locker has equal size and fixed count.

Why Arrays Are Useful

- When you **already know how many items** you want to store
- When you want **fast and direct access** using index
- When memory efficiency matters

Real-Time Industry Example

Suppose a finance team tracks monthly revenue data. We know months = 12 (fixed). So the array is perfect.

```
int[] monthlySales = { 12000, 15000, 18000, 20000 };  
  
Console.WriteLine(monthlySales[2]); // Output: 18000
```

Common Interview Mistakes

- ✗ Saying “arrays are dynamic”
 - ✗ Saying “array grows automatically”
-

Q2. Why is array index access $O(1)$?

Improved Explanation

Array elements are stored **continuously in memory**.

So to access an element, computer directly calculates its location using formula:

$$\text{Address} = \text{BaseAddress} + (\text{Index} \times \text{SizeOfEachElement})$$

No loop, no searching, no scanning → **Direct jump**, so it's **constant time O(1)**

Real-Time Example

Fetching employee salary by index

```
int[] salaries = { 30000, 40000, 50000 };  
int empSalary = salaries[1];
```

Interview Winning Line

“It is O(1) because arrays support direct memory indexing due to contiguous allocation.”

Q3. Difference between Array and List in C#

Improved Explanation

Feature	Array	List
Size	Fixed	Dynamic
Performance	Faster	Slightly slower
Memory	Compact	Uses extra memory
Best For	Known size data	Unknown / growing data

Real-Time Example

API returning unknown number of users → we cannot decide size in advance → so List is better

```
List<string> users = new List<string>();  
users.Add("John");  
users.Add("Mary");
```

When to Use

- ✓ Array → Fixed dataset (months, weekdays, fixed config values)
 - ✓ List → Data coming from APIs / DB / User input
-

Q4. What happens when array size is exceeded?

Improved Explanation

Array has fixed boundaries.

If you try to insert or read outside its limit, C# throws:

`IndexOutOfRangeException`

Real-Time Example

```
int[] arr = new int[3];  
arr[3] = 10; // Crash at runtime
```

Interview Tip

Always validate index before usage

Q5. Why are strings immutable in C#?

Improved, Clear Reasoning

Once a string is created, it **cannot be changed**.
Any modification creates a **new string** in memory.

Why Microsoft Designed It This Way?

1 Security

Password strings should not be modifiable in memory

2 Thread Safety

Multiple threads can safely share same string instance

3 Performance

String interning & pooling improves performance

Real Project Example

```
string role = "Admin";  
  
role += "User";  
  
// Actually creates new string "AdminUser"
```

Q6. Why is StringBuilder better in loops?

Improved Explanation

- String creates **new object every modification**
- Thousands of concatenations = thousands of objects = slow + memory waste
- StringBuilder maintains **single modifiable buffer**
- So faster + memory efficient

Real-Time Example

Creating logs, reports, big text responses

```
StringBuilder log = new StringBuilder();
```

```
for(int i = 1; i <= 1000; i++)  
{  
    log.Append("Log entry " + i);  
}
```

Rule

Use `StringBuilder` when working in **loops / large concatenations**

Q7. Reverse an array

Uses **Two Pointer Technique**

Swap from start & end until both meet.

```
int[] arr = {1, 2, 3, 4};  
  
int left = 0, right = arr.Length - 1;  
  
while(left < right)  
{  
    int temp = arr[left];  
    arr[left] = arr[right];  
    arr[right] = temp;  
    left++;  
    right--;  
}
```

Real-Time Usage

- Reversing arrays in analytics
 - Reversing queue orders
 - Algorithms problem solving
-

Q8. Find duplicate elements in array

Improved Explanation

Use a Dictionary to count frequency.

```
int[] nums = {1,2,3,2,4,1};

Dictionary<int,int> freq = new();

foreach(var n in nums)
{
    if(freq.ContainsKey(n))
        freq[n]++;
    else
        freq[n] = 1;
}
```

Real-Time Usage

- Detecting duplicate employee IDs
 - Duplicate order entries
 - Duplicate transactions
-

Q9. What is the time complexity of Linear Search?

Improved Explanation

Linear search checks **every element one-by-one**

Worst case → element is last or not available

So complexity = **$O(n)$**

```
bool found = false;

foreach(int n in arr)
{
    if(n == target)
    {
        found = true;
        break;
    }
}
```

Real-World Understanding

Imagine checking each name in attendance register manually

Q10. When NOT to use arrays?

Avoid arrays when:

- ✗ Size is unknown
- ✗ You need frequent insert/delete
- ✗ Real-time dynamic growing data

Real Example

User list from API – count unknown → so use List

Q11. Length vs Count()

Improved Explanation

- `Length` → Built-in property → Faster
- `Count()` → LINQ extension → Slight overhead

```
int[] marks = { 80, 85, 90 };
```

```
int len1 = marks.Length;
```

```
int len2 = marks.Count();
```

Rule

Prefer **Length** for arrays

Q12. Find maximum element

Simple looping or built-in method

```
int[] sales = { 1000, 5000, 3000 };
```

```
int max = sales[0];
```

```
foreach(int s in sales)
```

```
{
```

```
    if(s > max)
```

```
        max = s;
```

```
}
```

Real Use

- Max salary
 - Max temperature
 - Max visitors in analytics
-

Q13. What is a jagged array?

Array of arrays with **different length rows**

```
int[][] scores = new int[2][];  
  
scores[0] = new int[] { 80, 90 };  
  
scores[1] = new int[] { 70, 75, 85 };
```

Real Usage

- Marks per student
 - Orders per customer
 - Seats per row in cinema
-

Q14. Jagged vs Multidimensional

Jagged	Multi-Dimensional
Flexible row size	Fixed rows × columns
Memory efficient	Uses continuous matrix

Q15. Why should string comparison be careful?

Because:

- Case sensitivity matters
- Culture settings matter

```
bool equal = string.Equals("admin", "Admin",  
StringComparison.OrdinalIgnoreCase);
```

Real Use

Login username comparison

Q16. Check Palindrome

Two pointer check

Q17. Remove duplicate characters

Uses HashSet to ensure unique

Q18. Why is string concatenation slow in loops?

Because immutable → new object on every change

Q19. Sliding Window Technique

Used to optimize repetitive calculations

Q20. When to avoid strings?

Avoid when:

- Too many modifications
- Logging large data
- Generating huge documents

Use `StringBuilder`

1.1 ARRAYS & STRINGS (Q21–Q40)

(Asked in TCS, Infosys, Accenture, Cognizant, Amazon, Microsoft)

Q21. (TCS – Screening Round)

How do you find the second largest element in an array?

Why interviewer asks

To see whether you can solve logically without shortcuts like sorting.

Explanation

We scan array only once and maintain:

- `first` → largest number
- `second` → second largest

Whenever we find a bigger value than first, we shift `first` → `second`.

Real-time C# Example

```
int[] scores = { 65, 85, 90, 70 };

int first = int.MinValue, second = int.MinValue;

foreach(int s in scores)
{
    if(s > first)
    {
        second = first;
        first = s;
    }
    else if(s > second && s != first)
    {
        second = s;
    }
}
```

Edge Cases

- All values same → second largest doesn't exist
- Array length < 2 → invalid

Real Project Usage

Finding:

- 2nd highest revenue month
 - Runner-up performance score
-

Q22. (Infosys – Technical Round)

What happens internally when List capacity increases?

Explanation

`List<T>` is backed by an internal array.

When it runs out of space:

- 1) Allocates **new bigger array** (usually double capacity)
- 2) Copies old values to new array
- 3) Discards old array

This operation is expensive.

C# Example

```
List<int> data = new();  
  
for(int i = 0; i < 100; i++)  
    data.Add(i);
```

Real Concept

This is why adding millions of items repeatedly hurts performance.

Interview Tip

Say:

“Capacity increase causes resizing + copying overhead. Pre-define capacity when possible.”

Q23. (Accenture – Developer Round)

Difference between `Array.Copy` and manual loop?

Explanation

- `Array.Copy` uses optimized low-level memory operations
- Faster than a normal `for` loop
- Less error-prone

C# Example

```
int[] src = {1,2,3};  
int[] dest = new int[3];  
Array.Copy(src, dest, src.Length);
```

Real Usage

- Migrating cache blocks
- Buffer copying
- High-performance systems

Q24. (Wipro – Fresher Round)

What is default value of array elements in C#?

Explanation

C# initializes arrays automatically:

Type	Default
int	0

string null

bool false

reference types null

Example

```
int[] nums = new int[3]; // {0,0,0}
```

Why?

Avoids garbage values → more safe.

Q25. (Microsoft – Technical Round)

Why are arrays faster than Lists?

Explanation

Arrays are:

- Fixed size
- No resizing logic
- Direct contiguous memory access
- No overhead of capacity management

Lists internally use arrays PLUS extra management.

Real Example

Processing 1M sensor values per second — arrays perform better.

Q26. (Amazon – Interview Favorite)

Move all zeros to end of array

Explanation

Use two-pointer technique:

- One scans
- One stores non-zero values

C# Example

```
int[] arr = {0,1,0,3,12};  
  
int index = 0;  
  
for(int i = 0; i < arr.Length; i++)  
{  
    if(arr[i] != 0)  
        arr[index++] = arr[i];  
}  
  
while(index < arr.Length)  
    arr[index++] = 0;
```

Real Usage

Cleaning transaction records with null/zero placeholders.

Q27. (Capgemini – Screening Round)

Time complexity of reversing an array?

Explanation

Each element swapped once → $O(n)$

Real Example

Reversing log history to show latest first.

Q28. (Accenture – Technical Round)

How do you check if an array is sorted?

Explanation

Compare every element with previous.

C#

```
bool sorted = true;

for(int i = 1; i < arr.Length; i++)
{
    if(arr[i] < arr[i-1])
    {
        sorted = false;
        break;
    }
}
```

Real Use

Validating sorted analytics data.

Q29. (Infosys – Interview Favorite)

Difference between `string.Empty` and `""`?

Explanation

Both represent empty strings.

But:

- `string.Empty` – improves readability, avoids typo confusion
- `""` – literal empty string

Real Example

Useful in validation, APIs, form handling.

Q30. (Amazon – Coding Round)

Find first non-repeating character in string

Explanation

Count frequency → return first char with count 1.

```
string input = "swiss";
```

```
Dictionary<char,int> freq = new();
```

```
foreach(char c in input)
```

```
    freq[c] = freq.ContainsKey(c) ? freq[c] + 1 : 1;
```

```
char result = input.First(c => freq[c] == 1);
```

Real Use

Fraud detection, analytics, NLP.

Q31. (TCS – Screening Round)

What is a two-pointer technique?

Explanation

Use two indices at same time instead of nested loops.

Uses

- Reverse array
- Remove duplicates
- Pair sum problems
- Palindrome check

Real Example

Finding pairs matching a target purchase amount.

Q32. (Accenture – Coding Round)

Check if two strings are anagrams

Explanation

Two strings are anagrams if characters match (order doesn't matter).

```
bool isAnagram =  
  
    string.Concat(str1.OrderBy(c => c)) ==  
  
    string.Concat(str2.OrderBy(c => c));
```

Real Use

Spam detection, text comparison.

Q33. (Microsoft – Advanced Round)

Why is string comparison slow sometimes?

Explanation

Comparison happens:

- character by character
- larger string → more time
- culture rules may apply

Real Example

Comparing large JSON keys / API headers.

Q34. (Amazon – DSA Round)

Remove duplicates from array

```
int[] unique = arr.Distinct().ToArray();
```

Real Use

Cleaning duplicate transaction records.

Q35. (Wipro – Fresher Round)

What is an immutable object?

Explanation

Objects whose value CANNOT be modified after creation.

Examples

- string
- DateTime
- record types

Benefits

- Thread safe
 - Predictable behavior
-

Q36. (Accenture – Developer Round)

Why is `Span<T>` faster in some cases?

Explanation

- Works with stack & unmanaged memory
- Avoids memory allocation
- Zero-copy slicing

Real Example

Used in high-performance APIs, parsers, serializers.

Q37. (Infosys – Technical Round)

What is `IndexOf` time complexity?

Explanation

Checks elements one by one → **$O(n)$**

Q38. (Amazon – Coding Round)

Count vowels in string

```
int count = 0;

foreach(char c in input.ToLower())
{
    if("aeiou".Contains(c))
        count++;
}
```

Real Use

NLP processing, text analytics.

Q39. (Capgemini – Screening Round)

Why are arrays not thread-safe?

Explanation

Multiple threads can modify the same index simultaneously → race conditions.

Impact

- Corrupted data
- Random behavior

Use lock / immutable data.

Q40. (Microsoft – Interview Favorite)

When do you prefer `ReadOnlySpan<char>` over `string`?

Explanation

Use when:

- Performance critical
- No need to create new string
- Need temporary read-only view

Real Example

Parsing files, log processing.

Q41. (TCS – Screening Round)

What is Linear Search and when should you use it?

Explanation

Check each element one by one until found.

Works On

- ✓ Sorted
- ✓ Unsorted data

C#

```
int[] ids = {101,105,110};
```

```
int target = 105;
```

```
bool found = false;
```

```
foreach(int id in ids)
```

```
{
```

```
    if(id == target)
```

```
    {
```

```
        found = true;

        break;
    }
}
```

Best Use

Small lists, rarely searched data.

Q42. (Infosys – Technical Round)

Time complexity of Linear Search?

Worst case:

- element at end or not present → $O(n)$
-

Q43. (Accenture – Interview Favorite)

What is Binary Search?

Explanation

Divide the array into halves repeatedly.
Works ONLY on sorted arrays.

Q44. (Microsoft – Technical Round)

Why is Binary Search faster than Linear Search?

Explanation

Removes half elements every step → $O(\log n)$

Perfect for

Large sorted datasets.

Q45. (Wipro – Fresher Round)

What happens if Binary Search is used on unsorted data?

Explanation

Output becomes unreliable / wrong.
Binary search assumes sorted order.

Interview Tip

Always confirm data sorted before using.

SORTING

Q46. (TCS – Screening Round)

What is Sorting and why is it needed?

Explanation

Sorting means arranging data in a defined order such as ascending or descending.
It is required because sorted data:

- Makes searching much faster
- Helps in better decision-making and analytics
- Produces meaningful reports and dashboards
- Improves readability and user experience

In real systems, sorting is everywhere — UI tables, leaderboards, billing systems, analytics, dashboards, etc.

Real-time Example

Bank statement sorting transactions by **date**, **amount**, or **latest first**.

Q47. (Infosys – Technical Round)

What is Bubble Sort?

Explanation

Bubble Sort is a simple comparison-based sorting technique.

It repeatedly compares **adjacent elements** and swaps them when they are in the wrong order.

After each iteration, the largest element “bubbles up” to its correct position.

Real-time C#

```
int[] arr = { 5, 3, 2, 4 };

for (int i = 0; i < arr.Length; i++)
{
    for (int j = 0; j < arr.Length - i - 1; j++)
    {
        if (arr[j] > arr[j + 1])
        {
            int temp = arr[j];
            arr[j] = arr[j + 1];
            arr[j + 1] = temp;
        }
    }
}
```

Interview Note

Used mainly for learning purposes, not real production.

Q48. (Amazon – Interview Favorite)

Why is Bubble Sort not used in production?

Explanation

Because Bubble Sort has:

- **Time Complexity = $O(n^2)$**
- Performs too many unnecessary comparisons and swaps
- Becomes extremely slow for large datasets

Modern applications prefer **Merge Sort, Quick Sort, Heap Sort**, or built-in optimized sorting algorithms.

Q49. (Accenture – Developer Round)

What is Selection Sort?

Explanation

Selection Sort finds the **smallest element** in the unsorted part and places it at the correct position.

Fewer swaps but still $O(n^2)$.

Real-time C#

```
for (int i = 0; i < arr.Length; i++)
{
    int min = i;

    for (int j = i + 1; j < arr.Length; j++)
        if (arr[j] < arr[min])
```



```
        min = j;

    int temp = arr[i];
    arr[i] = arr[min];
    arr[min] = temp;
}
```

Q50. (Microsoft – Technical Round)

Difference between Bubble Sort and Selection Sort

Explanation

Bubble Sort

- Swaps many times
- Repeated adjacent comparisons

Selection Sort

- Finds minimum first
- Only **one swap per pass**

Interview Tip

Say clearly:

“Selection Sort does fewer swaps, but both are $O(n^2)$.”

Q51. (Capgemini – Screening Round)

What is Insertion Sort?

Explanation

Insertion Sort builds the final sorted array **one element at a time**.
It shifts elements and inserts the new element in its correct position.

Real-time C#

```
for (int i = 1; i < arr.Length; i++)  
{  
    int key = arr[i];  
    int j = i - 1;  
  
    while (j >= 0 && arr[j] > key)  
    {  
        arr[j + 1] = arr[j];  
        j--;  
    }  
    arr[j + 1] = key;  
}
```

Real Usage

Very efficient when data is:

- Small in size
- Already almost sorted (like logs, small UI lists)

Q52. (Amazon – DSA Round)

When is Insertion Sort preferred?

Explanation

Use Insertion Sort when:

- Dataset is **small**
- Data is **almost sorted**
- Memory is limited
- Need stable sorting

It performs close to $O(n)$ in these cases.

Q53. (Infosys – Technical Round)

What is the time complexity of Insertion Sort?

Explanation

- Best Case (almost sorted) → $O(n)$
 - Worst Case (reverse sorted) → $O(n^2)$
-

Q54. (Microsoft – Advanced Round)

What is Merge Sort?

Explanation

Merge Sort is a **Divide and Conquer** algorithm:

- Divide array into halves
- Sort each half
- Merge sorted halves

It guarantees $O(n \log n)$ performance.

Real-world Example

Used for **huge datasets**, sorting files, distributed systems.

Q55. (Amazon – Interview Favorite)

Why is Merge Sort stable?

Explanation

Merge Sort does not change the order of equal elements during merging.
So records having the same value retain original order.

Very important in:

- Banking statements
- Sorting students having same marks
- Sorting employees by salary then name

Q56. (Accenture – Developer Round)

What is Quick Sort?

Explanation

Quick Sort picks a **pivot**, partitions elements around it:

- Left side → smaller elements
- Right side → larger elements

Very fast in the average case.

Real-time Usage

Modern languages including .NET internally use **Introsort** which is based on Quick Sort.

Q57. (Microsoft – Technical Round)

Average time complexity of Quick Sort?

Answer: $O(n \log n)$

Efficient and widely used.

Q58. (Amazon – DSA Round)

Why is Quick Sort worst case $O(n^2)$?

Explanation

When pivot selection is poor such as:

- Always selecting first element in sorted array
- Highly skewed partitions

Then recursion depth increases and performance degrades.

Q59. (Capgemini – Screening Round)

What is stable sorting?

Explanation

A sorting algorithm is stable if:

When two elements have equal values → their original order remains the same.

Real-time Example

Sorting:

Employees by Department

Then name should not shuffle inside department

Q60. (Deloitte – Interview Favorite)

Which sorting is used in .NET internally?

Explanation

.NET uses **Introsort**:

- Combination of Quick Sort + Heap Sort + Insertion Sort

- Automatically selects best strategy
 - Gives best performance
-

◆ 1.3 STACK, QUEUE & LINKED LIST

STACK (LIFO)

Q61. (TCS – Screening Round)

What is a Stack and where is it used?

Explanation

Stack follows **LIFO** – Last In First Out.
The last inserted element is removed first.

Real-time Uses

- Undo / Redo feature
- Browser back navigation
- Function call execution

C#

```
Stack<string> actions = new Stack<string>();  
  
actions.Push("Type A");  
  
actions.Push("Type B");  
  
actions.Pop();
```

Q62. (Infosys – Technical Round)

What are the basic operations of Stack?

Explanation

- Push → insert element
 - Pop → remove top
 - Peek → check top without removing
-

Q63. (Accenture – Interview Favorite)

What is Stack Overflow?

Explanation

Occurs when stack memory gets full due to:

- Deep recursion
- Infinite recursion
- Too many nested calls

Example

```
void Test()  
{  
    Test(); // infinite recursion  
}
```

Q64. (Microsoft – Technical Round)

How is stack used in function calls?

Explanation

Every function call is stored in **call stack**:

- Stores parameters
- Local variables
- Return address

After execution → removed from stack.

Used in:

- Exception handling
- Debugging
- Execution tracing

Q65. (Amazon – DSA Round)

Why is stack used for expression evaluation?

Explanation

Expressions contain nested brackets and precedence rules.
Stack naturally supports nested processing.

Real Example

Evaluating:

$(a + b) * c$

Compiler uses stack.

QUEUE (FIFO)

Q66. (TCS – Screening Round)

What is Queue and where is it used?

Explanation

Queue works on **FIFO – First In, First Out**.

Real-time Uses

- Order processing
- Printer jobs
- Customer support systems
- API request handling

C# Example

```
Queue<string> requests = new Queue<string>();  
requests.Enqueue("Req1");  
requests.Dequeue();
```

Q67. (Infosys – Technical Round)

Basic operations of Queue

- Enqueue → Add
- Dequeue → Remove
- Peek → Read first element

Q68. (Accenture – Developer Round)

Difference between Stack and Queue

Stack	Queue
LIFO	FIFO
Undo operations	Scheduling
Function call stack	Task queues

Q69. (Microsoft – System Thinking)

Why queues are used in microservices?

Explanation

Queues help to:

- Handle high traffic smoothly
- Decouple services
- Avoid system crashes
- Enable async processing

Real-time Example

Order placed → goes to Queue → processed safely

Azure Service Bus, RabbitMQ, Kafka.

Q70. (Amazon – Advanced Round)

What is a Circular Queue?

Explanation

In Circular Queue:

- Last position connects to first
- Utilizes memory better than normal queue

Real-time Example

CPU scheduling

Round Robin scheduling

LINKED LIST

Q71. (Wipro – Fresher Round)

What is a Linked List?

Explanation

Data stored as **nodes**, each pointing to the next node.

Real-time C# example

```
class Node
{
    public int Data;
    public Node Next;
}
```

Q72. (Infosys – Technical Round)

Why is Linked List insertion faster than array?

Explanation

No shifting of elements, only pointer change.

Real-time example

Chat messages insertion.

Q73. (Amazon – DSA Round)

Why is Linked List search slow?

Explanation

No direct index access → traversal needed.

Q74. (Microsoft – Technical Round)

Singly vs Doubly Linked List

Explanation

Singly

Doubly

One pointer

Two pointers

Less memory

More memory

Q75. (Accenture – Interview Favorite)

Find length of Linked List

C# example

```
int Count(Node head)
{
    int count = 0;
    while(head != null)
    {
        count++;
        head = head.Next;
    }
    return count;
}
```

Q76. (Amazon – Coding Round)

Detect loop in Linked List

Explanation

Use slow and fast pointers.

Real-time C# example

```
bool HasLoop(Node head)
{
    Node slow = head, fast = head;
    while(fast != null && fast.Next != null)
    {
        slow = slow.Next;
        fast = fast.Next.Next;
    }
}
```

```
        if(slow == fast) return true;
    }

    return false;
}
```

Q77. (Microsoft – Advanced Round)

Why is Linked List not cache friendly?

Explanation

Nodes are not stored in contiguous memory.

Q78. (Deloitte – Senior Round)

Linked List vs Array – real project choice

Explanation

- Array → fast access
 - Linked List → dynamic size
-

Q79. (Amazon – DSA Round)

Reverse a Linked List

Real-time C# example

```
Node Reverse(Node head)
{
    Node prev = null, curr = head;
```

```
while(curr != null)
{
    Node next = curr.Next;

    curr.Next = prev;

    prev = curr;

    curr = next;
}

return prev;
}
```

Q80. (Capgemini – Screening Round)

Real-world use of Linked List

Answer

- Music playlist
- Browser navigation
- LRU Cache

◆ 1.4 HASHING & DICTIONARY (Q81–Q100)

(Very common in TCS, Infosys, Accenture, Amazon, Microsoft, Deloitte)

HASHING BASICS

Q81. (TCS – Screening Round)

What is hashing in simple words?

Explanation

Hashing means converting a **key into a number (index)** so we can **store and find data quickly**.

Instead of searching line by line, we jump directly to the location.

Real-time C# example

Finding user by email in memory:

```
Dictionary<string, string> users = new Dictionary<string, string>();  
users["john@mail.com"] = "John";
```

```
string name = users["john@mail.com"]; // Fast lookup
```

Q82. (Infosys – Technical Round)

Why is Dictionary lookup usually $O(1)$?

Explanation

Because the key is converted into a **hash code**, which directly points to the storage location.

Real-world example

Checking user permissions in middleware.

Q83. (Accenture – Interview Favorite)

Difference between List and Dictionary in real projects

Explanation

- List → search by looping

- Dictionary → search by key

C# example

```
// List
usersList.Find(u => u.Id == 10);

// Dictionary
usersDict[10];
```

Interview tip

Say: “A dictionary is preferred when a fast lookup is needed.”

Q84. (Wipro – Fresher Round)

What is a hash collision?

Explanation

When **two different keys generate the same hash value**, it's called a collision.

Real-life example

Two people assigned the same locker number.

Q85. (Amazon – DSA Round)

How does the Dictionary handle collisions?

Explanation

Internally uses **chaining** (multiple values at same index).

Why interviewer asks

To check understanding beyond usage.

Q86. (Microsoft – Technical Round)

Why is a Dictionary faster than searching in an array?

Explanation

Array search $\rightarrow$ loop $\rightarrow O(n)$

Dictionary $\rightarrow$ direct access $\rightarrow O(1)$

Real-time example

Token validation in authentication.



DICTIONARY USAGE

Q87. (Accenture – Coding Round)

Count frequency of elements using Dictionary

Real-time C# example

```
int[] nums = {1,2,2,3,1};  
Dictionary<int,int> freq = new();
```

```
foreach(int n in nums)  
{  
    if(freq.ContainsKey(n))  
        freq[n]++;  
    else  
        freq[n] = 1;
```

```
}
```

Real usage

Analytics, reports, logs.

Q88. (Infosys – Technical Round)

Why must Dictionary keys be unique?

Explanation

Key represents the direct **access path**.
Duplicate keys overwrite values.

Real-time example

UserId must be unique.

Q89. (Amazon – Interview Favorite)

Find first non-repeating character in a string

Explanation

Count characters, then scan again.

C# example

```
string input = "swiss";
```

```
Dictionary<char,int> map = new();
```

```
foreach(char c in input)
```

```
    map[c] = map.ContainsKey(c) ? map[c] + 1 : 1;
```

```
char result = input.First(c => map[c] == 1);
```

Q90. (Microsoft – Advanced Round)

Why does Dictionary performance degrade sometimes?

Explanation

Too many collisions → lookup becomes slower.

Real-world example

Poor cache key design.

ADVANCED HASHING CONCEPTS

Q91. (Capgemini – Screening Round)

Hashtable vs Dictionary

Explanation

- Hashtable → non-generic, slower
- Dictionary → generic, type-safe

Interview tip

Dictionaries are preferred in modern .NET.

Q92. (Accenture – Developer Round)

Can a dictionary have a null key?

Answer

✗ No, Dictionary does not allow null keys.

Why asked

Check hands-on experience.

Q93. (Amazon – System Thinking)

Why is Dictionary used in caching?

Explanation

Caching needs **fast read access**.

Real-time example

In-memory API response cache.

Q94. (Microsoft – Technical Round)

Why is Dictionary not thread-safe?

Explanation

Multiple threads can modify it simultaneously.

Q95. (Accenture – Advanced)

How to make Dictionary thread-safe?

C# example

```
ConcurrentDictionary<int,string> cache =  
    new ConcurrentDictionary<int,string>();
```

Q96. (Deloitte – Senior Round)

Dictionary vs ConcurrentDictionary

Explanation

ConcurrentDictionary handles multi-threading without manual locks.

Q97. (Amazon – Design Round)

Why is hashing used in authentication systems?

Explanation

Passwords are stored as **hash**, not plain text.

Real-world usage

Login systems.

Q98. (Infosys – Technical Round)

Why is hashing one-way?

Explanation

Hash cannot be reversed, only verified.

Q99. (Microsoft – Advanced Round)

When should you NOT use a Dictionary?

Explanation

- When order matters
 - When range queries are needed
-

Q100. (Amazon – Final DSA Round)

Real production use of Dictionary

Sample interview answer

"I used Dictionary to store role-permission mapping in a .NET Web API for fast authorization checks.

◆ 1.5 RECURSION & BACKTRACKING (Q101–Q120)

(Frequently asked in Infosys, Accenture, Amazon, Microsoft, Deloitte)

RECURSION

Q101. (TCS / Infosys – Screening Round)

What is recursion in simple words?

Explanation

Recursion is when a function **calls itself** to solve a smaller version of the same problem. It keeps breaking the problem into simpler parts until it becomes easy enough to solve.

But recursion must always have a **base condition**, otherwise the function keeps calling itself forever and crashes the application.

Real-time C# example

Printing numbers from 1 to N:

```
void PrintNumbers(int n)
{
    if (n == 0) return;        // base condition

    PrintNumbers(n - 1);
}
```

```
        Console.WriteLine(n);  
    }  
}
```

Q102. (Accenture – Technical Round)

Why is base condition important in recursion?

Explanation

Base condition is the **stop signal** for recursion.
It tells the function — “Do not call yourself anymore.”

Without base condition:

- Recursion never stops
- Stack memory keeps filling
- Application throws **StackOverflowException**

Real-time example

An API recursively calling another API without stop condition → system crash.

Q103. (Wipro – Fresher Round)

Factorial using recursion

Explanation

Factorial is naturally recursive because:

```
factorial(n) = n × factorial(n-1)
```

C# example

```
int Factorial(int n)
```



```
{  
  
    if (n == 1) return 1;  
  
    return n * Factorial(n - 1);  
  
}
```

This keeps breaking the problem into smaller parts until n becomes 1.

Q104. (Microsoft – Technical Round)

What happens in memory during recursion?

Explanation

Every recursive call is stored in the **call stack**.

For each call, stack stores:

- function parameters
- local variables
- return address

More recursive depth = more stack memory consumption.

Real-time example

Recursive JSON parsing

Deep folder scanning

Q105. (Amazon – Interview Favorite)

Why can recursion cause StackOverflowException?

Explanation

Stack memory is limited.

When too many recursive calls are created without stopping, the stack gets full.

Result → Application crash.

Happens due to:

- Missing base condition
 - Deep recursion
 - Very large input
-

Q106. (Accenture – Developer Round)

Recursion vs Loop – which is better?

Explanation

Recursion

- Cleaner logic
- Easier for tree/graph problems
- More readable

Loop

- Better performance
- Less memory usage
- No stack overflow risk

Real-time decision

Use recursion for:

- Tree traversal
- Complex hierarchical problems

Use loops for:

- Large data
 - Performance critical systems
-

Q107. (Infosys – Technical Round)

Convert recursion to iteration – why?

Explanation

Sometimes recursion is replaced by loops to:

- Reduce stack usage
- Prevent crashes
- Improve performance

Real-time example

Replacing recursive file processing with an iterative method in the production system to avoid crashes.

Q108. (Microsoft – Advanced Round)

What is tail recursion?

Explanation

Tail recursion means the **recursive call is the last statement** in the function. Nothing remains to execute after the call returns.

C# example

```
int Sum(int n, int total)
{
    if (n == 0) return total;

    return Sum(n - 1, total + n);
}
```

}

Tail recursion is memory efficient compared to normal recursion.

Q109. (Amazon – DSA Round)

Why is recursive Fibonacci inefficient?

Explanation

Recursive Fibonacci recalculates same values again and again.
This leads to exponential time complexity.

So:

- Slow
- High CPU usage
- Not suitable for real systems

Real-time example

Avoided in financial systems or performance-sensitive applications.

Q110. (Deloitte – Senior Round)

When should recursion be avoided?

Explanation

Avoid recursion when:

- Input size is very large
- Performance is critical
- Memory constraints exist
- Depth of recursion is unpredictable

Production systems prefer iterative approaches for stability.



BACKTRACKING

Q111. (Amazon – DSA Round)

What is backtracking in simple terms?

Explanation

Backtracking means:

- 1 Try a solution
- 2 If it doesn't work → go back
- 3 Try another path

It explores possible solutions and removes wrong ones.

Simple thinking:

“Try → Fail → Undo → Try another way”

Q112. (Microsoft – Technical Round)

Where is backtracking used?

Explanation

Backtracking is used where multiple possibilities exist.

Real-time examples

- Sudoku solving
- N-Queens problem
- Path finding in maze
- Puzzle and board games
- Decision engines

Q113. (Accenture – Interview Favorite)

Difference between recursion and backtracking

Explanation

Recursion

Technique → Function calling itself.

Backtracking

Strategy → Uses recursion to explore multiple options, and undo wrong paths.

So:

Backtracking = Recursion + Decision + Undo

Q114. (Amazon – Coding Round)

Why backtracking is slow sometimes?

Explanation

Backtracking tries many combinations.

Sometimes number of possibilities grows exponentially.

So:

- More time
- More computation
- Heavy CPU usage

That's why optimization and pruning are important.

Q115. (Infosys – Technical Round)

Solve simple backtracking example – permutations

C# example

```
void Permute(string s, string ans)
```

```

{
    if (s.Length == 0)
    {
        Console.WriteLine(ans);
        return;
    }

    for (int i = 0; i < s.Length; i++)
    {
        Permute(s.Remove(i, 1), ans + s[i]);
    }
}

```

Generates all possible arrangements of characters.

Q116. (Microsoft – Advanced Round)

How does backtracking reduces unnecessary work?

Explanation

Backtracking stops exploring a path **as soon as it becomes invalid**.
This is called **pruning**.

It prevents:

- Wasting time
- Exploring guaranteed wrong paths

Result → Faster execution.

Q117. (Amazon – Scenario Based)

Maze solving uses which technique?

Answer

Maze solving is done using:

- Backtracking
- Recursion

We try paths, and when blocked, we go back and try a new direction.

Q118. (Accenture – Conceptual)

Why do backtracking problems use recursion naturally?

Explanation

Because these problems:

- Break into smaller decisions
- Need exploration
- Require undoing choices

Recursion naturally supports this flow.

Q119. (Deloitte – Senior Round)

Backtracking vs Dynamic Programming

Explanation

Backtracking

- Tries all paths
- No memory reuse

- Can be expensive

Dynamic Programming

- Stores previous results
- Avoids recomputation
- Faster in repeated subproblems

Q120. (Amazon – Final Round)

Real-world use of backtracking

Sample interview answer

“Backtracking is widely used in scheduling engines, constraint-solving systems, AI decision engines, recommendation engines, and optimization problems where multiple solutions are evaluated and invalid paths are rolled back efficiently.”

◆ 1.6 TREES (Q121–Q140)

(Very frequently asked in Amazon, Microsoft, Infosys, Accenture, Deloitte)



TREE BASICS

Q121. (Infosys – Technical Round)

What is a Tree data structure?

Explanation

A Tree is a data structure used to represent **hierarchical data**. It is like a real life “family tree” where:

- There is a single root (top-most node)

- Each node has **one parent**
- Each node may have **zero or more children**

A tree helps represent relationships that are not linear.

Real-time C# example

Organization hierarchy:

```
class Employee
{
    public string Name;
    public List<Employee> Subordinates = new();
}
```

Perfect for:

- Company structure
- Menu navigation
- Folder directory

Q122. (TCS – Screening Round)

Why are trees preferred over arrays for hierarchy?

Explanation

Arrays represent **linear** data.

But hierarchy is **not linear**, it has levels and relationships.

Trees are naturally built to handle:

- Parent → Child relationships
- Multiple levels

- Expanding structures

Real-time example

Folder structure in Windows or Linux file system.

Folder

→ Subfolder

→ Subfolder inside it

This cannot be represented cleanly using just arrays.

Q123. (Microsoft – Technical Round)

What is a Binary Tree?

Explanation

A Binary Tree is a tree where **each node can have at most two children**:

- Left child
- Right child

It is one of the most common and powerful tree structures.

Real-time C# example

```
class Node
{
    public int Data;
    public Node Left, Right;
}
```

Used in:

- Expression evaluation

- Decision trees
 - Search operations
-

Q124. (Amazon – Interview Favorite)

Binary Tree vs Binary Search Tree (BST)

Explanation

Binary Tree

Just a tree with two children.

No ordering rule.

Binary Search Tree (BST)

It maintains sorted structure:

Left < Root < Right

This property makes searching efficient.

Real-time C# example

```
if (value < root.Data)
    root.Left = Insert(root.Left, value);
```

BST helps in:

- Fast searching
 - Range queries
 - Ordered data storage
-

Q125. (Accenture – Developer Round)

Why is BST search faster than array search?

Explanation

In BST, at each step half of the search space is eliminated.

So search becomes:

- $O(\log n)$ in balanced BST
- Much faster than $O(n)$ linear search in arrays

Real-time usage

Searching users by ID

Searching product in sorted catalog

TREE TRAVERSALS

Q126. (Amazon – Coding Round)

What is Inorder traversal and where is it used?

Explanation

Inorder traversal visits nodes:

Left → Root → Right

In Binary Search Tree, this gives output in **sorted order**, which is extremely powerful.

C# example

```
void InOrder(Node root)
{
    if (root == null) return;

    InOrder(root.Left);

    Console.Write(root.Data + " ");

    InOrder(root.Right);
}
```

Used in:

- Getting sorted data
 - Validating BST
-

Q127. (Microsoft – Interview Favorite)

Preorder traversal – real use case

Explanation

Preorder order:

Root → Left → Right

It is useful when we want to:

- Copy the tree
- Serialize (store) the tree
- Re-build structure later

C# example

```
void PreOrder(Node root)
{
    if (root == null) return;

    Console.Write(root.Data + " ");

    PreOrder(root.Left);

    PreOrder(root.Right);
}
```

Q128. (Infosys – Technical Round)

Postorder traversal – where is it used?

Explanation

Postorder:

Left → Right → Root

This is used when children must be processed **before parents**.

Practical Uses

- Deleting trees safely
- Memory cleanup
- Expression evaluation

C# example

```
void PostOrder(Node root)
{
    if (root == null) return;

    PostOrder(root.Left);

    PostOrder(root.Right);

    Console.Write(root.Data + " ");
}
```

Q129. (Amazon – DSA Round)

What is Level Order traversal?

Explanation

Level Order means:

Visit nodes **level by level** from top to bottom.

This uses **Queue**, so it is also called BFS for trees.

Real-time C# example

```
void LevelOrder(Node root)
{
    Queue<Node> q = new Queue<Node>();
    q.Enqueue(root);

    while (q.Count > 0)
    {
        Node node = q.Dequeue();
        Console.Write(node.Data + " ");

        if (node.Left != null) q.Enqueue(node.Left);
        if (node.Right != null) q.Enqueue(node.Right);
    }
}
```

Used in:

- Finding shortest path
- Processing tree in layers

Q130. (Microsoft – Conceptual Round)

Why does BFS use Queue and DFS uses Stack?

Explanation

BFS → visits level by level → needs FIFO → Queue
DFS → goes deep → needs LIFO → Stack / Recursion

Very natural mapping.

TREE PROBLEMS

Q131. (Amazon – Coding Round)

Find height of a Binary Tree

Explanation

Height means:
Longest path from root to leaf.

If no node → height 0
If one node → height 1

C# example

```
int Height(Node root)
{
    if (root == null) return 0;

    return Math.Max(Height(root.Left), Height(root.Right)) + 1;
}
```

Q132. (Microsoft – Advanced Round)

Why do deep trees cause stack overflow?

Explanation

Tree traversal mostly uses recursion.
Deep tree → deep recursion
Deep recursion → consumes stack memory

Eventually → StackOverflowException

Real-world example

Parsing deeply nested JSON or XML

Q133. (Accenture – Developer Round)

Check if two trees are identical

```
bool IsSame(Node a, Node b)
{
    if (a == null && b == null) return true;
    if (a == null || b == null) return false;

    return a.Data == b.Data &&
           IsSame(a.Left, b.Left) &&
           IsSame(a.Right, b.Right);
}
```

Checks data + structure both.

Q134. (Amazon – Interview Favorite)

Check if tree is balanced

Explanation

A tree is balanced if:
Difference between left height and right height ≤ 1
for every node.

Balanced tree = better performance.

Q135. (Deloitte – Senior Round)

Why are skewed trees bad?

Explanation

When tree becomes skewed:

- All nodes go in one side
- Tree acts like a linked list
- Searching becomes $O(n)$

Bad for performance.

REAL PROJECT SCENARIOS

Q136. (Microsoft – System Design)

Where trees are used in file systems?

The file system is a perfect tree example.

```
class Folder
{
    public string Name;

    public List<Folder> SubFolders = new();
}
```

Root Folder
→ Subfolders
→ Files & deeper folders

Q137. (Amazon – Architecture Round)

Tree vs Graph – key difference

Explanation

Tree

- No cycles
- One clear root
- Strict parent-child relationship

Graph

- May have cycles
 - No single root required
 - More flexible connections
-

Q138. (Accenture – Interview Favorite)

Why is recursion natural for tree traversal?

Explanation

Trees themselves are recursive in nature:
A tree is made of smaller trees.

So recursion naturally fits:

- Break problem
- Solve smaller part

- Combine back

This makes code clean and readable.

Q139. (Infosys – Technical Round)

Tree vs Dictionary – when is a tree better?

Explanation

Use Tree when:

- Hierarchy matters
- Order matters
- Parent-child relationships required

Dictionary is great for key-based fast lookup

Tree is great for structure + relationships

Q140. (Amazon – Final DSA Round)

One real production use of trees

Sample interview answer

“We used a tree structure to design Role-Based Access Control (RBAC) in a .NET enterprise application where every role had child roles, permissions were inherited, and access was evaluated hierarchically.”

◆ 1.7 GRAPHS (Q141–Q160)

(Highly asked in Amazon, Microsoft, Deloitte, Accenture – Mid to Senior Level)

GRAPH BASICS

Q141. (Infosys – Technical Round)

What is a Graph in simple terms?

Explanation

A Graph is a data structure that represents **connections or relationships** between objects. It has:

- **Nodes (Vertices)** → represent entities
- **Edges** → represent relationships between them

Graphs are powerful because they model **real-world networks**, which are not strictly hierarchical.

Real-time C# example

Social network connections:

```
Dictionary<int, List<int>> graph = new();  
  
graph[1] = new List<int> { 2, 3 };  
  
graph[2] = new List<int> { 1 };
```

Used in:

- Facebook friend connections
- LinkedIn network graph
- Google Maps routes

Q142. (TCS – Screening Round)

Difference between Tree and Graph

Explanation

Tree	Graph
No cycles	Cycles allowed
One parent per node	Multiple connections possible
Hierarchical	Network-based
One root	No fixed root required

Real-world example

Tree → Organization chart

Graph → Social networking system

Q143. (Microsoft – Technical Round)

What is an adjacency list?

Explanation

Adjacency List stores:

- Each node
- Along with a list of its connected neighbors

It is **memory efficient**, especially for large sparse graphs.

Real-time C# example

```
Dictionary<int, List<int>> adj = new();
```

```
adj[0] = new List<int> {1, 2};
```

Used in:

- Routing systems
 - Web crawlers
 - Graph-based search engines
-

Q144. (Amazon – Interview Favorite)

Adjacency List vs Adjacency Matrix

Explanation

Adjacency List

- Stores only actual connections
- Saves memory
- Best for sparse graphs

Adjacency Matrix

- Uses $N \times N$ matrix
- Faster edge lookup
- Uses more memory

Interview tip

Say this:

“For large graphs, an adjacency list is preferred because it avoids storing unnecessary edges.”



GRAPH TRAVERSAL

Q145. (Amazon – Coding Round)

What is BFS and where is it used?

Explanation

Breadth-First Search (BFS):

- Explores graph **level by level**
- Uses **Queue**
- Ensures closest neighbors are visited first

Real-time C# example

```
void BFS(Dictionary<int, List<int>> graph, int start)
{
    Queue<int> q = new();
    HashSet<int> visited = new();

    q.Enqueue(start);
    visited.Add(start);

    while (q.Count > 0)
    {
        int node = q.Dequeue();
        Console.Write(node + " ");

        foreach (int neighbor in graph[node])
        {
            if (!visited.Contains(neighbor))
            {
```

```
        visited.Add(neighbor);  
        q.Enqueue(neighbor);  
    }  
}  
}
```

Real-world use

- Finding shortest path in maps
- Finding nearest connections
- Social media friend suggestions

Q146. (Microsoft – Interview Favorite)

Why does BFS use Queue?

Explanation

Queue works on **FIFO (First In, First Out)**.

BFS needs to explore nearest nodes first → exactly what Queue does.

So:

BFS → Queue

DFS → Stack

Q147. (Accenture – Developer Round)

What is DFS and when to use it?

Explanation

Depth-First Search:

- Goes as deep as possible
- Uses Stack or Recursion
- Explores paths fully before backtracking

Real-time C# example

```
void DFS(Dictionary<int, List<int>> graph, int node, HashSet<int>
visited)
{
    visited.Add(node);

    Console.Write(node + " ");

    foreach (int n in graph[node])
        if (!visited.Contains(n))
            DFS(graph, n, visited);
}
```

When to use?

- Path finding
- Detecting cycles
- Topological sorting

Q148. (Amazon – DSA Round)

Why does DFS use Stack or Recursion?

Explanation

DFS follows:

- Last explored node should be revisited first → **LIFO**
- Stack naturally supports LIFO
- Recursion internally uses stack

So DFS and Stack are a natural match.

GRAPH APPLICATIONS

Q149. (Microsoft – System Design)

Detecting cycles in a graph – which traversal?

Explanation

DFS is commonly used to detect cycles because:

- It traces paths deeply
- Helps identify back edges

Real-time example

- Detecting circular dependencies
 - Preventing infinite workflows
 - Package and module dependency checks
-

Q150. (Amazon – Interview Favorite)

Graph vs Tree – which one to use in routing?

Explanation

Routing networks:

- Have multiple routes
- Have loops/cycles
- Need multiple paths

So → Use Graph

Trees cannot represent such complex routing structures.



SHORTEST PATH & CONNECTIVITY

Q151. (Accenture – Advanced Round)

What is the shortest path problem?

Explanation

Finding the **minimum cost / distance / steps** between two nodes in a graph.

Real-world example

- Google Maps shortest route
 - Logistics and delivery routes
 - Network routing systems
-

Q152. (Microsoft – Technical Round)

Why does BFS give the shortest path in an unweighted graph?

Explanation

Because BFS visits nodes **level by level**, meaning:

- First time you reach a node

- It is always via shortest path

So BFS guarantees the shortest path in unweighted graphs.

Q153. (Amazon – Coding Round)

Check if graph is connected

Explanation

Run BFS or DFS from any node.

If all nodes get visited → graph is connected.

C# example

```
bool IsConnected(Dictionary<int, List<int>> graph, int start)
{
    HashSet<int> visited = new();

    DFS(graph, start, visited);

    return visited.Count == graph.Count;
}
```

Q154. (Deloitte – Senior Round)

Difference between directed and undirected graph

Explanation

Directed Graph

- One-way direction
- Edges have arrows

Undirected Graph

- Two-way connection
 - No direction
-

Q155. (Infosys – Technical Round)

Real-world example of directed graph

Explanation

- Workflow and approval systems
- Task scheduling
- Payment transaction flows

Direction matters in these systems.

ADVANCED GRAPH THINKING

Q156. (Amazon – Architecture Round)

Why are graphs used in recommendation systems?

Explanation

Recommendation systems are built on relationships:

- User → interacts with → Product
- Product → related to → Other products

This forms a graph.

Graph traversal helps suggest related or nearest relationship items.

Examples:

- Amazon “Customers also bought”

- Netflix “Because you watched...”
-

Q157. (Microsoft – Advanced Round)

What is topological sorting used for?

Explanation

Topological sort gives a **valid execution order** of tasks that depend on each other.

Used in:

- Build pipelines
 - Task scheduling
 - Dependency management
-

Q158. (Accenture – Developer Round)

Why topological sort works only for DAG?

Explanation

Topological sort requires:

- No cycles
- Clear order

If cycles exist:

- No valid order
- Execution becomes impossible

So it only works for **Directed Acyclic Graphs (DAG)**.

Q159. (Amazon – Scenario Based)

Detecting deadlock in the system – which DS?

Explanation

Deadlock occurs when processes wait in a cycle.

So:

Represent processes as graph

If cycle exists → deadlock exists

Graph + Cycle detection solves deadlocks.

Q160. (Microsoft – Final Round)

One real project where you used graph

Sample interview answer

“We modeled microservice dependencies as a graph in our .NET application. This helped us detect circular service calls, identify failure impact areas, and prevent cascading failures in production.”

◆ 1.8 ADVANCED DSA SCENARIOS (Q161–Q200)

(Real Production + System-Level Interview Questions)



REAL-WORLD DSA SCENARIOS

Q161. (Amazon – Scenario Round)

API response is slow. Which DS will you use to cache data? Why?

Explanation

Use **Dictionary (HashMap)** because it provides **O(1) lookup**, meaning data can be fetched instantly without recomputation or repeated API/database calls.

It improves:

- Performance
- Latency
- User experience

C# example

```
Dictionary<int, string> cache = new();  
cache[101] = "UserData";
```

Used in:

- Response caching
- User profile lookup
- Product information caching

Q162. (Microsoft – System Design)

Design LRU Cache – which data structures are used?

Explanation

Use combination of:

- **Dictionary** → $O(1)$ lookup
- **Doubly Linked List** → maintain usage order

Most recently used stays in front, least recently used at back.

Why interviewer asks

To check:

- System design maturity
- Understanding of DS pairing

- Real-world performance thinking
-

Q163. (Amazon – Interview Favorite)

Why LRU Cache cannot be implemented with only List?

Explanation

List operations are costly:

- Searching $\rightarrow O(n)$
- Removing from middle $\rightarrow O(n)$

Cache must be **$O(1)$** , not $O(n)$, hence List alone is inefficient.

Q164. (Accenture – Backend Scenario)

How to find the most frequent element in a large dataset?

Explanation

Use a Dictionary **to count frequency**, then pick the highest count.
Efficient, scalable, and memory friendly.

C# example

```
var result = freq.OrderByDescending(x => x.Value).First();
```

Used in:

- Analytics
 - User behavior tracking
 - Trending feature detection
-

Q165. (Deloitte – Senior Round)

User session tracking – which DS fits best?

Explanation

Use Dictionary

Key → SessionId

Value → SessionData

Gives fast retrieval and effective session management.



MULTI-THREADING & PERFORMANCE SCENARIOS

Q166. (Microsoft – Advanced)

Why does the Dictionary fail in a multi-threaded environment?

Explanation

Multiple threads updating dictionary at same time cause:

- Race conditions
- Corruption
- Unexpected behavior

Because Dictionaries are **not thread-safe**.

Q167. (Amazon – Architecture Round)

Fix thread-safety issue in shared cache

C# example

```
ConcurrentDictionary<int,string> cache =  
    new ConcurrentDictionary<int,string>();
```

This prevents collisions and ensures safe parallel access.

Q168. (Accenture – Scenario)

How to process millions of records efficiently?

Explanation

- Avoid recursion (stack overflow risk)
- Use streaming (process in chunks)
- Use efficient DS like **Span**, **Hashing**, **Queues**

Goal → Reduce memory + improve throughput.

ALGORITHM THINKING

Q169. (Amazon – DSA Round)

Why does time complexity matter in real systems?

Explanation

Poor complexity = poor scalability.

More users + more data = system slows, costs increase, customer experience suffers.

Better complexity = cost saving + better performance.

Q170. (Microsoft – Interview Favorite)

Reduce $O(n^2)$ code to $O(n)$

Explanation

Replace nested loops with **HashSet** / **Dictionary** to achieve constant time lookup.

C# example

```
HashSet<int> set = new(arr);
```

PRODUCTION BUG SCENARIOS

Q171. (Accenture – Production Issue)

Application crashes with StackOverflowException – why?

Explanation

- Deep recursion
- Missing base condition
- Infinite chain of method calls

Stack memory gets exhausted → crash.

Q172. (Amazon – Debugging Round)

Memory leak in application – DS related cause?

Explanation

Keeping objects in:

- Static lists
- Dictionaries
- Caches

Without clearing → memory leak.

Q173. (Microsoft – Advanced Debugging)

Why does List grow memory unexpectedly?

Explanation

List automatically increases capacity (usually doubles) when needed.
If not managed smartly, memory consumption grows rapidly.

DESIGN THINKING

Q174. (Deloitte – Design Round)

Search feature for large data – which DS?

Explanation

- Hashing → fast exact search
- Trees → range search + sorted operations

Right DS = better user experience.

Q175. (Amazon – Architecture)

Why does the DB index use B-Trees instead of arrays?

Explanation

B-Trees:

- Stay balanced
- Reduce disk reads
- Support fast range queries

Very efficient for huge data storage systems.

INTERVIEW TRAP QUESTIONS

Q176. (Microsoft – Tricky)

Why is Binary Search not always best?

Explanation

Binary Search requires:

- Sorted data
- Random access structure

Sorting itself may be expensive.
So binary search is not always ideal.

Q177. (Accenture – Interview Trap)

Why is Linked List rarely used in APIs?

Explanation

- Poor cache locality
- Slow traversal
- Higher memory overhead

APIs need speed and predictability → arrays & lists preferred.

OPTIMIZATION & DECISION MAKING

Q178. (Amazon – Senior Round)

How to choose the right DS for a project?

Explanation

Consider:

- Access pattern
- Data size
- Concurrency
- Performance need
- Memory limitation

Right DS = right performance.

Q179. (Microsoft – Advanced)

Array vs Span – when to use Span?

Explanation

Use **Span** when:

- High performance needed
- Avoiding extra allocations
- Working with slices of data

Span reduces memory pressure.



CODING DECISION SCENARIOS

Q180. (Accenture – Coding Decision)

Sort in code or DB?

Explanation

For large datasets → Always sort in DB
DB engines are optimized for sorting + indexing.

SYSTEM LEVEL DSA

Q181. (Amazon – System Design)

Design rate limiter – which DS?

Explanation

Use:

- Queue or Sliding Window
- Store timestamps

Ensures fair request control.

Q182. (Microsoft – Advanced System)

Why is Queue used in message brokers?

Explanation

Queue ensures FIFO

- Fairness
- Order preservation
- Reliable processing

Used in Kafka, RabbitMQ, Azure Service Bus.



REAL INTERVIEW EXPERIENCE QUESTIONS

Q183. (Deloitte – Senior Interview)

Explain one DS you misused and learned from

Sample answer

“I initially used List instead of Dictionary for lookups and performance became poor. Replacing it with a Dictionary reduced response time significantly.”

Q184. (Amazon – Leadership + Tech)

Trade-off between memory and speed

Explanation

Faster systems often consume more memory, and memory-optimized systems may sacrifice speed.

Choosing depends on business needs.



SECURITY & DATA HANDLING

Q185. (Microsoft – Security Round)

Why is hashing important in security?

Explanation

Hashing protects sensitive data like passwords by converting them into irreversible values.

Q186. (Accenture – Scenario)

Prevent duplicate API requests – which DS?

Explanation

Use **HashSet** to track processed requests and prevent reprocessing.



SCALABILITY QUESTIONS

Q187. (Amazon – Scale Thinking)

Why does $O(n^2)$ fail at scale?

Explanation

As data grows, execution time increases dramatically.
Unacceptable for millions of users.

Q188. (Microsoft – Advanced)

DS used in load balancers

Explanation

Queue → request handling
Hashing → server routing



EDGE CASE HANDLING

Q189. (Accenture – Debugging)

Null reference issue in DS usage

Explanation

Occurs when accessing DS element without checking:

- Null
- Key existence

- Boundary checks
-

Q190. (Amazon – Coding)

Avoid duplicate processing

C# example

```
HashSet<int> processedIds = new();
```

FINAL INTERVIEW QUESTIONS

Q191. (Microsoft – Final Round)

Explain DSA choice in one of your projects

Q192. (Amazon – Final Round)

Which DS do you use most and why?

Q193. (Deloitte – Final Tech)

DS mistake juniors commonly make

Q194. (Accenture – Final HR + Tech)

How DSA knowledge improved your code quality

Q195. (Amazon – Architecture)

Why does DS choice impact cloud cost?

Q196. (Microsoft – Performance)

Memory vs speed trade-off example

Q197. (Deloitte – Leadership)

How do you explain DSA to juniors?

Q198. (Accenture – Real Project)

Biggest performance issue you fixed using DSA

Q199. (Amazon – Reflection)

One DSA topic you struggled with and mastered

Q200. (ALL MNCs – FINAL DSA QUESTION)

Why is DSA still important for Full-Stack Developers?

Best Interview Answer

“Because it directly affects performance, scalability, cloud cost, and user experience. A developer who understands DSA writes faster, more reliable, and more efficient production systems.”

CHAPTER 2 – OBJECT-ORIENTED PROGRAMMING (OOP)



OOP BASICS (Q201–Q220)

Q201. (TCS / Infosys – Screening Round)

What is Object-Oriented Programming in simple terms?

Explanation

OOP is a way of writing code by **thinking in terms of real-world objects**.

Each object represents something meaningful and contains:

- **Data (properties)** – what it has
- **Behavior (methods)** – what it does

This makes code:

- easier to understand
- easier to maintain
- closer to real-world thinking

Real-time C# example

Employee in HR system:

```
class Employee
{
    public int Id;
    public string Name;

    public void Work()
    {
        Console.WriteLine($"{Name} is working");
    }
}
```

}

Q202. (Accenture – Technical Round)

What is a Class and what is an Object?

Explanation

- **Class** → Blueprint / Template
- **Object** → Actual working instance created from class

Class defines structure; object represents a real thing in memory.

Real-time C# example

```
Employee emp = new Employee(); // object  
  
emp.Name = "John";  
  
emp.Work();
```

Interview tip

Say confidently:

“Class is logical, object is physical.”

Q203. (Wipro – Fresher Round)

Why do we need OOP instead of procedural programming?

Explanation

Procedural programming focuses only on **functions**, which becomes messy as projects grow.

OOP helps because it supports:

- Reusability
- Maintainability
- Scalability
- Better team collaboration

It allows modeling real business systems cleanly.

Real project example

Large enterprise apps like ERP, Banking, E-commerce cannot survive on functions alone. They need structured object models.

Q204. (Microsoft – Technical Round)

What are the four pillars of OOP?

Explanation

The four pillars are:

- 1 Encapsulation
- 2 Inheritance
- 3 Polymorphism
- 4 Abstraction

Each solves real coding and design problems.

Interview tip

Do not just name them. Be ready to explain at least one with an example.

ENCAPSULATION

Q205. (Infosys – Interview Favorite)

What is Encapsulation and why is it important?

Explanation

Encapsulation means **wrapping data and behavior together** and preventing direct access from outside.

It ensures:

- Data protection
- Controlled access
- No accidental modification

You expose only what is necessary and hide critical details.

Real-time C# example

```
class BankAccount
{
    private decimal balance;

    public void Deposit(decimal amount)
    {
        if (amount > 0)
            balance += amount;
    }

    public decimal GetBalance()
    {
        return balance;
    }
}
```

Real project usage

Banking, payroll, payment gateways — where data safety is critical.

Q206. (Accenture – Developer Round)

Why should fields be private in real projects?

Explanation

If fields are public:

- Anyone can modify them
- System may enter invalid state
- Data integrity breaks

Private fields ensure only valid operations can change data.

Interview tip

Use phrase: “**Encapsulation maintains data integrity.**”

Q207. (Amazon – Interview Favorite)

Difference between Encapsulation and Abstraction

Explanation

Encapsulation	Abstraction
Hides data	Hides complexity
Protects internal state	Shows only essential features

Uses access
modifiers

Uses interfaces / abstract
classes

INHERITANCE

Q208. (TCS – Technical Round)

What is Inheritance?

Explanation

Inheritance allows one class to **reuse** properties and behaviors of another class. This removes duplication and improves organization.

Real-time C# example

```
class Employee
{
    public string Name;
}

class Manager : Employee
{
    public void ApproveLeave() { }
}
```

Real project usage

Role-based systems, HR modules, product module hierarchies.

Q209. (Infosys – Technical Round)

Why inheritance is used in real applications?

Explanation

Inheritance helps to:

- Reuse functionality
- Reduce duplication
- Maintain consistency
- Extend behavior easily

This leads to cleaner and more scalable architecture.

Q210. (Microsoft – Advanced Round)

Problems with excessive inheritance

Explanation

Too much inheritance creates:

- Tight coupling
- Rigid design
- Difficult maintenance
- Fragile code

Modern design prefers:

👉 **Composition over inheritance**

Say this and the interviewer will smile.

POLYMORPHISM

Q211. (Accenture – Interview Favorite)

What is Polymorphism?

Explanation

Polymorphism means the same **method name but different behavior** depending on the object.

Real-time C# example

```
class Employee

{

    public virtual void CalculateSalary()

    {

        Console.WriteLine("Base salary");

    }

}

class ContractEmployee : Employee

{

    public override void CalculateSalary()

    {

        Console.WriteLine("Hourly salary");

    }

}
```

Q212. (Microsoft – Technical Round)

Compile-time vs Runtime Polymorphism

Explanation

Compile-time Polymorphism

- Method Overloading
- Decided during compile time

Runtime Polymorphism

- Method Overriding
 - Decided during runtime
-

Q213. (Amazon – Coding Round)

Why does method overriding need **virtual** and **override**?

Explanation

They tell CLR:

- **virtual** → method can be overridden
- **override** → method is overriding base version

This ensures correct runtime method execution.

ABSTRACTION

Q214. (Infosys – Technical Round)

What is Abstraction in simple words?

Explanation

Abstraction means:

“Show only what is necessary and hide complex internal logic.”

Real-life example

ATM machine:

You only press buttons — you don't know the banking algorithm behind it.

Q215. (Accenture – Developer Round)

Interface vs Abstract Class

Explanation

Interface	Abstract Class
Only contract	Partial implementation allowed
Multiple inheritance supported	Only single inheritance
Focus on capability	Focus on shared behavior

Q216. (Microsoft – Interview Favorite)

Why are interfaces heavily used in .NET projects?

Explanation

Because they help in:

- Loose coupling
- Dependency Injection
- Unit testing
- Clean architecture

Real-time C# example

```
interface IPaymentService  
{
```



```
void Pay();  
}
```

REAL PROJECT THINKING

Q217. (Deloitte – Scenario Based)

Why is OOP important in microservices?

Explanation

Each microservice models a business domain.
OOP helps:

- Clearly define business entities
- Keep responsibilities clean
- Maintain services easily

Q218. (Amazon – Architecture Round)

Encapsulation vs Security – relation?

Explanation

Encapsulation enforces **data protection at code level**.
You decide who can access what — reducing security risks.

Q219. (Microsoft – Senior Round)

One OOP mistake junior developers make

Explanation

They make everything:

- Public
- Non-structured
- Highly coupled

Leading to fragile systems.

Q220. (ALL MNCs – Interview Favorite)

How does OOP improve code maintainability?

Best Interview Answer

“OOP organizes code around meaningful objects, reduces duplication, localizes changes, improves reusability, and makes complex systems easier to understand, extend, and maintain.”

◆ **Advanced OOP Concepts (Q221–Q240)**

(Frequently asked in Accenture, Deloitte, Amazon, Microsoft, Infosys)



ACCESS MODIFIERS & DESIGN CHOICES

Q221. (Infosys – Technical Round)

What are access modifiers in C# and why do they matter?

Explanation

Access modifiers define **who can access a class member**.

They provide control, protect sensitive data, and enforce encapsulation.

They help:

- Prevent misuse of data
- Control visibility
- Maintain clean design boundaries

Real-time C# example

```
class User
{
    public string Name;      // accessible everywhere
    private string Password; // accessible only inside class
}
```

Real project usage

Protecting sensitive data like passwords, API keys, tokens, etc.

Q222. (Accenture – Interview Favorite)

Difference between private, protected, and public

Explanation

- **private** → accessible only inside same class
- **protected** → accessible in class + derived classes
- **public** → accessible everywhere

C# example

```
class Base
{
```

```
protected int id;  
}
```

These define how strictly data should be controlled.

Q223. (Microsoft – Technical Round)

Why should fields not be public in real projects?

Explanation

Public fields allow:

- Direct modification
- Invalid states
- Breaking business rules

This destroys encapsulation and makes the system unstable.

Real example

If balance is public:

```
account.balance = -500;
```

This can break financial systems.

Private fields + controlled access = data integrity.

CONSTRUCTORS & OBJECT LIFECYCLE

Q224. (TCS – Screening Round)

What is a constructor and why is it used?

Explanation

A constructor is a special method used to **initialize object state** when the object is created. It ensures the object always starts in a valid state.

Real-time C# example

```
class Employee
{
    public string Name;

    public Employee(string name)
    {
        Name = name;
    }
}
```

Q225. (Infosys – Technical Round)

Can a class have multiple constructors?

Explanation

Yes.

This is called **constructor overloading**, allowing objects to be created in different ways.

C# example

```
class User
{
    public User() { }

    public User(string name) { }
```

```
}
```

Useful when objects require flexible initialization.

Q226. (Microsoft – Advanced Round)

What is constructor chaining?

Explanation

Constructor chaining means **one constructor calls another** to reuse initialization logic.

C# example

```
class Product
{
    public Product() : this("Default") { }
    public Product(string name) { }
}
```

Benefits:

- Avoids duplicate code
- Keeps initialization logic centralized

METHOD OVERRIDING & HIDING

Q227. (Accenture – Interview Favorite)

Difference between override and new keyword

Explanation

- **override** → enables runtime polymorphism (replaces base behavior)
- **new** → hides base method (not true polymorphism)

Real-time C# example

```
class Base
{
    public virtual void Show() { }
}

class Child : Base
{
    public override void Show() { }
}
```

Override → preferred in real systems.

Q228. (Microsoft – Technical Round)

Why method overriding needs runtime binding?

Explanation

Because when you call a method using base reference, the actual function depends on the real **object type**, not reference type.

This is what enables polymorphism.

Runtime decides the final method.



ABSTRACT CLASSES & INTERFACES (DEEP)

Q229. (Amazon – Interview Favorite)

Why are interfaces preferred over inheritance in large systems?

Explanation

Interfaces:

- Reduce tight coupling
- Support Dependency Injection
- Promote flexibility
- Allow multiple implementations

They make systems more maintainable and testable.

Real-time C# example

```
interface INotification
{
    void Send();
}
```

Q230. (Infosys – Technical Round)

Can an interface have implementation in C#?

Explanation

Yes.

From C# 8, interfaces can have **default implementations**.

C# example


```
interface ILogger
{
    void Log();
    void Info() => Console.WriteLine("Info");
}
```

Useful for backward compatibility.

Q231. (Microsoft – Advanced Round)

Abstract class vs Interface – real project decision

Explanation

- Use **Abstract Class**
When you want shared base behavior and common logic.
- Use **Interface**
When you only want contract and loose coupling.

Real decision depends on:

- Reusability
 - Flexibility
 - Architecture
-



COMPOSITION VS INHERITANCE

Q232. (Deloitte – Senior Round)

What is composition and why is it preferred?

Explanation

Composition means building a class **using other classes**, instead of inheriting.

Real-time C# example

```
class Engine { }

class Car
{
    private Engine engine = new Engine();
}
```

Benefits:

- Loose coupling
 - More flexibility
 - Better control
-

Q233. (Amazon – Architecture Round)

Why “prefer composition over inheritance”?

Explanation

Because composition:

- Reduces dependency
- Avoids deep inheritance problems
- Easier to modify
- Easier to test

Systems remain more maintainable.

POLYMORPHISM EDGE CASES

Q234. (Microsoft – Tricky Question)

Why can base class references hold derived objects?

Explanation

Because C# supports **runtime polymorphism**.
Base reference can point to any child object.

C# example

```
Employee emp = new Manager();
```

This allows flexible design.

Q235. (Accenture – Interview Trap)

Why method overloading is not polymorphism at runtime?

Explanation

Because overloading is resolved at **compile time**, not runtime.
So it is compile-time polymorphism only.

IMMUTABILITY & DESIGN

Q236. (Amazon – Advanced Round)

What is an immutable class and why use it?

Explanation

Immutable class is a class whose **state cannot change** after creation.

Benefits:

- Thread safe
- Predictable behavior
- Reduces bugs

Real-time C# example

```
class Config
{
    public string Url { get; }

    public Config(string url)
    {
        Url = url;
    }
}
```

Q237. (Microsoft – System Design)

Why string is immutable in C#?

Explanation

Because immutability provides:

- Thread safety

- Security (prevents tampering)
- Memory optimization via interning

That's why string is one of the safest and most optimized types in .NET.

REAL-WORLD OOP SCENARIOS

Q238. (Deloitte – Scenario Based)

How does OOP help in unit testing?

Explanation

OOP + Interfaces allow:

- Dependency Injection
- Mocking
- Isolated testing

This results in stable testable systems.

Q239. (Amazon – Senior Round)

Biggest OOP mistake in enterprise apps

Explanation

Over-engineering inheritance:

- Deep hierarchies
- Hard to maintain
- Breaks easily

Simple design > Complex design.

Q240. (ALL MNCs – FINAL OOP QUESTION)

How do you apply OOP principles in daily coding?

Best Interview Answer

“By modeling real entities as objects, keeping data protected, using interfaces for loose coupling, preferring composition, and designing reusable, maintainable code that can scale without breaking.”

SOLID Principles (Q241–Q260)

(Very frequently asked in Amazon, Microsoft, Deloitte, Accenture, Infosys)

S – SINGLE RESPONSIBILITY PRINCIPLE (SRP)

Q241. (Infosys – Technical Round)

What is the Single Responsibility Principle (SRP)?

Explanation

SRP says:

“A class should do **ONLY** one job and have only one reason to change.”

If a class is handling multiple responsibilities (business logic + email + logging etc.), then any change in one part may break others.

Keeping one purpose per class:

- simplifies maintenance
- makes code cleaner
- makes testing easier

Bad C# example (SRP violated)

```
class UserService
{
    public void CreateUser() { }
    public void SendEmail() { }
}
```

Correct C# example

```
class UserService
{
    public void CreateUser() { }
}

class EmailService
{
    public void SendEmail() { }
}
```

Real project benefit

- Independent changes
- Easier debugging
- Smooth enhancements

Q242. (Accenture – Interview Favorite)

What problems occur when SRP is violated?

Explanation

When SRP is violated, your class becomes “God Class”.

Problems:

- Hard to modify (too many responsibilities)
- High chance of bugs
- Difficult to test
- Tightly coupled logic
- Code becomes spaghetti over time

Real-world example

Changing email logic breaks user creation flow — production disaster.



O – OPEN / CLOSED PRINCIPLE (OCP)

Q243. (Microsoft – Technical Round)

Explain Open/Closed Principle

Explanation

Software entities should be:

- **Open for extension** (we can add new functionality)
- **Closed for modification** (we should not modify existing code)

This avoids risking existing working code when adding features.

Bad approach

```
if(type == "CreditCard") { }  
else if(type == "UPI") { }
```

Every new payment adds another IF → risky.

Correct C# example

```
interface IPayment
```



```
{  
    void Pay();  
}  
  
class CreditCardPayment : IPayment  
{  
    public void Pay() { }  
}
```

Real project usage

New payment methods can be added without touching old code.
Safer updates → fewer bugs.

Q244. (Amazon – Interview Favorite)

Why is OCP important in large systems?

Explanation

Because large systems:

- continuously evolve
- keep adding features
- must not break existing functionality

OCP ensures:

- Stability of running systems
- Easy feature addition
- Fewer regression bugs

It directly supports agile development and CI/CD pipelines.

L – LISKOV SUBSTITUTION PRINCIPLE (LSP)

Q245. (Infosys – Technical Round)

What is the Liskov Substitution Principle?

Explanation

A child class should be able to replace its parent **without changing program behavior**.

If a child breaks expectations → design is wrong.

LSP violation example

```
class Bird
{
    public virtual void Fly() { }
}

class Penguin : Bird
{
    public override void Fly()
    {
        throw new Exception("Cannot fly");
    }
}
```

Penguin IS a bird but CANNOT fly → this breaks logic.

Correct design

Separate abstract behavior:

- FlyingBird
- NonFlyingBird

Q246. (Microsoft – Advanced Round)

Why do LSP violations cause runtime bugs?

Explanation

Because:

- Calling code assumes parent behavior
- But child behaves differently (throws exception / changes rules)

This leads to:

- Crashes
- Unexpected behavior
- Logical failures

LSP ensures predictable and safe inheritance.

I – INTERFACE SEGREGATION PRINCIPLE (ISP)

Q247. (Accenture – Interview Favorite)

What is Interface Segregation Principle?

Explanation

Clients should **NOT be forced** to implement methods they do not need.

Small, focused interfaces are better than giant “God interfaces”.

Bad interface

```
interface IWorker
{
    void Work();
    void Eat();
}
```

Robots implementing Work shouldn't be forced to implement Eat.

Correct design

```
interface IWork { void Work(); }
interface IEat { void Eat(); }
```

Q248. (Amazon – Architecture Round)

Why does ISP improve flexibility?

Explanation

Because smaller interfaces:

- Are easier to implement
- Reduce impact of changes
- Avoid unnecessary dependencies
- Make code cleaner

ISP leads to better modular architecture.



D – DEPENDENCY INVERSION PRINCIPLE (DIP)

Q249. (Microsoft – Interview Favorite)

Explain Dependency Inversion Principle

Explanation

DIP says:

- 1 High-level modules should not depend on low-level modules
- 2 Both should depend on abstractions (interfaces)

This means:

Stop tightly coupling classes.

Depend on interfaces instead.

Bad example

```
class OrderService
{
    FileLogger logger = new FileLogger();
```

```
}
```

Hard to change or test.

Correct C# example

```
class OrderService
{
    private readonly ILogger _logger;

    public OrderService(ILogger logger)
    {
        _logger = logger;
    }
}
```

Now any logger can be injected.

Q250. (Infosys – Technical Round)

How does DIP help in unit testing?

Explanation

Because dependencies can be replaced with:

- Mock objects
- Fake implementations
- Test doubles

This allows independent testing without a real database, email, or payment systems.

DIP = easy testing + loose coupling.

SOLID AS A WHOLE

Q251. (Deloitte – Senior Round)

Which SOLID principle is most important in your experience?

Sample interview answer

“Dependency Inversion Principle, because it enables loose coupling, improves testability, supports Dependency Injection, and keeps large systems maintainable.”

Q252. (Amazon – Scenario Based)

Which SOLID principle helps most in microservices?

Explanation

SRP + DIP.

SRP → Each service has one clear responsibility

DIP → Services communicate via abstractions (APIs/interfaces) instead of tight coupling

This improves scaling and independence.

Q253. (Accenture – Interview Trap)

Can a class follow all SOLID principles always?

Explanation

No.

Software is practical, not theoretical.

Overdesign can:

- Increase complexity
- Reduce performance
- Waste time

Balance is key.

Q254. (Microsoft – Real Project)

How SOLID reduced bugs in your project?

Sample answer

“Applying SRP ensured each class handled one responsibility. So when we changed email logic, user features were not impacted. This drastically reduced side effects and production bugs.”

Q255. (Infosys – Technical Round)

Which SOLID principle is most violated by juniors?

Explanation

SRP.

Most juniors put:

- database logic
- validation
- API logic
- logging

in a single service class — leading to chaos.



CODING & DESIGN DECISIONS

Q256. (Amazon – Architecture Round)

SOLID vs performance – conflict?

Explanation

Yes, sometimes abstractions introduce slight overhead.
But maintainability usually matters more than micro-performance.

Rule:

Use SOLID wisely, not blindly.

Q257. (Deloitte – Senior Developer)

How does SOLID help in CI/CD pipelines?

Explanation

SOLID promotes:

- independent modules
- safer deployments
- isolated testing
- fewer breaking changes

This supports faster and reliable CI/CD.



FINAL SOLID QUESTIONS

Q258. (Microsoft – Final Round)

Explain SOLID in one line

“SOLID principles help build maintainable, extensible, and testable software that scales safely.”

Q259. (Accenture – HR + Tech)

How do you explain SOLID to a fresher?

“Write small focused classes, avoid tight coupling, depend on interfaces, and design code that is easy to change without breaking existing functionality.”

Q260. (ALL MNCs – FINAL SOLID QUESTION)

Is SOLID mandatory or optional?

Best Interview Answer

“SOLID is a powerful guideline, not a strict rulebook. Use it where it adds clarity, stability, scalability, and maintainability to the system.”

CHAPTER 2 – OBJECT-ORIENTED PROGRAMMING (OOP)

♦ Design Patterns (Q261–Q300)

(Very common in Amazon, Microsoft, Deloitte, Accenture, Infosys – especially for 3+ years)

CREATIONAL + STRUCTURAL + BEHAVIORAL DESIGN PATTERNS (Q261–Q300)

DESIGN PATTERN BASICS

Q261. (Infosys – Technical Round)

What is a Design Pattern in simple words?

Explanation

A design pattern is a reusable, proven solution to a common software design problem.

It is not a library or framework.

It is a guideline / template / best practice that developers follow.

Real-life analogy

Like engineers reuse construction blueprints instead of designing a new house plan every time.

Q262. (Accenture – Interview Favorite)

Why do MNCs care about design patterns?

Explanation

Design patterns ensure:

- Consistency in codebase
- Better maintainability

- Easier onboarding for developers
- Scalable and flexible architecture

Real project example

In large enterprise projects, multiple developers follow the same structured pattern → fewer conflicts and predictable design.

SINGLETON PATTERN

Q263. (Microsoft – Technical Round)

What is the Singleton pattern?

Explanation

Singleton ensures:

- Only one instance of a class exists
- Same instance is reused everywhere
- Centralized access point

Mostly used for global shared resources.

Real-time C# example

```
public sealed class Logger
```

```
{
```

```
private static Logger _instance;

private static readonly object _lock = new();

private Logger() { }

public static Logger Instance
{
    get
    {
        lock (_lock)
        {
            return _instance ??= new Logger();
        }
    }
}
```

Q264. (Amazon – Interview Favorite)

Real-world use of Singleton

Explanation

Useful when only one instance makes sense:

- Logger
 - Configuration manager
 - Cache manager
 - **Database connection pool controller**
-

Q265. (Accenture – Interview Trap)

Why is Singleton sometimes considered bad?

Explanation

Because:

- Hard to unit test
- Creates hidden dependencies
- Introduces global state
- Encourages tight coupling

So should be used carefully, not everywhere.

FACTORY METHOD

Q266. (Infosys – Technical Round)

What problem does Factory pattern solve?

Explanation

Factory pattern centralizes object creation.

Instead of using new across projects, creation is handled in one place → easier to maintain and extend.

C# example

```
interface IPayment { void Pay(); }
```

```
class PaymentFactory
```

```
{
```

```
    public static IPayment GetPayment(string type)
```

```
    {
```

```
        return type switch
```

```
        {
```

```
            "Card" => new CardPayment(),
```

```
            "UPI" => new UpiPayment(),
```

```
            _ => throw new Exception("Invalid")
```

```
        };
```

```
}  
  
}
```

Q267. (Amazon – Interview Favorite)

Factory vs new keyword

Explanation

- new → tightly couples code to concrete class
- Factory → hides creation logic & supports OCP

The factory makes the system scalable and maintainable.

BUILDER PATTERN

Q268. (Microsoft – Advanced Round)

When to use the Builder pattern?

Explanation

Use Builder when:

- Object has many optional properties
- You want readable object creation

- Avoid telescopic constructors

Real-time C# example

```
var user = new UserBuilder()  
  
    .SetName("John")  
  
    .SetEmail("a@b.com")  
  
    .Build();
```

Used heavily in:

Fluent APIs, Entity creation, Test data generation.

STRUCTURAL DESIGN PATTERNS

ADAPTER PATTERN

Q269. (Accenture – Real Project)

What is the Adapter pattern?

Explanation

Adapter allows two incompatible systems to work together by translating one interface to another.

Real-time example

Integrating third-party payment gateway into your application when their API doesn't match your interface.

Q270. (Microsoft – Technical Round)

Real C# adapter example

```
class RazorPayAdapter : IPayment
{
    private RazorPay _razorPay = new RazorPay();

    public void Pay() => _razorPay.MakePayment();
}
```

Adapter = Bridge between your system and external system.

FACADE PATTERN

Q271. (Amazon – Interview Favorite)

What is the Facade pattern?

Explanation

Facade provides a simple unified interface over a complex system.
Instead of calling 5 classes → call only one.

Real project example

Service layer hiding complex business operations or multiple APIs behind single entry service.

Q272. (Infosys – Technical Round)

Why is the facade used in layered architecture?

Explanation

Because UI should not deal with complex internals.

UI → Facade (Service Layer) → Complex subsystem

It improves:

- Simplicity
 - Security
 - Code organization
-

BEHAVIORAL DESIGN PATTERNS

STRATEGY PATTERN

Q273. (Microsoft – Interview Favorite)

Explain Strategy pattern

Explanation

Strategy allows changing behavior dynamically without changing main code.

We encapsulate multiple behaviors in separate classes and swap them when needed.

C# example

```
interface IDiscount
```

```
{
```

```
    decimal Apply(decimal amount);
```

```
}
```

Q274. (Amazon – Scenario Based)

Where Strategy pattern is used in real projects?

Explanation

Used where business rules change frequently:

- Pricing strategy
- Discount calculation
- Tax logic
- Sorting algorithms

Just plug in a new strategy → no code rewrite.

OBSERVER PATTERN

Q275. (Infosys – Technical Round)

What is the Observer pattern?

Explanation

Observer pattern notifies multiple dependent objects automatically when state changes.

Real-time example

- Email subscription
 - Notification systems
 - Realtime dashboards
-

Q276. (Microsoft – Advanced)

Observer pattern in .NET

Explanation

.NET implements Observer via:

- Events
- Delegates

So when the publisher fires an event → subscribers automatically get notified.

COMMAND PATTERN

Q277. (Amazon – Senior Round)

What is the Command pattern?

Explanation

Command pattern encapsulates a request as an object so it can be:

- Queued
- Logged
- Undone

Real-time example

Undo/Redo functionality in editors and IDEs.

TEMPLATE METHOD

Q278. (Infosys – Technical Round)

What is the Template Method pattern?

Explanation

Defines the skeleton of the algorithm in the base class while allowing subclasses to customize certain steps.

Used when algorithm steps are the same but implementation differs.

CHAIN OF RESPONSIBILITY

Q279. (Microsoft – Architecture Round)

What is the Chain of Responsibility pattern?

Explanation

Requests are passed along a chain of handlers.
Each handler decides:

- to process
- or pass to next handler

Real-time example

Middleware pipeline in ASP.NET Core.

REAL INTERVIEW COMPARISONS

Q280. (Accenture – Interview Favorite)

Factory vs Builder

Explanation

Factory → Chooses object type

Builder → Configures and builds complex object

PRACTICAL DESIGN QUESTIONS

Q281. (Amazon – Design Thinking)

Which pattern is used in Dependency Injection?

Answer

Factory + Strategy concepts.

Q282. (Microsoft – Senior Round)

Which pattern improves testability most?

Answer

Strategy + Dependency Injection → because they remove tight coupling.

DESIGN PATTERN PITFALLS

Q283. (Deloitte – Senior Round)

Common mistake with design patterns

Explanation

Using patterns even when the problem is simple.
Over-engineering kills simplicity.

Q284. (Infosys – Interview Trap)

Is Singleton a design pattern or anti-pattern?

Explanation

Both.

Good → Config, logging

Bad → Overused everywhere causing tight coupling

It depends on usage.

REAL PROJECT EXPERIENCE

Q285. (Amazon – Experience Based)

Design pattern you used most in your project

Q286. (Microsoft – Experience Based)

Design pattern you removed and why

ADVANCED THINKING

Q287. (Accenture – Senior Round)

How design patterns support SOLID?

Explanation

Design patterns are practical implementation of SOLID:

- Strategy → OCP + DIP
 - Factory → OCP
 - Facade → SRP
 - Observer → Loose coupling
-

Q288. (Deloitte – Architecture)

Design patterns vs frameworks

Explanation

Patterns = Concepts / Guidelines

Frameworks = Implementations using those concepts

TRICKY QUESTIONS

Q289. (Microsoft – Tricky)

Can multiple design patterns exist in one class?

Answer

Yes, but ensure responsibilities remain clear and justified.

FINAL DESIGN PATTERN QUESTIONS

Q290. (Amazon – Final Round)

Most overused design pattern?

Answer

Singleton.

Q291. (Infosys – HR + Tech)

How design patterns help team collaboration?

Q292. (Accenture – HR + Tech)

How do you explain design patterns to juniors?

Q293. (Microsoft – Principal Engineer)

When not to use design patterns?

Q294. (Deloitte – Leadership)

Design pattern mistake that caused production issue

Q295. (Amazon – Reflection)

Pattern you struggled with initially

Q296. (Microsoft – Advanced)

Strategy vs State pattern (difference)

Q297. (Accenture – Architect)

Design patterns used in ASP.NET Core

Explanation

- Dependency Injection → Factory pattern
 - Middleware pipeline → Chain of Responsibility
 - Options pattern → Builder style configuration
-

Q298. (Deloitte – Final Tech)

Do design patterns improve performance?

Explanation

Directly no.

Indirectly yes — by enabling better structure, easier optimization, and safer scaling.

Q299. (Amazon – Final Reflection)

One pattern every full-stack dev should know

Answer

Strategy pattern — because business logic changes frequently.

Q300. (ALL MNCs – FINAL DESIGN PATTERN QUESTION)

Why do design patterns still matter today?

Best Interview Answer

“They help developers solve recurring design problems in a clean, proven, scalable, and maintainable way — especially in large enterprise systems where good architecture matters the most.”

ATIONAL + STRUCTURAL + BEHAVIORAL DESIGN PATTERNS (Q261–Q300)

DESIGN PATTERN BASICS

Q261. (Infosys – Technical Round)

What is a Design Pattern in simple words?

Explanation

A design pattern is a **reusable, proven solution** to a common software design problem.
It is not a library or framework.
It is a **guideline / template / best practice** that developers follow.

Real-life analogy

Like engineers reuse **construction blueprints** instead of designing a new house plan every time.

Q262. (Accenture – Interview Favorite)

Why do MNCs care about design patterns?

Explanation

Design patterns ensure:

- Consistency in codebase
- Better maintainability
- Easier onboarding for developers
- Scalable and flexible architecture

Real project example

In large enterprise projects, multiple developers follow the same structured pattern → fewer conflicts and predictable design.

SINGLETON PATTERN

Q263. (Microsoft – Technical Round)

What is Singleton pattern?

Explanation

Singleton ensures:

- Only **one instance** of a class exists
- Same instance is reused everywhere
- Centralized access point

Mostly used for global shared resources.

Real-time C# example

```
public sealed class Logger
{
    private static Logger _instance;

    private static readonly object _lock = new();

    private Logger() { }

    public static Logger Instance
    {
        get
        {
            lock (_lock)
            {
                return _instance ??= new Logger();
            }
        }
    }
}
```

Q264. (Amazon – Interview Favorite)

Real-world use of Singleton

Explanation

Useful when only one instance makes sense:

- Logger
- Configuration manager
- Cache manager
- Database connection pool controller

Q265. (Accenture – Interview Trap)

Why is Singleton sometimes considered bad?

Explanation

Because:

- Hard to unit test
- Creates hidden dependencies
- Introduces global state
- Encourages tight coupling

So should be used carefully, not everywhere.



FACTORY PATTERN

Q266. (Infosys – Technical Round)

What problem does Factory pattern solve?

Explanation

Factory pattern **centralizes object creation**.

Instead of using `new` across projects, creation is handled in one place → easier to maintain and extend.

C# example

```
interface IPayment { void Pay(); }

class PaymentFactory
{
    public static IPayment GetPayment(string type)
    {
        return type switch
        {
            "Card" => new CardPayment(),
            "UPI" => new UpiPayment(),
            _ => throw new Exception("Invalid")
        };
    }
}
```

Q267. (Amazon – Interview Favorite)

Factory vs new keyword

Explanation

- `new` → tightly couples code to concrete class
- Factory → hides creation logic & supports OCP

The factory makes the system scalable and maintainable.



BUILDER PATTERN

Q268. (Microsoft – Advanced Round)

When to use the Builder pattern?

Explanation

Use Builder when:

- Object has many optional properties
- You want readable object creation
- Avoid telescopic constructors

Real-time C# example

```
var user = new UserBuilder()  
    .SetName("John")  
    .SetEmail("a@b.com")  
    .Build();
```

Used heavily in:

Fluent APIs, Entity creation, Test data generation.



STRUCTURAL DESIGN PATTERNS

ADAPTER PATTERN

Q269. (Accenture – Real Project)

What is the Adapter pattern?

Explanation

Adapter allows **two incompatible systems to work together** by translating one interface to another.

Real-time example

Integrating third-party payment gateway into your application when their API doesn't match your interface.

Q270. (Microsoft – Technical Round)

Real C# adapter example

```
class RazorPayAdapter : IPayment
{
    private RazorPay _razorPay = new RazorPay();
    public void Pay() => _razorPay.MakePayment();
}
```

Adapter = Bridge between your system and external system.

FACADE PATTERN

Q271. (Amazon – Interview Favorite)

What is the Facade pattern?

Explanation

Facade provides a **simple unified interface** over a complex system.
Instead of calling 5 classes → call only one.

Real project example

Service layer hiding complex business operations or multiple APIs behind single entry service.

Q272. (Infosys – Technical Round)

Why is the facade used in layered architecture?

Explanation

Because UI should not deal with complex internals.
UI → Facade (Service Layer) → Complex subsystem

It improves:

- Simplicity
 - Security
 - Code organization
-



BEHAVIORAL DESIGN PATTERNS



STRATEGY PATTERN

Q273. (Microsoft – Interview Favorite)

Explain Strategy pattern

Explanation

Strategy allows changing behavior dynamically without changing main code.

We encapsulate multiple behaviors in separate classes and swap them when needed.

C# example

```
interface IDiscount
{
    decimal Apply(decimal amount);
}
```

Q274. (Amazon – Scenario Based)

Where Strategy pattern is used in real projects?

Explanation

Used where business rules change frequently:

- Pricing strategy
- Discount calculation
- Tax logic
- Sorting algorithms

Just plug in a new strategy → no code rewrite.



OBSERVER PATTERN

Q275. (Infosys – Technical Round)

What is the Observer pattern?

Explanation

Observer pattern notifies multiple dependent objects automatically when state changes.

Real-time example

- Email subscription
 - Notification systems
 - Realtime dashboards
-

Q276. (Microsoft – Advanced)

Observer pattern in .NET

Explanation

.NET implements Observer via:

- Events
- Delegates

So when the publisher fires an event → subscribers automatically get notified.

COMMAND PATTERN

Q277. (Amazon – Senior Round)

What is the Command pattern?

Explanation

Command pattern encapsulates a request as an object so it can be:

- Queued
- Logged
- Undone

Real-time example

Undo/Redo functionality in editors and IDEs.

TEMPLATE METHOD

Q278. (Infosys – Technical Round)

What is the Template Method pattern?

Explanation

Defines the skeleton **of the algorithm** in base class while allowing subclasses to customize certain steps.

Used when algorithm steps are the same but implementation differs.

CHAIN OF RESPONSIBILITY

Q279. (Microsoft – Architecture Round)

What is the Chain of Responsibility pattern?

Explanation

Requests are passed along a chain of handlers.
Each handler decides:

- to process
- or pass to next handler

Real-time example

Middleware pipeline in ASP.NET Core.

REAL INTERVIEW COMPARISONS

Q280. (Accenture – Interview Favorite)

Factory vs Builder

Explanation

Factory → Chooses object type

Builder → Configures and builds complex object

PRACTICAL DESIGN QUESTIONS

Q281. (Amazon – Design Thinking)

Which pattern is used in Dependency Injection?

Answer

Factory + Strategy concepts.

Q282. (Microsoft – Senior Round)

Which pattern improves testability most?

Answer

Strategy + Dependency Injection → because they remove tight coupling.

DESIGN PATTERN PITFALLS

Q283. (Deloitte – Senior Round)

Common mistake with design patterns

Explanation

Using patterns even when the problem is simple.
Over-engineering kills simplicity.

Q284. (Infosys – Interview Trap)

Is Singleton a design pattern or anti-pattern?

Explanation

Both.

Good → Config, logging

Bad → Overused everywhere causing tight coupling

It depends on usage.

REAL PROJECT EXPERIENCE

Q285. (Amazon – Experience Based)

Design pattern you used most in your project

Q286. (Microsoft – Experience Based)

Design pattern you removed and why

ADVANCED THINKING

Q287. (Accenture – Senior Round)

How design patterns support SOLID?

Explanation

Design patterns are practical implementation of SOLID:

- Strategy → OCP + DIP
 - Factory → OCP
 - Facade → SRP
 - Observer → Loose coupling
-

Q288. (Deloitte – Architecture)

Design patterns vs frameworks

Explanation

Patterns = Concepts / Guidelines

Frameworks = Implementations using those concepts

TRICKY QUESTIONS

Q289. (Microsoft – Tricky)

Can multiple design patterns exist in one class?

Answer

Yes, but ensure responsibilities remain clear and justified.

FINAL DESIGN PATTERN QUESTIONS

Q290. (Amazon – Final Round)

Most overused design pattern?

Answer

Singleton.

Q291. (Infosys – HR + Tech)

How design patterns help team collaboration?

Q292. (Accenture – HR + Tech)

How do you explain design patterns to juniors?

Q293. (Microsoft – Principal Engineer)

When not to use design patterns?

Q294. (Deloitte – Leadership)

Design pattern mistake that caused production issue

Q295. (Amazon – Reflection)

Pattern you struggled with initially

Q296. (Microsoft – Advanced)

Strategy vs State pattern (difference)

Q297. (Accenture – Architect)

Design patterns used in ASP.NET Core

Explanation

- Dependency Injection → Factory pattern
 - Middleware pipeline → Chain of Responsibility
 - Options pattern → Builder style configuration
-

Q298. (Deloitte – Final Tech)

Do design patterns improve performance?

Explanation

Directly no.

Indirectly yes — by enabling better structure, easier optimization, and safer scaling.

Q299. (Amazon – Final Reflection)

One pattern every full-stack dev should know

Answer

Strategy pattern — because business logic changes frequently.

Q300. (ALL MNCs – FINAL DESIGN PATTERN QUESTION)

Why do design patterns still matter today?

Best Interview Answer

“They help developers solve recurring design problems in a clean, proven, scalable, and maintainable way — especially in large enterprise systems where good architecture matters the most.”

CHAPTER 3 – C# LANGUAGE & INTERNALS

◆ C# Fundamentals & Core Concepts (Q301–Q320)

(Asked in TCS, Infosys, Accenture, Deloitte, Cognizant, Microsoft, Amazon)

C# BASICS THAT MNCs CARE ABOUT (Q301–Q320)

Q301. (TCS / Infosys – Screening Round)

What is C# and where is it used in real projects?

Detailed Explanation

C# is a **strongly typed, modern, object-oriented programming language** mainly used to build enterprise-grade applications.

It runs on the **.NET Runtime**, which manages:

- Memory
- Security
- Execution pipeline

Because of stability, performance, and tooling support, C# is heavily used in:

- Web APIs (ASP.NET Core)
- Enterprise Applications (Banking, Healthcare, ERP)
- Cloud-Based Solutions (Azure)
- Desktop Applications
- Game Development (Unity)

Real-time C# example

```
[ApiController]

[Route("api/users")]

public class UsersController : ControllerBase
{
    [HttpGet]

    public IActionResult GetUsers()
    {
        return Ok("Users list");
    }
}
```

Real project usage

Used in:

- Banking portals
- Insurance applications
- Hospital systems
- Government portals
- Enterprise SaaS platforms

Q302. (Accenture – Technical Round)

What is the difference between Value Types and Reference Types?

Detailed Explanation

Value Types

- Store actual data
- Stored in Stack
- Copy is created when assigned

Reference Types

- Store memory address / reference
- Stored in Heap
- Multiple variables can point to same object

Real-time C# example

```
int a = 10;
```

```
int b = a;
```

```
b = 20; // a remains 10
```

```
class User { public string Name; }
```

```
User u1 = new User { Name = "John" };
```

```
User u2 = u1;
```

```
u2.Name = "Mike"; // u1.Name also becomes "Mike"
```

Real interview expectation

Always mention → **Stack vs Heap** and **behavior difference**.

Q303. (Microsoft – Technical Round)

What is Boxing and Unboxing? Why is it bad for performance?

Detailed Explanation

- **Boxing** = Converting value type → object
- **Unboxing** = Converting object → value type

This creates **extra memory allocation + CPU processing**, hence reduces performance.

Real-time C# example

```
int x = 10;  
  
object obj = x;      // Boxing  
  
int y = (int)obj;    // Unboxing
```

Real project impact

Avoid boxing in:

- Loops
- High-traffic APIs
- Performance-critical services

Prefer:

- Generics
- Strong typing

Q304. (Amazon – Interview Favorite)

What is var and does it make C# loosely typed?

Detailed Explanation

`var` = compiler inferred type.

Type is decided at compile time → **still strongly typed**.

Real-time C# example

```
var count = 10;      // int  
  
var name = "Admin"; // string
```

Interview trap

✗ Saying “var is dynamic”

Correct: `var` is **compile-time type inference**.

Q305. (Infosys – Technical Round)

Difference between dynamic and var

var	dynamic
Compile-time typing	Runtime typing
Faster	Slower
Safe	Risky

Real-time C# example

```
dynamic data = "Hello";  
  
data = 10; // Valid – checked at runtime
```

Real project usage

Used in:

- JSON parsing
 - Reflection
 - COM interop
-

MEMORY & EXECUTION

Q306. (Microsoft – Advanced Round)

What is Stack and Heap in C#?

Detailed Explanation

Stack

- Stores value types
- Method calls
- Function parameters
- Very fast

Heap

- Stores reference objects
- Larger memory blocks
- Managed by Garbage Collector

Real-time example

Stack → local variables

Heap → service objects in APIs

Q307. (Amazon – System Thinking)

Why is heap allocation slower than stack?

Explanation

Heap requires:

- Memory lookup
- Allocation tracking
- Garbage Collection monitoring

Stack is simple push-pop → instant.

Real-world impact

Creating too many objects = slow APIs & high memory usage.

Q308. (Accenture – Interview Favorite)

What is Garbage Collection (GC)?

Detailed Explanation

Garbage Collection automatically:

- Frees unused memory
- Prevents memory leaks
- Improves app stability

Real-time C# example

```
User user = new User();  
  
user = null;    // Eligible for GC
```

Interview Tip

Mention GC generations → interviewer happy.

Q309. (Microsoft – Advanced Round)

What are GC Generations and why do they matter?

Detailed Explanation

- **Gen 0** → short-lived objects (local vars)
- **Gen 1** → mid-lived objects
- **Gen 2** → long-lived objects (caches, static data)

GC uses Generational collection = better performance.



NULL, SAFETY & ERRORS

Q310. (TCS – Screening Round)

What is null and why is it dangerous?

Detailed Explanation

null = reference pointing to nothing.

Accessing null leads to:

`NullReferenceException`

Real-time C# example

```
User user = null;
```

```
Console.WriteLine(user.Name);
```

Q311. (Microsoft – Interview Favorite)

What is a null-conditional operator ?.

Detailed Explanation

Prevents null crashes.

Real-time C# example

```
string name = user?.Name;
```

Real project usage

API response mapping safely.

Q312. (Accenture – Developer Round)

Difference between == and Equals()

Detailed Explanation

- `==` → Value or Reference comparison (operator overloaded)
- `Equals()` → Logical equality meaning

Real project usage

Comparing objects in collections or domain models.



METHODS & PARAMETERS

Q313. (Infosys – Technical Round)

What is the ref and out keyword?

Detailed Explanation

- `ref` → value must be initialized before passing
- `out` → method must assign value

Real-time C# example

```
void GetData(out int value)
{
    value = 10;
}
```

Q314. (Microsoft – Advanced Round)

Why avoid ref and out in large systems?

Explanation

They:

- Reduce code readability
- Increase coupling
- Make debugging harder

Use DTOs or return objects instead.



EXCEPTIONS & ERROR HANDLING

Q315. (Amazon – Interview Favorite)

Difference between throw and throw ex

Detailed Explanation

- `throw` → preserves original stack trace
- `throw ex` → resets stack trace (bad)

Correct C# usage

```
catch(Exception)
{
    throw;    // Correct
}
```

Q316. (Deloitte – Senior Round)

When should you use custom exceptions?

Explanation

Use when representing **business errors**, like:

- `InvalidPaymentException`
- `OrderNotFoundException`
- `AccessDeniedException`

They improve readability and meaningful debugging.



IMMUTABILITY & DESIGN

Q317. (Microsoft – Advanced)

Why is DateTime immutable?

Explanation

To ensure:

- Thread safety
- Predictability
- Consistent state

Changing value returns a new object → safe.

Q318. (Accenture – Real Project)

How does immutability help multi-threading?

Explanation

Immutable objects:

- No shared modification
- No race conditions
- Safe parallel execution

Perfect for concurrent systems.



INTERVIEW REFLECTION

Q319. (Amazon – Senior Round)

Most common C# mistake by juniors

Answer

- Creating too many objects
- Ignoring memory impact
- Misusing dynamic
- Not handling null properly

Q320. (ALL MNCs – FINAL C# FUNDAMENTALS QUESTION)

Why strong C# fundamentals matter in enterprise apps?

Best Interview Answer

“Because application performance, memory efficiency, scalability, and reliability directly depend on how well developers understand C# fundamentals and runtime behavior.”

◆ ASYNC & MULTITHREADING (Q321–Q340)

(Very important for Accenture, Microsoft, Amazon, Deloitte – 2+ years)

Q321. (Infosys – Technical Round)

What is asynchronous programming in C#?

Detailed Explanation

Asynchronous programming allows long-running operations to execute **without blocking the main thread**.

Instead of waiting, the thread is released to handle other work, which keeps applications responsive and scalable.

Best for:

- API calls
- Database queries
- File IO
- Network calls

Real-time C# example (Web API)

```
public async Task<IActionResult> GetUsers()  
{  
    var users = await _userService.GetUsersAsync();  
    return Ok(users);  
}
```

Real project usage

Calling databases, microservices, payment gateways, cloud APIs etc.

Q322. (Accenture – Interview Favorite)

Difference between synchronous and asynchronous execution

Synchronous	Asynchronous
Blocks thread	Non-blocking

Waits until operation completes Continues execution

Slower under load Highly scalable

Real example

Synchronous DB call → UI freezes
Async DB call → UI stays responsive.

Q323. (Microsoft – Technical Round)

What is async and await?

Explanation

- `async` → marks method as asynchronous
- `await` → pauses execution **without blocking** the thread until task completes

🔥 Execution resumes when the result is ready.

C# example

```
await Task.Delay(2000);
```

Q324. (Amazon – Interview Favorite)

Why does async not create a new thread?

Explanation

Async programming is **Task-based**, not thread-based.
When awaiting:

- Thread is released back to ThreadPool
- No thread is blocked
- Execution resumes later

This improves scalability massively.

Q325. (Infosys – Technical Round)

What is Task in C#?

Explanation

Task represents:

- An asynchronous operation
- A promise of future result

C# example

```
Task t = Task.Run(() => DoWork());
```

Q326. (Microsoft – Advanced Round)

Difference between Thread and Task

Thread	Task
Heavy	Lightweight
Manual control	ThreadPool managed

Expensive

Optimized

Low-level

High-level abstraction

Q327. (Accenture – Developer Round)

When should you use multithreading?

Explanation

Use when work is **CPU-bound**, meaning heavy computation.

Real example

- Image processing
 - Report generation
 - Data analytics
 - Complex calculations
-

Q328. (Amazon – System Thinking)

Why does async improve scalability in Web APIs?

Explanation

While API waits for DB/External service:

- Thread is released
- Web server handles more users
- Less thread blocking

Result = Better throughput + lower resource usage.

Q329. (Microsoft – Advanced)

What is a deadlock in async programming?

Explanation

Deadlock occurs when:

- Two operations wait on each other forever

Real cause

Using `.Result` or `.Wait()` on async methods.

Q330. (Accenture – Interview Trap)

Why `.Result` is dangerous?

Explanation

Because it:

- Blocks thread
- Can cause deadlock
- Breaks async flow

Always prefer `to wait`.

Q331. (Deloitte – Senior Round)

What is Parallel programming?

Explanation

Parallel programming executes multiple tasks **simultaneously** to utilize multiple CPU cores.

C# example

```
Parallel.ForEach(list, item => Process(item));
```

Q332. (Microsoft – Advanced)

Task.WhenAll vs Task.WhenAny

Explanation

- `Task.WhenAll` → waits for all tasks to finish
- `Task.WhenAny` → returns when first task finishes

Q333. (Amazon – Coding Round)

How to run tasks in parallel safely?

Explanation

Avoid:

- Shared mutable state
- Writing on shared variables

Prefer:

- Immutable objects
- Thread-safe collections

Q334. (Infosys – Technical Round)

What is lock and why is it used?

Explanation

Lock prevents multiple threads from accessing shared data at same time.

C# example

```
lock(_obj)

{

    count++;

}
```

Q335. (Microsoft – Advanced)

Why is excessive locking bad?

Explanation

Because:

- It blocks threads
- Causes contention
- Reduces performance

Use wisely.

Q336. (Accenture – Real Project)

Thread-safe collection example

C# example

```
ConcurrentDictionary<int,string> dict = new();
```

Useful in:

- Caching
 - Logging
 - Multi-thread data storage
-

Q337. (Amazon – Architecture)

Async vs Multithreading – which to choose?

Explanation

- Async → I/O bound work (API calls, DB, file)
 - Multithreading → CPU bound work (processing heavy tasks)
-

Q338. (Microsoft – Debugging)

Why do async exceptions behave differently?

Explanation

Because exceptions are wrapped inside `Task`.
They need to be awaited to observe errors.

Q339. (Deloitte – Senior Round)

Best practices for async methods

Explanation

- Avoid `async void`
- Use `async Task`

- Use await everywhere
 - Avoid `.Result` and `.Wait()`
-

Q340. (ALL MNCs – FINAL ASYNC QUESTION)

Why is async mandatory in modern .NET apps?

Best Answer

“Because async improves scalability, performance, responsiveness, and resource usage — which is critical in modern cloud-based and high-traffic applications.”

◆ LINQ & COLLECTIONS (Q341–Q360)

(Extremely common in Accenture, Infosys, Microsoft, Amazon)

Q341. (TCS – Screening Round)

What is LINQ?

Detailed Explanation

LINQ provides:

- SQL-like querying
- On collections, objects, DB data
- With clean and readable syntax

C# example

```
var activeUsers = users.Where(u => u.IsActive);
```

Q342. (Infosys – Technical Round)

Why is LINQ preferred over loops?

Explanation

- Clean
- Readable
- Less error-prone
- Faster development

Q343. (Accenture – Interview Favorite)

LINQ deferred execution – explain

Explanation

Query runs **only when iterated**.

```
var query = list.Where(x => x > 10); // not executed yet
```

Executes when:

- foreach
- ToList()
- Count()

Q344. (Microsoft – Advanced)

Difference between IEnumerable and IQueryable

IEnumerable

IQueryable

In-memory

Executes at DB

Client-side filtering

Server-side filtering

Q345. (Amazon – Interview Favorite)

Select vs SelectMany

Select

SelectMany

one-to-one

flatten nested
collections

Q346. (Accenture – Developer Round)

First vs FirstOrDefault

- `First()` → throws exception if not found
 - `FirstOrDefault()` → returns default(null/0)
-

Q347. (Microsoft – Technical Round)

Where clause real example

```
var orders = list.Where(o => o.Amount > 1000);
```

Q348. (Infosys – Technical Round)

OrderBy vs ThenBy

OrderBy → primary sorting

ThenBy → secondary sorting

Q349. (Amazon – Coding Round)

GroupBy real use case

```
var grouped = orders.GroupBy(o => o.CustomerId);
```

Q350. (Microsoft – Advanced)

Why can LINQ be slow sometimes?

Explanation

- Deferred execution
 - Multiple enumeration
 - Hidden allocations
-

Q351. (Accenture – Interview Favorite)

List vs ArrayList

List → generic + type-safe

ArrayList → non-generic + boxing issue

Q352. (Infosys – Technical Round)

List vs IEnumerable

List → concrete collection

IEnumerable → read only forward iteration

Q353. (Amazon – Scenario)

When to use Dictionary instead of List?

When:

- Fast lookup needed
 - Search by key frequently
-

Q354. (Microsoft – Advanced)

HashSet vs List

HashSet:

- Prevents duplicates
 - Faster search $O(1)$
-

Q355. (Accenture – Developer Round)

Why are collections not thread-safe by default?

Performance reasons.

Thread-safety adds overhead.

Q356. (Deloitte – Senior Round)

Thread-safe collection example

```
ConcurrentQueue<int> queue = new();
```

Q357. (Amazon – Performance)

Why is LINQ in loops dangerous?

Creates:

- Extra allocations
 - Performance issues
-

Q358. (Microsoft – Best Practice)

When to avoid LINQ?

- High-performance loops
 - Game engines
 - Real-time systems
-

Q359. (Accenture – Interview Favorite)

Most used LINQ operators in real projects

Where, Select, Any, FirstOrDefault, GroupBy.

Q360. (ALL MNCs – FINAL LINQ QUESTION)

Why is LINQ important for Full-Stack Developers?

Best Interview Answer

“Because LINQ simplifies data processing, reduces boilerplate code, improves readability, and helps write clean business logic efficiently.”

CHAPTER 4 – ASP.NET CORE & WEB API

◆ Core Concepts & Fundamentals (Q361–Q380)

(Very important for Full-Stack .NET roles – Junior to Senior)

ASP.NET CORE BASICS (Q361–Q380)

Q361. (TCS / Infosys – Screening Round)

What is ASP.NET Core and why do companies use it?

Detailed, explanation

ASP.NET Core is a **modern, cross-platform, high-performance web framework** used to build:

- REST APIs
- Microservices
- Web applications
- Cloud applications

Companies love it because ASP.NET Core is:

- **Fast** – one of the fastest web frameworks
- **Lightweight** – minimal overhead
- **Cross-platform** – runs on Windows, Linux, Mac
- **Cloud-friendly** – works great with Docker, Kubernetes, Azure
- **Easy to scale** – handles large traffic easily

Real-time C# example

Simple Health Check API:

```
[ApiController]
[Route("api/health")]
public class HealthController : ControllerBase
{
    [HttpGet]
    public IActionResult Check()
    {
        return Ok("API is running");
    }
}
```

Real project usage

Used for:

- Load balancer health checks
- Monitoring tools
- Cloud deployment readiness

Q362. (Accenture – Technical Round)

Difference between ASP.NET and ASP.NET Core

Explanation (real interview style)

ASP.NET Core

- Cross-platform (Windows / Linux / Mac)
- Much faster performance
- Modular & lightweight
- Built for microservices & cloud
- New dependency injection pipeline
- Supports minimal APIs & modern features

Old ASP.NET (.NET Framework)

- Windows-only
- Heavy
- Limited performance
- Monolithic by nature

Interview Tip

Say confidently:

“ASP.NET Core is the future and preferred for modern enterprise development.”

Q363. (Microsoft – Technical Round)

What is Kestrel and why is it important?

Explanation

Kestrel is the **built-in web server** in ASP.NET Core.
It handles HTTP requests and responses.

Why is it important?

- Extremely fast
- Cross-platform
- Always used internally in ASP.NET Core

Real-time production

- On Windows → Kestrel + IIS
- On Linux → Kestrel + Nginx
- In Docker → runs directly



MIDDLEWARE PIPELINE

Q364. (Amazon – Interview Favorite)

What is middleware in ASP.NET Core?

Explanation

Middleware is **a component that runs on every HTTP request and response**.

Flow:

Request → Middleware → Controller → Middleware → Response

Real-time C# example

```
app.Use(async (context, next) =>
```

```
{  
  
    Console.WriteLine("Request started");  
  
    await next();  
  
    Console.WriteLine("Request finished");  
  
});
```

Real project usage

Middleware is commonly used for:

- Logging
- Authentication
- Authorization
- Exception handling
- CORS
- Request modification

Q365. (Infosys – Technical Round)

Order of middleware – why does it matter?

Explanation

Middleware executes **in the exact order added**.

Wrong order = unexpected bugs.

Example:

- Authentication must happen before Authorization
- Exception middleware must be early
- Static files middleware should be before MVC

Bad order causes:

- Auth always failing
- Exceptions not caught
- Responses breaking



CONTROLLERS & ROUTING

Q366. (Accenture – Developer Round)

What is a Controller in Web API?

Explanation

Controller:

- Receives HTTP requests
- Executes business logic
- Returns HTTP response

Real-time C# example

```
[ApiController]
[Route("api/users")]
public class UsersController : ControllerBase
{
    [HttpGet("{id}")]
    public IActionResult GetUser(int id)
    {
```

```
        return Ok($"User {id}");  
    }  
}
```

Q367. (Microsoft – Interview Favorite)

Attribute routing vs Conventional routing

Explanation

Attribute Routing

- Route written on controller
- Mostly used in APIs

Conventional Routing

- Route defined in Program.cs
- Mostly used in MVC web apps

Real-world

APIs → Attribute Routing (almost always)



MODEL BINDING & VALIDATION

Q368. (Infosys – Technical Round)

What is model binding?

Explanation

Model binding automatically maps:

- JSON request body
- Query parameters
- Route values

to C# objects.

Real-time C# example

```
public IActionResult CreateUser(UserDto user)
{
    return Ok(user.Name);
}
```

Q369. (Amazon – Interview Favorite)

How validation works in Web API?

Explanation

Uses **Data Annotations**.

Real-time C# example

```
public class UserDto
{
    [Required]
    public string Email { get; set; }
}
```

If invalid → automatic

400 Bad Request

ASP.NET Core automatically validates `[ApiController]` annotated controllers.

DEPENDENCY INJECTION (VERY IMPORTANT)

Q370. (Microsoft – Technical Round)

What is Dependency Injection in ASP.NET Core?

Explanation

Instead of creating objects manually using `new`, ASP.NET Core **creates and injects dependencies** automatically.

Real-time C# example

```
public class OrderController : ControllerBase
{
    private readonly IOrderService _service;

    public OrderController(IOrderService service)
    {
        _service = service;
    }
}
```

Benefits:

- Loose coupling
 - Easy testing
 - Clean architecture
-

Q371. (Accenture – Interview Favorite)

Service lifetimes – Singleton, Scoped, Transient

Lifetime	Meaning	Use case
Singleton	One instance per app	Config, cache
Scoped	One per request	DB context
Transient	New instance each time	Lightweight services



ERROR HANDLING & FILTERS

Q372. (Amazon – Scenario Based)

How do you handle exceptions globally?

Explanation

Use Global Exception Handling Middleware.

Real-time example

```
app.UseExceptionHandler("/error");
```


Or custom middleware is commonly used in real projects.

Q373. (Microsoft – Advanced Round)

Filters vs Middleware – difference

Explanation

Middleware:

- Runs for entire pipeline
- Works for all requests

Filters:

- Work inside MVC pipeline
- Specific to controllers/actions

Filters are ideal for:

- Authorization filters
 - Action filters
 - Logging filters
-



REST & HTTP CONCEPTS

Q374. (Infosys – Screening Round)

What is REST API?

Explanation

REST uses standard HTTP verbs to perform actions on resources.

Method	Meaning
GET	Read
POST	Create
PUT	Full Update
PATCH	Partial Update
DELETE	Remove

Q375. (Amazon – Interview Favorite)

Difference between PUT and PATCH

Explanation

PUT:

- Replaces entire object
- Full update

PATCH:

- Updates only specific fields
- Partial update

- Used in modern APIs
-

AUTHENTICATION & AUTHORIZATION

Q376. (Microsoft – Technical Round)

Authentication vs Authorization

Explanation

Authentication → Who are you? (login identity)

Authorization → What can you do? (permissions)

Q377. (Accenture – Developer Round)

JWT authentication – real usage

Explanation

JWT is used for:

- Secure API access
- Stateless authentication
- Microservices

Real header

Authorization: Bearer <token>

Widely used in mobile apps, SPAs, microservices.



PERFORMANCE & BEST PRACTICES

Q378. (Amazon – System Design)

Why are async controllers important?

Explanation

Async controllers:

- Free threads while waiting
- Handle more requests
- Improve scalability

Essential for high-traffic APIs.

Q379. (Microsoft – Best Practice)

Why are DTOs used instead of entities?

Explanation

- Security (avoid exposing DB structure)
 - Clean contracts
 - Performance control
 - Prevent over-posting attacks
-

Q380. (ALL MNCs – FINAL ASP.NET CORE QUESTION)

How does ASP.NET Core fit into microservices architecture?

Best Interview Answer

“ASP.NET Core is lightweight, extremely fast, cloud-ready, supports Docker & Kubernetes, integrates with gRPC, messaging systems, and is ideal for building scalable microservices.”

- ◆ **Advanced Web API: Security, Filters & Versioning (Q381–Q410)**



SECURITY IN WEB API (Q381 – Q410)

Q381. (Microsoft – Technical Round)

How do you secure an ASP.NET Core Web API?

Detailed explanation (Easy way)

Security in Web API is never one feature; it is a **layered protection strategy**. A secure API ensures:

- 1 **Authentication** – verify who the user is
- 2 **Authorization** – ensure only allowed users can access
- 3 **Validation & Filtering** – block malicious input
- 4 **HTTPS only** – secure transport
- 5 **Secure Headers** – prevent attacks
- 6 **Logging & Monitoring** – track malicious activity

Most modern .NET APIs use **JWT authentication** because it is:

- Stateless
- Lightweight
- Cloud-friendly

Real-time C# example

Register JWT Authentication:

```
builder.Services.AddAuthentication("Bearer")
    .AddJwtBearer();
```

Enable middleware:

```
app.UseAuthentication();
app.UseAuthorization();
```

Real project usage

Used heavily in:

- Banking APIs
- Payment systems
- Government portals
- Microservices authentication

Q382. (Accenture – Interview Favorite)

What is JWT and why is it used?

Explanation

JWT (JSON Web Token) is a **secure, digitally signed token** that contains user identity and claims.

Why do companies use JWT?

- No session storage needed → **Stateless**
- Works perfectly in distributed systems and cloud
- Fast validation
- Built-in expiration

Real-time flow

- 1 User logs in
 - 2 Server generates JWT
 - 3 Client sends token in every request
 - 4 Server validates & grants access
-

Q383. (Amazon – Security Round)

Where is JWT stored on the client side and why?

Explanation

Most common storage options:

✓ HTTP-Only Cookies

- Best security
- Prevents XSS token theft

✓ In-memory (for SPA apps)

- Safer than localStorage
- Token cleared when tab closes

✗ Avoid localStorage for sensitive apps because:

- Vulnerable to XSS
 - Token theft risk
-

Q384. (Microsoft – Technical Round)

How do you protect API endpoints using JWT?

Real-time C# example

```
[Authorize]
[HttpGet("profile")]
public IActionResult GetProfile()
```

```
{  
    return Ok("Secure data");  
}
```

Interview Tip

Say confidently:

“We protect endpoints using `[Authorize]` and configure authentication middleware.”

Q385. (Infosys – Screening Round)

Authentication vs Authorization – practical difference

Explanation

Authentication → **Are you a valid user?**

Authorization → **What are you allowed to do?**

Real project example

- User logs in = Authentication
 - Only Admin can delete user = Authorization
-

ROLE & POLICY-BASED AUTHORIZATION

Q386. (Microsoft – Advanced Round)

Role-based authorization – real usage

C# example

```
[Authorize(Roles = "Admin")]
```



```
public IActionResult DeleteUser(int id)
{
    return Ok();
}
```

Real usage

- Admin-only APIs
 - Management portals
 - Corporate dashboards
-

Q387. (Accenture – Senior Round)

Policy-based authorization – when to use?

Explanation

Use Policy Authorization when:

- Rules are complex
- Role-based is not enough
- Need claim-based logic

C# example

```
builder.Services.AddAuthorization(options =>
{
    options.AddPolicy("AdultOnly",
        policy => policy.RequireClaim("Age", "18"));
});
```

Usage:

```
[Authorize(Policy = "AdultOnly")]
```



FILTERS (VERY IMPORTANT FOR INTERVIEWS)

Q388. (Microsoft – Interview Favorite)

What are filters in ASP.NET Core?

Explanation

Filters allow executing code:

- Before an action
- After an action
- Before returning response
- On exceptions

They help avoid duplicate logic.

Real project usage

- Logging
- Validation
- Auditing
- Authorization
- Exception handling

Q389. (Infosys – Technical Round)

Types of filters in ASP.NET Core

Filter Type	Purpose
-------------	---------

Authorization	Security checks
Action	Execute before/after action
Result	Modify response
Exception	Handle errors globally

Q390. (Amazon – Coding Round)

Action filter real-time example

```
public class LogActionFilter : IActionFilter
{
    public void OnActionExecuting(ActionExecutingContext context)
    {
        Console.WriteLine("Action started");
    }

    public void OnActionExecuted(ActionExecutedContext context)
    {
        Console.WriteLine("Action finished");
    }
}
```

Apply:

```
[ServiceFilter(typeof(LogActionFilter))]
```

Q391. (Microsoft – Advanced)

Filter vs Middleware – real difference

Explanation (Easy way)

Middleware:

- Runs for **every request**

- Works outside MVC
- Best for: Logging, Auth pipeline, CORS

Filters:

- Run only inside MVC
- Controller + Action specific
- Best for: Validation, Action logging, Exception handling

EXCEPTION HANDLING (PRODUCTION LEVEL)

Q392. (Amazon – Scenario Based)

Best way to handle exceptions globally in Web API

Explanation

Use **Global Exception Handling Middleware**, not try-catch everywhere.

C# example

```
app.UseExceptionHandler(errorApp =>
{
    errorApp.Run(async context =>
    {
        context.Response.StatusCode = 500;
        await context.Response.WriteAsync("Something went wrong");
    });
});
```

Q393. (Deloitte – Senior Round)

Why not use try-catch in every controller?

Explanation

- Code duplication
- Hard maintenance
- Missed corner cases
- Less control

Global handling = cleaner & safer.



API VERSIONING

Q394. (Microsoft – Interview Favorite)

Why is API versioning needed?

Explanation

APIs evolve. Clients may still use old versions.
Versioning ensures:

- Backward compatibility
- No breaking changes
- Smooth upgrades

Real example

Mobile apps using v1 while new clients use v2.

Q395. (Accenture – Technical Round)

Different API versioning strategies

Explanation

① URL-based

`/api/v1/users`

② Header-based

`x-api-version: 1`

③ Query string

`/api/users?version=1`

Q396. (Amazon – Coding Round)

URL-based API versioning – C# example

```
[ApiVersion("1.0")]
[Route("api/v{version:apiVersion}/users")]
public class UsersV1Controller : ControllerBase
{
}
```

Q397. (Microsoft – Advanced Round)

Which versioning strategy is most popular?

Explanation

URL Versioning

- Visible
- Simple
- Easy to manage

Most used in real projects.



SECURITY BEST PRACTICES

Q398. (Accenture – Security Round)

How do you prevent over-posting attacks?

Explanation

Never bind domain/entity directly.
Always use **DTOs**.

Q399. (Amazon – System Design)

Why is HTTPS mandatory for APIs?

Explanation

- Encrypts data
 - Prevents sniffing
 - Protects credentials & tokens
 - Required by security standards
-

Q400. (Microsoft – Best Practice)

How to secure sensitive configuration data?

Explanation

Use:

- Environment variables
- Azure Key Vault
- AWS Secrets Manager
- User Secrets (Development)

✗ Never store secrets in appsettings.json

PERFORMANCE & SCALABILITY

Q401. (Amazon – Architecture Round)

Why is caching important in Web APIs?

Explanation

Caching:

- Reduces database hits
 - Improves speed
 - Reduces cost
 - Improves scalability
-

Q402. (Microsoft – Technical Round)

In-memory caching example

```
IMemoryCache cache;
```

```
cache.Set("users", users, TimeSpan.FromMinutes(5));
```

Q403. (Accenture – Senior Round)

Why is response compression used?

Explanation

- Reduces response size
 - Faster API response
 - Better user experience
-



REAL INTERVIEW SCENARIOS

Q404. (Deloitte – Scenario Based)

How do you rate-limit an API?

Explanation

Limit number of requests per user/client.

Used to prevent:

- Abuse
 - Spam
 - DDoS
-

Q405. (Amazon – Advanced)

What happens if a token expires during request?

Explanation

API returns:

401 Unauthorized

Client must:

- Refresh token
- Retry request

FINAL ADVANCED QUESTIONS

Q406. (Microsoft – Final Round)

How do filters improve maintainability?

They avoid duplicated logic and centralize cross-cutting concerns.

Q407. (Accenture – Final Tech)

JWT vs OAuth – difference

- JWT → Token format
- OAuth → Authorization framework

OAuth often uses JWT.

Q408. (Amazon – Final Architecture)

How does the Web API support microservices communication?

Using:

- REST
 - gRPC
 - Messaging (Kafka / RabbitMQ)
-

Q409. (Deloitte – Leadership)

Biggest Web API security mistake you fixed

Common answer:

“Developers exposing entities instead of DTOs and missing authorization on sensitive endpoints.”

Q410. (ALL MNCs – FINAL WEB API QUESTION)

How do you design a secure, scalable Web API?

Best Interview Answer

“By combining authentication, authorization, validation, HTTPS, secure configuration, caching, async programming, monitoring, and clean versioned API contracts to ensure both **security and scalability**.”

CHAPTER 5 – DATABASE & SQL

◆ SQL Fundamentals & RDBMS Concepts (Q411–Q440)

(Asked in Accenture, Deloitte, Infosys, Cognizant, Amazon, Microsoft)



DATABASE BASICS (Q411–Q440)

Q411. (TCS / Infosys – Screening Round)

What is a database and why do applications need it?

Easy explanation

A database is a structured place where applications **store data permanently** so it can be:

- Retrieved quickly
- Updated safely
- Shared by thousands/millions of users
- Protected from crashes and data loss

Without a database:

- Data would disappear when app restarts
- No multi-user access
- No reliable storage

Real-world examples

- Bank transactions stored in SQL Server
- User accounts saved in MySQL
- Orders stored in PostgreSQL or Oracle

Q412. (Accenture – Technical Round)

Difference between DBMS and RDBMS

DBMS	RDBMS
Stores data	Stores data in tables with relations
No relationships	Supports Foreign Keys

Less strict rules Enforces constraints & integrity

Used in small apps Used in enterprise systems

Real Usage

- Simple local storage → DBMS
- Corporate, banking, enterprise systems → RDBMS



TABLES, KEYS & CONSTRAINTS

Q413. (Microsoft – Technical Round)

What is a Primary Key? Why is it important?

Explanation

Primary Key:

- Uniquely identifies each row
- Prevents duplicate records
- Speeds up searching
- Maintains data integrity

SQL Example

```
CREATE TABLE Users (  
    Id INT PRIMARY KEY,  
    Email VARCHAR(100) UNIQUE  
);
```

Q414. (Infosys – Interview Favorite)

Primary Key vs Unique Key

Primary Key	Unique Key
Cannot be NULL	Can allow one NULL
Only 1 per table	Multiple unique keys allowed
Identifies record	Ensures uniqueness

Q415. (Amazon – DSA + SQL Round)

What is a Foreign Key?

Explanation

Foreign Key creates **relationships between tables** and ensures data consistency.

SQL Example

```
ALTER TABLE Orders  
ADD CONSTRAINT FK_UserOrder  
FOREIGN KEY(UserId) REFERENCES Users(Id);
```

Real Project Usage

Order must belong to a valid user.

Q416. (Accenture – Developer Round)

What are constraints and why are they used?

Explanation

Constraints protect data quality.

Examples:

- **NOT NULL** → field cannot be empty
- **UNIQUE** → prevents duplicate values

- **CHECK** → validates condition
- **FOREIGN KEY** → relational integrity

They prevent wrong data from entering DB.



NORMALIZATION (VERY IMPORTANT)

Q417. (Microsoft – Interview Favorite)

What is normalization in databases?

Explanation

Normalization is organizing tables to:

- Remove duplicate data
- Reduce storage waste
- Improve consistency
- Avoid anomalies

Real example

Customer stored once
→ not repeated in every order row.

Q418. (Deloitte – Senior Round)

Explain 1NF, 2NF, 3NF in simple words

1NF:

- No repeating columns

- Atomic values only

2NF:

- Must be in 1NF
- No partial dependency on composite key

3NF:

- Must be in 2NF
 - No dependency on non-key column
-

Q419. (Amazon – Scenario Based)

When should you avoid normalization?

Explanation

Avoid heavy normalization when:

- System is read-heavy
- Performance matters more than storage
- Analytics/reporting use cases

Then we use **Denormalization** for faster reads.

SQL QUERIES

Q420. (TCS – Screening Round)

Difference between DELETE, TRUNCATE, DROP

Command	Removes Data	Removes Structure	Rollback
DELETE	Yes	No	Yes
TRUNCATE	Yes	No	No
DROP	Yes	Yes	No

Q421. (Infosys – Technical Round)

What is an index and why is it used?

Explanation

Index works like a book index:

- Speeds up SELECT queries
- Faster searching
- Improves performance

SQL Example

```
CREATE INDEX idx_email ON Users(Email);
```

Q422. (Amazon – Interview Favorite)

Clustered vs Non-Clustered Index

Clustered Index	Non-Clustered Index
Sorts actual table data	Separate index structure
Only one per table	Multiple allowed
Faster retrieval	Slightly slower



PERFORMANCE & OPTIMIZATION

Q423. (Microsoft – Advanced Round)

Why are too many indexes bad?

Because:

- Slows INSERT, UPDATE, DELETE
- Takes extra storage
- Adds maintenance overhead

Indexes = speed for reading, cost for writing.

Q424. (Accenture – Real Project)

How do you find slow SQL queries?

Use:

- Execution Plan
 - SQL Profiler
 - Performance Monitor
 - Query Store
-



TRANSACTIONS & CONCURRENCY

Q425. (Microsoft – Technical Round)

What is a transaction?

Explanation

Transaction = **All-or-Nothing**

Either all operations succeed or all fail.

SQL Example

```
BEGIN TRAN
```

```
UPDATE Accounts SET Balance -= 100 WHERE Id = 1
```

```
UPDATE Accounts SET Balance += 100 WHERE Id = 2
```

```
COMMIT
```

If something fails → ROLLBACK.

Q426. (Amazon – Scenario Based)

What are ACID properties?

- **Atomicity** → all or nothing
 - **Consistency** → valid state maintained
 - **Isolation** → transactions don't disturb each other
 - **Durability** → data persists after commit
-

Q427. (Deloitte – Senior Round)

Isolation levels in SQL

- Read Uncommitted
- Read Committed
- Repeatable Read
- Serializable

Higher level = stronger safety but slower performance.

JOINS

Q428. (Infosys – Interview Favorite)

Types of SQL joins

- INNER JOIN
 - LEFT JOIN
 - RIGHT JOIN
 - FULL JOIN
-

Q429. (Amazon – Coding Round)

INNER JOIN real example

```
SELECT u.Name, o.Amount  
FROM Users u  
INNER JOIN Orders o  
ON u.Id = o.UserId;
```

Fetches only matching records.

Q430. (Microsoft – Advanced Round)

LEFT JOIN vs INNER JOIN – real usage

LEFT JOIN

- Returns all left table rows
- Even without match

INNER JOIN

- Returns only matching rows

Use LEFT JOIN when missing related data is acceptable.



STORED PROCEDURES & VIEWS

Q431. (Accenture – Developer Round)

What is a Stored Procedure and why use it?

Benefits

- Faster execution
 - Precompiled
 - Secure
 - Reusable
 - Centralized logic
-

Q432. (Microsoft – Technical Round)

Stored Procedure vs Function

Stored Procedure	Function
May return or not	Must return
Supports transactions	No
Can modify data	Mostly not used for modification



DATABASE + C# INTEGRATION

Q433. (Amazon – Real Project)

How does ASP.NET Core communicate with SQL?

Using:

- ADO.NET
 - Entity Framework Core
 - Dapper
-

Q434. (Infosys – Technical Round)

Parameterized queries – why important?

Prevents **SQL Injection attacks**
Ensures secure input handling

C# Example

```
cmd.Parameters.AddWithValue("@id", userId);
```



SECURITY & BEST PRACTICES

Q435. (Microsoft – Security Round)

What is SQL Injection and how to prevent it?

Explanation

Attacker injects malicious SQL via input to steal or damage data.

Prevention

- Parameterized queries
 - Stored procedures
 - ORM like EF Core
 - Input validation
-



SCALABILITY & DESIGN

Q436. (Amazon – Architecture Round)

Vertical vs Horizontal scaling

Vertical Scaling	Horizontal Scaling
Upgrade same server	Add more servers
Limited growth	Highly scalable
Easier	Complex



REAL INTERVIEW SCENARIOS

Q437. (Deloitte – Scenario Based)

Deadlock in SQL – what is it?

Two transactions wait on each other forever.

Example:

- Transaction A locks row1, waits row2

- Transaction B locks row2, waits row1

System halts.

Q438. (Microsoft – Advanced Round)

How to reduce deadlocks?

- Keep transactions short
 - Access tables in same order
 - Use proper indexes
 - Avoid unnecessary locks
-



FINAL SQL QUESTIONS

Q439. (Accenture – Final Tech)

Most common SQL performance mistake

Missing indexes and poorly written joins.

Q440. (ALL MNCs – FINAL DATABASE QUESTION)

How do you design a scalable database?

Best Interview Answer

“By designing a proper schema, using correct normalization, indexing wisely, managing transactions carefully, planning for scaling, handling concurrency properly, and always balancing performance with data integrity.”

◆ Advanced SQL & Optimization (Q441–Q480)

(Very important for 3–8+ years roles – Amazon, Microsoft, Deloitte, Accenture)

QUERY OPTIMIZATION BASICS (Q441–Q480)

Q441. (Microsoft – Technical Round)

What is a query execution plan and why is it important?

Easy explanation

A Query Execution Plan explains **how the database engine will execute your SQL query**. It shows:

- Which index is used
- Whether SQL uses Seek or Scan
- Join strategy (Nested Loop / Hash / Merge)
- Cost of each step

It is like Google Maps for SQL query execution.

Real project usage

- Used to find slow queries
 - Helps identify missing indexes
 - Reveals expensive operations and bottlenecks
-

Q442. (Accenture – Interview Favorite)

Index Scan vs Index Seek

Index Seek	Index Scan
Very fast	Slower
Uses index efficiently	Reads many rows
Fetches specific data	Reads entire index/table

Interview tip

Always aim for **Index Seek**.

If you see Index Scan frequently → need tuning.

Q443. (Amazon – Scenario Based)

Why does a query become slow even with index?

Explanation

Because of:

- Wrong or missing composite index
- Index fragmentation
- Outdated statistics
- Using functions on indexed columns
- Returning too much data
- Poor query design

Indexes help only when **designed correctly**.



INDEXING STRATEGIES

Q444. (Microsoft – Advanced Round)

What is a Covering Index and why is it useful?

Explanation

A Covering Index contains:

- All columns in WHERE
- All columns in SELECT

So SQL does not go back to table = **super-fast query**

SQL Example

```
CREATE INDEX idx_cover  
ON Orders(CustomerId)  
INCLUDE (OrderDate, Amount);
```

Benefit

- Reduces lookups
 - Improves performance dramatically
-

Q445. (Deloitte – Senior Round)

When should you avoid indexing?

Explanation

Avoid indexes on:

- Very small tables
- Tables with massive INSERT/UPDATE activity
- Frequently changing columns

Too many indexes slow writes.

Q446. (Accenture – Real Project)

How many indexes are too many?

Explanation

There is no fixed number.

Rule = **Balance read speed vs write cost.**

- Read-heavy → more indexes okay
- Write-heavy → fewer indexes better

Monitor performance and decide.



ADVANCED JOINS & SUBQUERIES

Q447. (Amazon – Interview Favorite)

JOIN vs Subquery – Which is better?

Explanation

- JOIN is usually faster and more optimized
- JOIN is more readable
- JOINS allow better indexing

Subqueries are fine sometimes, but **JOIN is preferred in performance queries.**

Q448. (Microsoft – Advanced Round)

Correlated Subquery – explain with example

A correlated subquery runs **for every row** of the outer query.

Example

```
SELECT *  
FROM Orders o  
WHERE Amount > (  
    SELECT AVG(Amount)  
    FROM Orders  
    WHERE CustomerId = o.CustomerId  
);
```

Useful but slower than window functions usually.

WINDOW FUNCTIONS (VERY IMPORTANT)

Q449. (Amazon – DSA + SQL Round)

What are window functions?

Explanation

Window functions allow calculations **across rows without grouping data**.

Examples:

- Ranking
- Running totals
- Moving averages

They use:

```
OVER (PARTITION BY ... ORDER BY ...)
```

Q450. (Microsoft – Interview Favorite)

ROW_NUMBER vs RANK vs DENSE_RANK

Function	Gap on duplicates?
ROW_NUMBER	Yes
RANK	Yes
DENSE_RANK	No gaps

Q451. (Accenture – Real Project)

Find duplicate records using window function

```
SELECT *  
FROM (  
    SELECT *,  
    ROW_NUMBER() OVER(PARTITION BY Email ORDER BY Id) rn  
    FROM Users  
) t  
WHERE rn > 1;
```

Used in data cleanup & migration.



COMMON TABLE EXPRESSIONS (CTE)

Q452. (Microsoft – Technical Round)

What is a CTE and why use it?

Explanation

CTE (Common Table Expression):

- Makes queries cleaner

- Improves readability
- Supports recursion

Better alternative to long subqueries.

Q453. (Amazon – Coding Round)

Recursive CTE – real example

```
WITH OrgCTE AS (  
    SELECT Id, ManagerId FROM Employees WHERE ManagerId IS NULL  
    UNION ALL  
    SELECT e.Id, e.ManagerId  
    FROM Employees e  
    JOIN OrgCTE o ON e.ManagerId = o.Id  
)  
SELECT * FROM OrgCTE;
```

Used in:

- Organization hierarchies
 - Tree structures
 - Category hierarchies
-



TRANSACTIONS & LOCKING

Q454. (Microsoft – Advanced)

What causes SQL deadlocks?

Explanation

Deadlocks happen when:

- Transaction A locks resource 1, waits for resource 2
- Transaction B locks resource 2, waits for resource 1

Both wait forever.

Q455. (Accenture – Scenario Based)

How to reduce locking issues?

- Use proper indexes
 - Keep transactions short
 - Access tables in same order
 - Avoid long-running transactions
-

Q456. (Amazon – Senior Round)

What is READ COMMITTED SNAPSHOT and why use it?

Explanation

It reduces blocking by using **row versioning**.

- Readers don't block writers
- Writers don't block readers

Improves concurrency in high load systems.



PERFORMANCE TUNING

Q457. (Microsoft – Advanced Round)

Why is **SELECT *** bad in production?

Explanation

Because:

- Fetches unnecessary columns
- Breaks covering indexes
- Increases network load

Always select only required columns.

Q458. (Deloitte – Senior Round)

Why does parameter sniffing cause problems?

Explanation

SQL optimizes the execution plan using the first **parameter**.
If later inputs differ heavily → performance drops.

Q459. (Amazon – Architecture)

How to fix parameter sniffing?

- Use **OPTION (RECOMPILE)**
 - Use **optimize for unknown**
 - Parameterize smartly
-



Q460. (Microsoft – Design Round)

OLTP vs OLAP

OLTP	OLAP
Day-to-day transactions	Analytics & reporting
Small + frequent writes	Big read queries
Banking, ecommerce	Data warehouse

Q461. (Accenture – Senior Round)

When to denormalize data?

- Read-heavy systems
- Analytics
- Reporting
- Performance-first systems

Trade space for speed.



BATCHING & BULK OPERATIONS

Q462. (Amazon – Performance)

Why are batch inserts faster?

- Fewer network trips
- Reduced overhead

- Efficient transaction handling
-

Q463. (Microsoft – Real Project)

Bulk insert from C# example

```
using SqlBulkCopy bulk = new SqlBulkCopy(conn);  
bulk.WriteToServer(table);
```

Used in:

- Data migration
 - Import large files
-



CLOUD & MODERN SQL

Q464. (Accenture – Cloud Round)

SQL Server vs Azure SQL

Azure SQL:

- Managed service
- Auto scaling
- Built-in backup
- Security + HA

Perfect for cloud environments.

Q465. (Amazon – Architecture)

Why is connection pooling important?

Because opening a DB connection is expensive.

Pooling:

- Reuses connections
 - Improves performance
 - Reduces resource usage
-



DATA CONSISTENCY & RELIABILITY

Q466. (Microsoft – Advanced)

Soft delete vs Hard delete

Soft Delete	Hard Delete
Marks record deleted	Physically removes
Recoverable	Permanent
Good for audit	Saves space

Q467. (Deloitte – Senior Round)

How to archive old data safely?

- Move to archive tables
 - Partition tables
 - Use backup and purge strategies
-



INTERVIEW SCENARIOS

Q468. (Amazon – Scenario Based)

The table has millions of records and is slow – what steps?

- Check indexes
 - Check execution plan
 - Avoid SELECT *
 - Archive old data
 - Partition if needed
-

Q469. (Accenture – Debugging)

Query fast in SSMS but slow in application – why?

- Parameter sniffing
 - Network latency
 - Different SET options
 - Different execution context
-



FINAL ADVANCED SQL QUESTIONS

Q470. (Microsoft – Final Round)

Biggest SQL optimization you did

Q471. (Deloitte – Leadership)

How you guide juniors on SQL performance

Q472. (Amazon – Reflection)

SQL mistake you made and learned from

Q473. (Accenture – Senior Round)

Trade-off between consistency and performance

Q474. (Microsoft – Architecture)

CAP theorem relevance to databases

Q475. (Amazon – Cloud)

How cloud databases scale automatically

Q476. (Deloitte – Security)

Encrypting data at rest and in transit

Q477. (Accenture – Compliance)

Handling PII data in databases

Q478. (Microsoft – Advanced)

How indexing impacts cloud cost

Q479. (Amazon – Final Reflection)

One SQL topic every backend dev must master

Answer: Indexing + Query Plans.

Q480. (ALL MNCs – FINAL ADVANCED SQL QUESTION)

How do you ensure database performance at scale?

Best Interview Answer

“By designing the right schema, indexing correctly, avoiding bad queries, monitoring execution plans, batching operations, handling concurrency properly, tuning continuously, and making database decisions based on real production usage.”

CHAPTER 6 – REACT (MNC INTERVIEW LEVEL)

◆ React Fundamentals & Core Concepts (Q481–Q520)

(Highly asked for Full-Stack roles – 1 to 8+ years)

S / Infosys – Screening Round)

What is React and why do companies use it?

Easy, real explanation

React is a JavaScript library used to build **fast, dynamic, and interactive user interfaces**. Instead of reloading the full page, React updates only the changed part of the UI.

Companies love React because

- UI updates are extremely fast
- Code is reusable (components)
- Easy to scale large applications
- Huge community + ecosystem

Real-world examples

- Admin dashboards
 - Social media feeds
 - E-commerce websites
 - SaaS dashboards
-

Q482. (Accenture – Technical Round)

What problem does React solve compared to plain JavaScript?

Explanation

Without React:

- Developers manually update DOM
- Managing UI state becomes messy
- Code becomes complex very fast

With React:

- UI auto-updates based on state
- Cleaner and maintainable code
- Predictable UI changes

Real example

Cart count auto updates when user adds item
→ Developer does not manually refresh DOM.



COMPONENTS (VERY IMPORTANT)

Q483. (Microsoft – Interview Favorite)

What is a component in React?

Explanation

A component is a **reusable UI block** that contains:

- UI (JSX)
- Logic
- State (if needed)

It is like a Lego block used to build UI.

Real React Example

```
function Welcome({ name }) {  
  return <h2>Welcome {name}</h2>;  
}
```

Real project usage

Header, Navbar, Sidebar, Footer, Cards, Forms

Q484. (Infosys – Technical Round)

Functional vs Class Components

Functional Components	Class Components
Simple	More complex
Uses Hooks	Uses lifecycle methods
Preferred in modern React	Old legacy approach

Interview Tip

Say confidently:

“Modern React uses functional components with Hooks.”

JSX & RENDERING

Q485. (Accenture – Interview Favorite)

What is JSX and why is it used?

Explanation

JSX allows writing **HTML-like syntax inside JavaScript**, making UI easier to create and understand.

Example

```
const element = <h1>Hello React</h1>;
```

It improves readability and developer experience.

Q486. (Microsoft – Technical Round)

How does JSX work behind the scenes?

Explanation

JSX is not understood by the browser.
During build, JSX is converted into:

```
React.createElement()
```

which generates Virtual DOM objects.



STATE & PROPS (MOST ASKED)

Q487. (Amazon – Interview Favorite)

What is the state of React?

Explanation

State is **data that can change over time** and directly affects the UI.

Examples

- Cart items
- Logged-in user
- Form input value

Code

```
const [count, setCount] = useState(0);
```

Q488. (Infosys – Technical Round)

Props vs State

Props	State
Read-only	Mutable
Passed from parent	Managed inside component
Used to communicate	Used to store internal data

Q489. (Accenture – Real Project)

Why should state not be mutated directly?

Explanation

If you change state directly, React **cannot detect the change**, so the UI will not update correctly.

Wrong

```
count++;
```

Correct

```
setCount(count + 1);
```



LIFECYCLE & HOOKS

Q490. (Microsoft – Interview Favorite)

What are Hooks and why were they introduced?

Explanation

Hooks allow functional components to:

- Manage state
- Handle lifecycle
- Use reusable logic

They replaced complex class components.

Q491. (Amazon – Technical Round)

Explain useEffect with real example

Explanation

`useEffect` runs **side effects** such as:

- API calls
- Subscriptions
- Timers

Example

```
useEffect(() => {  
  fetchUsers();  
}, []);
```

Real project usage

Load data when page loads.

Q492. (Accenture – Interview Trap)

What happens if the dependency array is missing in `useEffect`?

Explanation

The effect runs **on every render**, causing:

- Performance issues
- Infinite loops in some cases

So always use dependency arrays wisely.



EVENT HANDLING

Q493. (Infosys – Technical Round)

How do events work in React?

Explanation

React uses **Synthetic Events** to provide consistent behavior across browsers.

Example

```
<button onClick={handleClick}>Click</button>
```



CONTROLLED COMPONENTS

Q494. (Microsoft – Interview Favorite)

What is a controlled component?

Explanation

A form element controlled by React state.

Example

```
<input value={name} onChange={e => setName(e.target.value)} />
```

Real usage

Forms, login pages, validations.



CONDITIONAL RENDERING

Q495. (Accenture – Developer Round)

How to render UI conditionally in React?

Example

```
{isLoggedIn && <Dashboard />}
```

Used in authentication flows frequently.

KEYS & LIST RENDERING

Q496. (Amazon – Interview Favorite)

Why are keys important in lists?

Explanation

Keys help React:

- Identify which items changed
- Improve rendering performance

Example

```
items.map(item => <li key={item.id}>{item.name}</li>);
```

PERFORMANCE CONCEPTS

Q497. (Microsoft – Advanced Round)

What is Virtual DOM?

Explanation

Virtual DOM is:

- A lightweight copy of Real DOM
- Stored in memory

- Used to detect changes efficiently

React compares old vs new Virtual DOM → updates only changed parts.

Q498. (Accenture – Interview Favorite)

Why is React faster than traditional DOM manipulation?

Explanation

Because of:

- Virtual DOM
- Diffing algorithm
- Efficient re-rendering strategy

No unnecessary page reloads.



REAL INTERVIEW SCENARIOS

Q499. (Amazon – Scenario Based)

Page re-renders too often – how do you fix it?

Explanation

- Use `React.memo`
- Use `useMemo`
- Use `useCallback`
- Optimize dependency arrays

- Avoid unnecessary state
-

Q500. (Microsoft – Final Round)

How do you structure React projects in enterprise apps?

Best Interview Answer

“By organizing code into clear folders such as components, pages, services, hooks, contexts, assets, and utilities; following reusable component design, using proper naming conventions, and ensuring performance and maintainability from day one.”

◆ Advanced React – Hooks, Context & Performance (Q501–Q540)

(Asked in Amazon, Microsoft, Deloitte, Accenture, Infosys – 2+ to 8+ years)

ADVANCED HOOKS (CORE INTERVIEW AREA)

(Q501–Q520)

Q501. (Microsoft – Interview Favorite)

Why were Hooks introduced in React?

Easy explanation

Before Hooks, React relied heavily on **class components**, which had problems like:

- Complex lifecycle methods
- Hard reusable logic
- “this” confusion

- Larger boilerplate code

Hooks solved all this by allowing:

- State in functional components
- Lifecycle handling in functions
- Reusable logic through custom hooks
- Cleaner and shorter code

Real-world impact

- Cleaner and maintainable applications
 - Easier testing
 - Faster development
 - Better performance
-

Q502. (Accenture – Technical Round)

Explain useState with a real production example

Explanation

`useState()` helps store and update component-specific data.

Real example – Login Form

```
const [email, setEmail] = useState("");

<input
  value={email}
  onChange={e => setEmail(e.target.value)}
/>
```

Real usage in production

- Login forms
 - Search filters
 - Cart updates
 - Toggle themes
 - Modal open/close states
-

Q503. (Amazon – Interview Favorite)

Why are state updates asynchronous?

Detailed Explanation

React **batches state updates** to improve performance. Instead of updating UI every time instantly, React:

- Groups multiple updates together
- Updates UI once efficiently

So the latest value may not be immediately available.

Correct Pattern

```
setCount(prev => prev + 1);
```

This ensures the update is reliable.

Q504. (Microsoft – Advanced Round)

What problem does `useEffect` solve?

Explanation

`useEffect()` is used to handle **side effects** like:

- API calls
- Logging
- Subscriptions
- Timers

Anything that is outside normal UI rendering logic.

Real example

```
useEffect(() => {  
  fetchUsers();  
}, []);
```

Without useEffect

Developers would need complex lifecycle methods.

Q505. (Accenture – Interview Trap)

Why does an infinite loop happen in useEffect?

Explanation

Infinite loops happen when:

- There is no dependency array
- Or wrong dependencies are used
- State is updated continuously inside the effect

Wrong

```
useEffect(() => {  
  setData(data + 1);  
});
```

Correct

Use dependency arrays wisely.

Q506. (Amazon – Real Project)

Cleanup function in useEffect – why is it important?

Explanation

Cleanup prevents:

- Memory leaks
- Multiple subscriptions
- Performance issues

Real example

```
useEffect(() => {  
  const timer = setInterval(fetchData, 5000);  
  return () => clearInterval(timer);  
}, []);
```

Real usage

- Clearing intervals
 - Removing event listeners
 - Cancelling API calls
-



CUSTOM HOOKS (VERY IMPORTANT)

Q507. (Microsoft – Interview Favorite)

What is a custom hook and why do we need it?

Explanation

Custom Hooks allow **reusing logic** without copying code.
They do NOT reuse UI — only functionality.

Example

```
function useOnlineStatus() {  
  const [isOnline, setIsOnline] = useState(true);  
  return isOnline;  
}
```

Real usage

- Authentication
- API fetching
- Permissions
- Theme management



USECALLBACK & USEMEMO

Q508. (Amazon – Performance Round)

Why is useCallback used?

Explanation

`useCallback()` prevents **unnecessary function recreation** and re-renders.

Real example

```
const handleClick = useCallback(() => {  
  setCount(c => c + 1);  
}, []);
```

Real-world use

Passing functions to:

- Child components
 - Optimized components
 - List rendering components
-

Q509. (Microsoft – Advanced)

Difference between useCallback and useMemo

useCallback	useMemo
Memoizes function	Memoizes computed value
Prevents re-creation of functions	Prevents expensive recalculations
Used with event handlers	Used for heavy calculations

Q510. (Accenture – Interview Favorite)

Real case where useMemo improves performance

Explanation

When filtering large datasets or running heavy calculations.

Example

```
const filtered = useMemo(() => {  
  return users.filter(u => u.active);  
}, [users]);
```

Real impact

- Faster UI
- Smooth dashboards

- Better user experience
-

CONTEXT API (STATE MANAGEMENT)

Q511. (Microsoft – Interview Favorite)

What problem does Context API solve?

Explanation

It avoids **prop drilling**, where props must be passed through multiple levels unnecessarily.

Real-world example

- Authentication user data
 - Theme (dark/light)
 - Language setting
-

Q512. (Accenture – Real Project)

Create AuthContext – real example

```
const AuthContext = createContext();

function AuthProvider({ children }) {
  const [user, setUser] = useState(null);

  return (
    <AuthContext.Provider value={{ user, setUser }}>
      {children}
    </AuthContext.Provider>
  );
}
```


}

Q513. (Amazon – Technical Round)

How to consume Context safely?

```
const { user } = useContext(AuthContext);
```

Q514. (Microsoft – Advanced Round)

Why Context causes re-render issues?

Explanation

When context value changes:

- All consumers re-render
- Even components not using affected fields

This can hurt performance.



PERFORMANCE OPTIMIZATION

Q515. (Accenture – Interview Favorite)

What is React.memo and when to use it?

Explanation

Prevents re-render when props do not change.

Example

```
const Header = React.memo(({ title }) => <h1>{title}</h1>);
```

Use when

- Pure functions
 - Static content
 - Performance-critical UI
-

Q516. (Amazon – Senior Round)

Why is premature optimization bad?

Explanation

- Increases complexity
- Makes debugging harder
- May not provide real benefit

Optimize only when:

- There is performance issue
 - Metrics prove it
-



RENDERING & RECONCILIATION

Q517. (Microsoft – Advanced)

How does React decide to re-render components?

Explanation

React re-renders when:

- State changes
 - Props change
 - Parent re-renders
 - Context value changes
-



REAL PRODUCTION SCENARIOS

Q518. (Amazon – Scenario Based)

Dashboard slow with 10k records – what to do?

Solutions

- Pagination
 - Infinite scrolling
 - Virtualized lists (React Window / React Virtualized)
 - Memoization
 - Avoid unnecessary state
-

Q519. (Deloitte – Senior Round)

Context vs Redux – when Context is not enough?

Explanation

Use Context for:

- Small apps

- Few states

Use Redux for:

- Complex state
- Global logic
- Async data flows
- Predictable state management



FINAL ADVANCED REACT QUESTION

Q520. (ALL MNCs – FINAL REACT QUESTION)

How do you write scalable React code?

Best Interview Answer

“By keeping components small and reusable, using hooks to share logic, following proper folder structure, avoiding unnecessary re-renders with memoization, preferring Context or Redux wisely, and optimizing only when performance actually matters.”

♦ Authentication, API Integration & Forms (Q541–Q580)

(Very important for Full-Stack roles – Accenture, Deloitte, Amazon, Microsoft, Infosys)



AUTHENTICATION IN REACT (REAL PROJECT WAY)

Q541 – Q580 (Deep, Practical, Interview-Ready)

Q541. (Microsoft – Interview Favorite)

How authentication works in a React application?

Easy Explanation

React **does not authenticate users itself**. Authentication is always managed at the backend.

React simply calls APIs and stores tokens securely.

Real Flow

- 1 User enters credentials in login form
- 2 React sends them to Backend API
- 3 Backend verifies credentials and generates JWT
- 4 Backend returns JWT token to React
- 5 React stores the token securely
- 6 React attaches token to every API call
- 7 If token valid → user is authenticated

Real-World Uses

Banking dashboards, admin portals, CRM systems, SaaS dashboards.

Q542. (Accenture – Technical Round)

Where should JWT token be stored in React?

Explanation

Best professional storage options:

✓ HTTP-Only Cookies (Most Secure)

- Cannot be accessed by JavaScript
- Protects against XSS

✓ In-Memory (Redux / Context)

- Secure but lost on refresh
- Common in SPAs + refresh token strategy

✗ Avoid

LocalStorage / sessionStorage for critical apps because they are vulnerable to XSS.

Q543. (Amazon – Interview Favorite)

Login API call – Real React Example

```
async function login(email, password) {
  const res = await fetch("/api/login", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ email, password })
  });

  const data = await res.json();
  return data.token;
}
```

Real Interview Tip

Mention:

- JSON handling
 - Error cases
 - Token handling afterward
-

Q544. (Microsoft – Advanced Round)

How do you attach a token to every API request?

Real-Time Fetch Wrapper

```
function authFetch(url) {
  const token = getToken();
  return fetch(url, {
    headers: {
```

```
    Authorization: `Bearer ${token}`  
  }  
});  
}
```

Real Project Usage

Create **centralized API service** and never attach a token manually in each call.

Q545. (Accenture – Interview Favorite)

How do you protect routes in React?

Explanation

Use **Protected Routes** so only authenticated users access secure pages.

```
function PrivateRoute({ children }) {  
  return isAuthenticated() ? children : <Navigate to="/login" />;  
}
```

Real Project Use

Admin dashboards, user profile pages, payment pages.

API INTEGRATION

Q546. (Infosys – Technical Round)

Best way to call APIs in React?

Explanation

Two preferred ways:

- ✓ `fetch()` – Native & lightweight
- ✓ `axios` – Popular, powerful, cleaner

Interview Tip

Always mention:

- Interceptors
 - Centralized service layer
-

Q547. (Microsoft – Interview Favorite)

Why is axios preferred over fetch?

Explanation

- Automatic JSON parsing
- Request/Response interceptors
- Better error handling
- Default headers support

Used heavily in enterprise systems.

Q548. (Amazon – Real Project)

Axios instance with interceptor

```
import axios from "axios";

const api = axios.create({
  baseURL: "/api"
});

api.interceptors.request.use(config => {
  config.headers.Authorization = `Bearer ${getToken()}`;
  return config;
});
```



```
});
```

Benefit

You never manually pass a token in every call.

Q549. (Accenture – Scenario Based)

How do you handle API loading & error states?

Explanation

Maintain UI states:

- loading
- error
- data

```
const [loading, setLoading] = useState(false);
```

Why?

Improves UX and debugging.

Q550. (Microsoft – Best Practice)

Why should API calls NOT be inside render?

Explanation

Because React keeps re-rendering → will trigger endless API calls.

Correct Approach

Use `useEffect()`.



ERROR HANDLING & TOKEN EXPIRY

Q551. (Amazon – Advanced)

How do you handle API errors globally?

Professional Approach

- Axios interceptors
 - Centralized toast notifications
 - Meaningful error messages
-

Q552. (Deloitte – Senior Round)

How to handle token expiry in React?

Explanation

- 1 Detect **401 Unauthorized**
- 2 Request **refresh token**
- 3 Generate new JWT
- 4 Retry original request

Production applications always use Refresh Tokens.



FORMS (Core Interview Area)

Q553. (Infosys – Interview Favorite)

Controlled vs Uncontrolled Components

Controlled

Uncontrolled

React controls value	Browser controls value
----------------------	------------------------

More reliable	Less control
---------------	--------------

Best for validation	Simple cases only
---------------------	-------------------

Q554. (Accenture – Technical Round)

Simple Controlled Form Example

```
const [name, setName] = useState("");  
  
<input  
  value={name}  
  onChange={e => setName(e.target.value)}  
>
```

Q555. (Microsoft – Interview Favorite)

Why controlled forms in enterprise?

Explanation

- Full validation control
 - Better UX
 - Custom rules & formatting
 - Prevents inconsistent data
-



FORM VALIDATION

Q556. (Amazon – Real Project)

How do you validate forms in React?

Options

- ✓ Manual validation
 - ✓ Formik + Yup (industry standard)
-

Q557. (Microsoft – Advanced Round)

Formik Example

```
<Formik
  initialValues={{ email: "" }}
  onSubmit={values => login(values)}
>
```

Q558. (Accenture – Interview Favorite)

Why Yup with Formik?

Explanation

- Schema based validation
 - Reusable rules
 - Clean and readable
-



FORM SUBMISSION & UX

Q559. (Infosys – Technical Round)

Prevent multiple submissions

```
<button disabled={loading}>Submit</button>
```

Q560. (Amazon – UX Focused)

Why an optimistic UI?

Explanation

The user feels the application is fast and responsive.

SECURITY BEST PRACTICES

Q561. (Microsoft – Security Round)

Prevent XSS in React

- Avoid `dangerouslySetInnerHTML`
 - Sanitize dynamic HTML
 - Validate user inputs
-

Q562. (Accenture – Security)

CSRF in React

Handled mostly by backend + secure cookies + tokens.

REAL INTERVIEW CASES

Q563. (Amazon – Scenario Based)

User refreshes & gets logged out – why?

Token stored in **memory only**.

Q564. (Deloitte – Senior Round)

How to persist login?

- HTTP only cookies
 - Secure storage
 - Refresh tokens
-

PERFORMANCE

Q565. (Microsoft – Advanced)

Why debounce search APIs?

Prevents hitting API on every keystroke.

Q566. (Accenture – Real Project)

Debounce Example

```
const debouncedSearch = debounce(searchUsers, 300);
```

ARCHITECTURE DECISIONS

Q567. (Amazon – Architecture Round)

Where should API logic live?

In a separate **service layer**, not components.

Cleaner. Maintainable. Testable.

FINAL INTERVIEW QUESTIONS

Q568. (Microsoft – Final Round)

How do you design secure auth?

- JWT + Refresh tokens
 - Secure cookies
 - Protected routes
 - Centralized API layer
 - Role-based UI
-

Q569. (Accenture – HR + Tech)

Biggest Auth Bug Answer

“Stored token in localStorage, exposed to XSS. Moved to HttpOnly cookies → secured application.”

Q570. (Amazon – Final)

Explain full login flow confidently (already covered above).

Q571–Q579

Handled via:

- Multi-step forms
 - File uploads
 - Role-based content
 - Global logout
 - Refresh tokens
-

Q580. (ALL MNCs – Final React Question)

How to build secure, scalable React apps?

Best Interview Answer

“By separating concerns, securing tokens wisely, handling API errors globally, managing state efficiently, following best security practices, and continuously optimizing performance based on real usage.”

CHAPTER 7 – ANGULAR (ENTERPRISE & MNC LEVEL)

♦ Angular Fundamentals & Architecture (Q581–Q620)

(Highly asked for Full-Stack roles – 2 to 10+ years)



ANGULAR BASICS (Q581 – Q620)

Q581. (Infosys / TCS – Screening Round)

What is Angular and why do enterprises use it?

Easy Explanation

Angular is a **full-fledged front-end framework** developed and maintained by Google. It is not just a UI library like React; it provides everything required to build large-scale applications.

Why Enterprises Love Angular

- ✓ Strong architecture
- ✓ Built-in tools (routing, forms, DI, HTTP, validation)
- ✓ TypeScript = safer & maintainable code
- ✓ Highly scalable for big teams

Real-World Usage

Banking dashboards, government portals, healthcare systems, ERP platforms.

Q582. (Accenture – Technical Round)

Angular vs React – when Angular is preferred?

Explanation

Choose Angular when:

- ✓ Large enterprise teams
- ✓ Strict coding standards needed
- ✓ Complex forms, validations, workflows
- ✓ Long-term maintainability required

Summary

React = flexible

Angular = structured enterprise framework

ANGULAR ARCHITECTURE

Q583. (Microsoft – Interview Favorite)

Explain Angular architecture in simple terms

Explanation

An Angular app is built using:

- **Modules** → Organize features
- **Components** → UI + logic
- **Services** → Business logic / API calls
- **Dependency Injection** → Provides services automatically

Everything is predictable and structured.

Q584. (Deloitte – Senior Round)

What is a Module in Angular?

Explanation

A module groups related features together.

```
@NgModule({  
  declarations: [UserComponent],  
  imports: [CommonModule],  
})  
export class UserModule {}
```

Real Use

Create feature modules:

- UserModule
- AdminModule
- OrdersModule

Makes enterprise applications maintainable.



COMPONENTS (CORE OF ANGULAR)

Q585. (Infosys – Interview Favorite)

What is a Component in Angular?

Explanation

A component controls:

- The HTML UI
- CSS styling
- Logic / behavior

Example

```
@Component({
  selector: 'app-user',
  template: `<h2>{{name}}</h2>`
})
export class UserComponent {
  name = 'John';
}
```

Q586. (Accenture – Developer Round)

Component vs Directive

Component	Directive
Has template	No template
Creates UI block	Modifies behavior



DATA BINDING (MOST ASKED)

Q587. (Microsoft – Interview Favorite)

Types of Data Binding in Angular

- ✓ Interpolation → `{{ value }}`
 - ✓ Property Binding → `[value]=" "`
 - ✓ Event Binding → `(click)=" "`
 - ✓ Two-way Binding → `[(ngModel)]`
-

Q588. (Infosys – Technical Round)

Two-way Binding Example

```
<input [(ngModel)]="username" />
<p>{{username}}</p>
```

Usage

Forms, real-time input handling.



DIRECTIVES

Q589. (Accenture – Interview Favorite)

Structural vs Attribute Directives

Structural	Attribute
Change DOM structure	Change appearance/behavior
<code>*ngIf, *ngFor</code>	<code>ngClass, ngStyle</code>

Q590. (Microsoft – Technical Round)

ngIf vs hidden

✓ `ngIf` → Removes element from DOM

✓ `hidden` → Only hides visually

Performance Tip:

Say confidently → `ngIf` is better for optimization.

SERVICES & DEPENDENCY INJECTION

Q591. (Amazon – Interview Favorite)

What is a Service?

Explanation

Service = Business logic holder
NOT responsible for UI.

```
@Injectable()
export class UserService {
  getUsers() {}
}
```

Used for:

- Reusable logic
 - API communication
 - Shared state
-

Q592. (Accenture – Technical Round)

What is Dependency Injection?

Simple Explanation

Instead of creating objects manually, Angular automatically injects them.

- ✓ Cleaner
 - ✓ Testable
 - ✓ Maintainable
-



HTTP & API INTEGRATION

Q593. (Microsoft – Interview Favorite)

How does Angular call backend APIs?

Using **HttpClient Service**

```
this.http.get('/api/users');
```

Q594. (Infosys – Technical Round)

Observable vs Promise

Observable	Promise
Multiple values	Single value
Cancelable	Not cancelable
Built into Angular via RxJS	



ROUTING

Q595. (Accenture – Developer Round)

What is Angular Routing?

Loads components based on URL.

```
{ path: 'users', component: UserComponent }
```

Q596. (Microsoft – Interview Favorite)

Lazy Loading – Why Important?

- ✓ Improves performance
- ✓ Loads modules only when required
- ✓ Reduces initial bundle size

Example:

Admin module loads only for admins.



GUARDS & SECURITY

Q597. (Amazon – Scenario Based)

How to protect routes?

Use Route Guards.

```
canActivate(): boolean {  
  return this.authService.isLoggedIn();  
}
```



FORMS (ENTERPRISE MUST)

Q598. (Infosys – Interview Favorite)

Template vs Reactive Forms

Template Driven	Reactive
Simple	Scalable
HTML based	Code driven
Beginner friendly	Enterprise level

Q599. (Microsoft – Advanced Round)

Reactive Form Example

```
this.form = this.fb.group({  
  email: ['', Validators.required]  
});
```



LIFECYCLE HOOKS

Q600. (Accenture – Interview Favorite)

Important Lifecycle Hooks

- `ngOnInit()` → initialization
 - `ngOnDestroy()` → cleanup
-



PERFORMANCE OPTIMIZATION

Q601. (Microsoft – Senior Round)

What is Change Detection?

Angular checks when data changes and updates the UI.

Q602. (Amazon – Performance Round)

What is OnPush and why use it?

- ✓ Runs fewer change checks
 - ✓ Better performance
 - ✓ Ideal for large enterprise apps
-



REAL ENTERPRISE SCENARIOS

Q603. (Deloitte – Scenario Based)

A large Angular app is slow – What to do?

- Use Lazy Loading
 - Enable OnPush
 - Use trackBy in lists
 - Reduce unnecessary watchers
-

Q604. (Accenture – Interview Favorite)

Why use TrackBy in ngFor?

Prevents Angular from re-rendering the entire list.



ERROR HANDLING & SECURITY

Q605. (Microsoft – Advanced)

Global Error Handling

Use `ErrorHandler`.

Q606. (Amazon – Security Round)

How Angular prevents XSS?

- ✓ Built-in DOM sanitization
 - ✓ Auto-escapes HTML
-



PROJECT STRUCTURE

Q607. (Infosys – Interview Favorite)

Enterprise Angular Structure

- ✓ Feature modules
 - ✓ Shared module
 - ✓ Core services
 - ✓ Clean folder separation
-



INTERVIEW EXPERIENCE

Q608. (Accenture – Senior Round)

Explain biggest Angular performance issue & fix
(You can answer using Lazy loading + OnPush + optimizing lists)

Q609. (Microsoft – Final Round)

Angular vs React → choose logically based on project size, team, and flexibility.

FINAL ANGULAR QUESTIONS

Q610 – Q619

Covers:

- Role-based UI
 - Token handling
 - API interceptors
 - Form validation
 - Lazy loading challenges
-

Q620. (ALL MNCs – FINAL ANGULAR QUESTION)

How do you build scalable Angular apps?

Best Interview Answer

“By using modular architecture, lazy loading, reactive forms, dependency injection, strong typing, optimized change detection, and following Angular best practices.”

CHAPTER 8 – CI/CD, DOCKER & AZURE (DEVOPS FOR FULL-STACK)

◆ Enterprise DevOps Concepts (Q621–Q680)

(Very important for 3–10+ years Full-Stack roles)



CI/CD FUNDAMENTALS (Q621 – Q650)

Q621. (Infosys / TCS – Screening Round)

What is CI/CD in simple words?

Easy Explanation

CI/CD means **automating build, testing, and deployment** so software reaches users safely and faster.

✓ CI – Continuous Integration

Developers frequently merge code → code is built & tested automatically.

✓ CD – Continuous Deployment

Code is automatically deployed to environments without manual steps.

Real-World Example

Every code push triggers pipeline:

Build → Test → Quality checks → Deploy to Server

No manual copying files, no “it works on my machine” mistakes.

Q622. (Accenture – Interview Favorite)

Difference between CI and CD

CI (Continuous Integration)	CD (Continuous Deployment)
------------------------------------	-----------------------------------

Frequently merging code

Automatically deploying code

Builds + runs tests

Releases to environments

Developer-focused

Operations/Release-focused

Interview Tip:

Say: "CI/CD reduces manual effort and Easy error."

Q623. (Microsoft – Technical Round)

Why is CI/CD critical in enterprise projects?

Explanation

- ✓ Faster release cycles
- ✓ Early bug detection
- ✓ Consistent deployments
- ✓ Easy rollback and recovery
- ✓ Improves team productivity

Strong Interview Line

"CI/CD ensures reliability, confidence, and repeatable deployments."



BUILD & PIPELINE CONCEPTS

Q624. (Amazon – DevOps Round)

What happens in a typical CI pipeline?

Explanation

- 1 Code checkout
- 2 Restore dependencies
- 3 Build application
- 4 Run tests

5 Code quality scan

6 Generate artifacts

Artifacts = packaged build ready for deployment.

Q625. (Accenture – Technical Round)

Example: .NET CI pipeline steps

```
dotnet restore
```

```
dotnet build
```

```
dotnet test
```

```
dotnet publish
```

Real Usage

Executed through:

- Azure DevOps
 - GitHub Actions
 - Jenkins
-



GIT & BRANCHING STRATEGY

Q626. (Microsoft – Interview Favorite)

What branching strategy do you use?

Explanation

Most enterprises use:

✓ **GitFlow**

- develop

- feature branches
- release
- hotfix

✓ **Trunk-based development**

Small frequent merges → continuous delivery.

Q627. (Deloitte – Senior Round)

Why are Pull Requests mandatory in MNCs?

Explanation

Pull Requests ensure:

- ✓ Code review
- ✓ Quality checks
- ✓ Security scans
- ✓ Peer validation

Result → Better code + fewer bugs.



DOCKER (VERY IMPORTANT)

Q628. (Amazon – Interview Favorite)

What is Docker and why do we need it?

Easy Explanation

Docker packages:

Application + Dependencies + Runtime → into a container.

So the app runs the **same everywhere**.

It kills the problem:

“Works on my machine but not on the server.”

Q629. (Microsoft – Technical Round)

Difference between Virtual Machine and Docker

Virtual Machine	Docker
Heavy	Lightweight
Separate OS	Shares OS
Slow startup	Fast startup
More resource usage	Less resource usage

Q630. (Accenture – Developer Round)

What is a Dockerfile?

A Dockerfile defines how to build an image.

Real Example (.NET API)

```
FROM mcr.microsoft.com/dotnet/aspnet:7.0
WORKDIR /app
COPY . .
ENTRYPOINT ["dotnet", "MyApi.dll"]
```

Q631. (Microsoft – Advanced Round)

Docker Image vs Container

- ✓ Image → Blueprint or template
 - ✓ Container → Running instance of the image
-



DOCKER REAL PROJECT SCENARIOS

Q632. (Amazon – Scenario Based)

Why is Docker useful in microservices?

Explanation

Each service:

- ✓ Runs independently
- ✓ Has its own dependencies
- ✓ Can scale separately

Perfect for cloud & Kubernetes.

Q633. (Accenture – Interview Favorite)

How does Docker help CI/CD pipelines?

Explanation

- ✓ Same environment across Dev, QA, Prod
 - ✓ Faster deployments
 - ✓ Easy rollback
 - ✓ No configuration issues
-



AZURE BASICS (MNC MUST)

Q634. (Microsoft – Technical Round)

What is Microsoft Azure?

Explanation

Azure is a cloud platform providing:

- ✓ Compute
- ✓ Storage
- ✓ Databases
- ✓ Networking
- ✓ DevOps tools

Used to host and scale applications globally.

Q635. (Accenture – Interview Favorite)

Azure vs AWS – which one do you use and why?

Balanced Answer

Azure is best when:

- ✓ .NET applications
- ✓ Enterprise ecosystem
- ✓ Integration with Active Directory

AWS is amazing for:

- ✓ Cloud-native
 - ✓ High-scale distributed systems
-



AZURE CORE SERVICES

Q636. (Microsoft – Technical Round)

What is Azure App Service?

Explanation

Managed hosting platform for:

- Web Apps
- APIs

- Background services

No server maintenance needed.

Q637. (Amazon – Architecture Round)

Azure App Service vs Virtual Machine

App Service	Virtual Machine
-------------	-----------------

Fully managed	Full control
---------------	--------------

Auto scaling	Manual setup
--------------	--------------

Less Ops work	More Ops work
---------------	---------------



AZURE DEVOPS (VERY IMPORTANT)

Q638. (Accenture – Interview Favorite)

What is Azure DevOps?

Explanation

Azure DevOps provides:

- ✓ Repos
- ✓ Pipelines
- ✓ Boards
- ✓ Artifacts

Complete DevOps ecosystem.

Q639. (Microsoft – Technical Round)

Azure Pipeline Example (.NET)

trigger:

```
- main
```

```
steps:
```

```
- task: DotNetCoreCLI@2
```

```
  inputs:
```

```
    command: 'build'
```



DEPLOYMENT STRATEGIES

Q640. (Amazon – Senior Round)

Blue-Green vs Rolling Deployment

✓ Blue-Green

Two environments (Blue & Green)

Switch traffic instantly → zero downtime

✓ Rolling Deployment

Gradually updates instances → safer rollout



SECURITY IN DEVOPS

Q641. (Microsoft – Security Round)

How do you secure secrets in CI/CD?

✓ Azure Key Vault

✓ Pipeline secret variables

✓ Never store in code

Q642. (Accenture – Interview Favorite)

What is Azure Key Vault?

Stores:

- Passwords
- API Keys
- Certificates
- Connection Strings

Centralized & secure.



MONITORING & LOGGING

Q643. (Amazon – Observability Round)

How do you monitor applications in Azure?

- ✓ Azure Monitor
 - ✓ Application Insights
 - ✓ Alerts + Dashboards
-

Q644. (Microsoft – Technical Round)

What is Application Insights?

Tracks:

- Requests
- Errors
- Performance
- Dependencies

Helps debug production issues.



SCALING & PERFORMANCE

Q645. (Accenture – Scenario Based)

How do you scale an API in Azure?

- ✓ Horizontal scaling (scale out)
 - ✓ Auto-scale rules (CPU / Requests threshold)
-



REAL INTERVIEW SCENARIOS

Q646. (Deloitte – Scenario)

Deployment failed in production – what do you do?

- 1 Rollback safely
- 2 Check logs & pipeline
- 3 Fix issue
- 4 Redeploy

Shows maturity & ownership.

Q647. (Amazon – Senior Round)

How CI/CD reduced bugs in your project?

Talk about:

- ✓ Early testing
 - ✓ Automated quality gates
 - ✓ Consistent deployments
 - ✓ Reduced Easy mistakes
-



FINAL DEVOPS QUESTIONS

Q648. (Microsoft – Final Round)

Explain end-to-end CI/CD for .NET + React app

(Talk about Repo → Pipeline → Docker → Azure App Service → Monitoring)

Q649. (Accenture – HR + Tech)

Biggest CI/CD challenge you solved

(E.g., rollback automation, build failures, environment config issues)

Q650. (ALL MNCs – FINAL DEVOPS QUESTION)

Why is DevOps important for Full-Stack Developers?

Best Interview Answer

“Because full-stack developers must know how code is built, validated, deployed, monitored, and scaled in real production environments. CI/CD + DevOps ensures faster delivery, fewer failures, and reliable systems.”

CHAPTER 9 – CLOUD & AZURE (DEEP INTERVIEW LEVEL)

◆ Azure Architecture, Services & Real-World Scenarios (Q681–Q720)

(Very important for 3–10+ years roles)



CLOUD FUNDAMENTALS **(INTERVIEW START POINT)**

Q681. (TCS / Infosys – Screening Round)

What is Cloud Computing in simple words?

Easy explanation

Cloud computing means using **servers over the internet** instead of buying and maintaining physical machines.

You do not worry about:

- hardware
- maintenance
- electricity
- data center space

And...

You **pay only for what you use**.

Real-world example

Instead of hosting your API on a company server, you host it on **Azure App Service / AWS EC2**.

Q682. (Accenture – Interview Favorite)

IaaS vs PaaS vs SaaS with real examples

Model	Meaning	Who manages?	Azure Example
IaaS	Infrastructure as a Service	You manage OS + App	Azure VM

PaaS Platform as a Service Azure manages infra Azure App Service

SaaS Software as a Service Everything managed Office 365

Interview Tip

Say confidently:

*“Most .NET applications in enterprises prefer **PaaS** because it reduces infrastructure effort.”*



AZURE CORE ARCHITECTURE

Q683. (Microsoft – Technical Round)

What is a Resource Group in Azure?

Explanation

*A Resource Group is a **logical container** that holds Azure resources like:*

- *App Service*
- *Database*
- *Storage*
- *VM*

Real-world usage

*Delete the entire environment (API + Database + Storage) **in one click**.*

Q684. (Deloitte – Senior Round)

Azure Region vs Availability Zone

Explanation

✓ **Region**

Geographic location (e.g., East US, Central India)

✓ **Availability Zone**

*Multiple **independent data centers** inside a region.*

Why it matters

High availability

Disaster recovery

Fault tolerance

COMPUTE SERVICES (VERY IMPORTANT)

Q685. (Microsoft – Interview Favorite)

Azure App Service vs Azure VM – which to choose?

Easy explanation

Choose **App Service** when:

- *You want fast deployment*
- *Minimum infrastructure work*
- *Built-in scaling and monitoring*

Choose **VM** when:

- *You need full OS control*
- *Need to install custom software*
- *Need backend services not supported by PaaS*

Q686. (Amazon – Architecture Round)

Azure App Service real production example

Example

Host:

- *ASP.NET Core API*
- *React/Angular frontend*

With:

- *Auto scaling*
- *HTTPS*
- *Deployment slots*
- *Monitoring*



AZURE FUNCTIONS (SERVERLESS)

Q687. (Accenture – Interview Favorite)

What is Azure Function and why use it?

Explanation

Azure Function lets you run **small code pieces** without servers.

- ✓ *No server management*
- ✓ *Pay only when it runs*

Real-world example

Send email after order placed
Process background jobs
Generate reports

Q688. (Microsoft – Advanced Round)

Azure Function vs Web API

Azure Function	Web API
<i>Event-driven</i>	<i>Request-driven</i>
<i>Very cheap</i>	<i>Continuous cost</i>
<i>Serverless</i>	<i>Server-based</i>

STORAGE & DATA SERVICES

Q689. (Infosys – Technical Round)

Types of Azure Storage

- ✓ **Blob Storage** → files, images, videos
 - ✓ **Table Storage** → NoSQL structured data
 - ✓ **Queue Storage** → message communication
 - ✓ **File Storage** → shared file system
-

Q690. (Amazon – Scenario Based)

Store user-uploaded images – which Azure service?

Answer: Azure Blob Storage

Why: Cheap, scalable, secure, globally available



DATABASES IN AZURE

Q691. (Microsoft – Interview Favorite)

Azure SQL vs SQL Server on VM

Explanation

✓ Azure SQL

- *Fully managed*
- *Auto backups*
- *Auto scaling*

✓ SQL Server on VM

- *Full control*
 - *Manual backup & maintenance*
-

Q692. (Deloitte – Senior Round)

Azure Cosmos DB – when to use it?

Use when

- *Global distribution required*
- *Ultra-low latency*

- *Massive scalability*

Real-world example

IoT

Gaming telemetry

Large-scale apps

SECURITY & IDENTITY

Q693. (Microsoft – Security Round)

What is Azure Active Directory (AAD)?

Handles:

- ✓ *Authentication*
- ✓ *Authorization*
- ✓ *Single Sign-On*

Real use

Login with Microsoft / Organizational account.

Q694. (Accenture – Interview Favorite)

Managed Identity – why important?

*Allows apps to access Azure resources **without storing credentials**.*

Real use

App Service accessing Key Vault securely.

AZURE KEY VAULT

Q695. (Microsoft – Advanced Round)

Why is Azure Key Vault mandatory in enterprises?

Stores:

- Secrets
- Keys
- Certificates

Prevents:

- ✗ Hardcoding passwords
- ✗ Storing secrets in config files

NETWORKING (INTERVIEW FAVORITE)

Q696. (Amazon – Architecture Round)

What is Azure Virtual Network (VNet)?

A secure **private network** inside Azure.

Real-world usage

Secure communication between:

- App Service
 - Database
 - VMs
-

Q697. (Microsoft – Advanced)

Public Endpoint vs Private Endpoint

- ✓ *Public endpoint = accessible via internet*
- ✓ *Private endpoint = accessible only inside Azure network*

Better security, no exposure.



MONITORING & LOGGING

Q698. (Accenture – Real Project)

How do you monitor Azure applications?

Use:

- ✓ *Azure Monitor*
- ✓ *Application Insights*

Tracks:

- *Failures*
 - *Logs*
 - *Performance*
 - *User behavior*
-

Q699. (Microsoft – Advanced Round)

What metrics do you monitor in production?

- ✓ *CPU*
- ✓ *Memory*
- ✓ *Response time*
- ✓ *Error rate*
- ✓ *Throughput*

Shows the real health of the app.

SCALABILITY & RESILIENCE

Q700. (Amazon – Senior Round)

How does auto-scaling work in Azure App Service?

Azure automatically adds/removes instances based on:

- *CPU % usage*
- *Request count*
- *Schedule rules*

Reduces cost + improves performance.

DISASTER RECOVERY & BACKUP

Q701. (Deloitte – Senior Round)

How do you design DR strategy in Azure?

- ✓ *Multi-region deployment*
 - ✓ *Backups enabled*
 - ✓ *Geo-replication*
 - ✓ *Failover testing*
-

CLOUD COST MANAGEMENT

Q702. (Accenture – Interview Favorite)

How do you reduce Azure cloud cost?

- ✓ *Auto-scale*
 - ✓ *Right-size resources*
 - ✓ *Shut unused services*
 - ✓ *Reserved instances*
-



REAL INTERVIEW SCENARIOS

Q703. (Amazon – Scenario Based)

The app works locally but fails in Azure – why?

Common reasons:

- *Missing configuration*
 - *Wrong connection strings*
 - *Firewall / networking*
 - *Environment variables missing*
-

Q704. (Microsoft – Debugging)

How do you debug production issues in Azure?

Use:

- ✓ *Logs*
 - ✓ *App Insights traces*
 - ✓ *Failure diagnostics*
-



ARCHITECTURE DECISIONS

Q705. (Deloitte – Architecture Round)

Monolith vs Microservices in Azure

Right answer:

Start with **monolith**

Move to **microservices** when app grows

Shows maturity.



SECURITY BEST PRACTICES

Q706. (Microsoft – Security)

How do you secure APIs in Azure?

- ✓ Azure AD Auth
 - ✓ Azure API Management
 - ✓ HTTPS enforced
 - ✓ Key Vault secrets
-



AZURE API MANAGEMENT

Q707. (Accenture – Interview Favorite)

What is Azure API Management?

Gateway to provide:

- *Rate limiting*
 - *Authentication*
 - *Monitoring*
 - *Analytics*
-



CI/CD + AZURE

Q708. (Microsoft – Advanced)

How does CI/CD integrate with Azure?

Pipeline:

Build → Test → Deploy → Monitor

Tools:

Azure DevOps / GitHub Actions



CLOUD MIGRATION

Q709. (Amazon – Senior Round)

Challenges while migrating to Azure

- ✓ *Data migration*
 - ✓ *Downtime planning*
 - ✓ *Security & networking*
 - ✓ *Cost estimation*
-



COMPLIANCE & GOVERNANCE

Q710. (Deloitte – Compliance Round)

How does Azure support compliance?

Supports:

GDPR

ISO

HIPAA

SOC

Built-in certifications.



FINAL CLOUD QUESTIONS

Q711. (Microsoft – Final Round)

Explain Azure architecture of your project

Q712. (Accenture – HR + Tech)

Biggest cloud mistake you fixed

Q713. (Amazon – Reflection)

When not to use the cloud?

Example:

Very small apps

High offline security environments

Q714–Q719 (Scenario + Hands-on Mix)

- ✓ Secure secrets
 - ✓ Handle traffic spike
 - ✓ Unexpected bill increase
 - ✓ Multi-region strategy
 - ✓ Failover planning
-

Q720. (ALL MNCs – FINAL CLOUD QUESTION)

Why is cloud knowledge mandatory for Full-Stack Developers?

Best Interview Answer

“Because modern applications are built, deployed, scaled, secured, and monitored in the cloud. Without cloud knowledge, a developer cannot design real-world production-ready systems.”

CHAPTER 10 – HR & SCENARIO-BASED MNC QUESTIONS

◆ Behavioral, Managerial & Real-World Scenarios (Q721–Q780)

(Asked in Accenture, Deloitte, Infosys, Amazon, Microsoft, TCS, Cognizant)

INTRODUCTION & COMMUNICATION (Q721–Q780)

Q721. (ALL MNCs – FIRST QUESTION)

Tell me about yourself.

Easy, Ready-to-Speak Answer

"I am a Full-Stack Developer with solid experience in .NET, C#, React, Angular, and SQL. I've worked on building scalable Web APIs, modern front-end applications, and cloud-based solutions on Azure.

I enjoy solving real business problems, improving system performance, and writing clean, maintainable code. I've also worked closely with teams, participated in architecture discussions, and contributed to production deployments.

Currently, I'm looking for a challenging opportunity where I can contribute to large-scale enterprise applications and continue growing technically."

Interview Tip

Keep it **2 minutes max**.

Do **not** talk about family, schooling, or personal life unless asked.

Q722. (Accenture – HR Round)

Why do you want to join our company?

Best Easy Answer

"Your company works on large-scale enterprise projects and follows modern engineering practices like cloud, microservices, and DevOps. I see a strong learning environment and an opportunity to work with experienced teams while contributing real value to impactful projects."

Q723. (Infosys – HR Favorite)

Why should we hire you?

Answer

"I bring a strong combination of technical expertise, problem-solving mindset, and real project experience. I don't just deliver code—I understand business impact, performance, scalability, and production stability. I take ownership and ensure work is completed with quality and reliability."



WORK ETHIC & OWNERSHIP

Q724. (Amazon – Leadership Principle)

Describe a challenging problem you solved.

STAR Answer

Situation: Our API response was slow and affecting user experience.

Task: Improve performance and reduce latency.

Action: Implemented database indexes, caching, async calls, and optimized queries.

Result: Response time improved by ~60% and the system became more stable.

Q725. (Microsoft – Scenario)

What do you do if you don't know the answer to a technical question?

Honest Answer

"I acknowledge it, explain how I would analyze and troubleshoot it, research the best approach, and come back with a well-tested solution. I believe honesty + learning mindset is more valuable than guessing."

Q726. (Deloitte – HR Round)

How do you handle pressure or tight deadlines?

Answer

"I prioritize tasks, break them into manageable parts, stay focused, and communicate early if any risk appears. Instead of panicking, I stay structured and ensure delivery without compromising quality."



TEAMWORK & CONFLICT

Q727. (Accenture – Behavioral)

Describe a conflict with a teammate.

Answer

“We had different opinions on implementation. Instead of arguing, I proposed creating a small POC. We compared results objectively and chose the best option. This kept the team in harmony and delivered the right solution.”

Q728. (Amazon – Leadership)

How do you handle disagreement with your manager?

Answer

“I present data, examples, and explain my viewpoint respectfully. If the final decision is different, I support it completely because alignment is important for team success.”



FAILURE & LEARNING

Q729. (Microsoft – Behavioral)

Describe a mistake you made and what you learned.

Answer

“I once deployed without sufficient testing, which caused a rollback. I learned the value of strong testing, reviews, and validation. Since then, I always ensure stability before delivery.”

Q730. (Infosys – HR Favorite)

Have you ever missed a deadline?

Answer

“Yes. Due to underestimated complexity. I informed stakeholders early, adjusted the plan, and delivered a stable solution shortly after. Communication and transparency helped maintain trust.”



GROWTH & MOTIVATION

Q731. (Accenture – HR Round)

Where do you see yourself in 5 years?

Answer

“As a senior engineer or technical lead working on scalable systems, guiding architecture decisions, and mentoring juniors.”

Q732. (Amazon – Leadership)

How do you stay updated with technology?

Answer

“By following documentation, experimenting hands-on, studying production learnings, participating in discussions, and continuously learning from real-world challenges.”



ETHICS & RESPONSIBILITY

Q733. (Deloitte – Compliance)

What would you do if you found a security issue in production?

Answer

“Immediately report and assess impact, secure the system, fix or rollback safely, communicate with stakeholders, and ensure preventive measures are documented.”



REAL PROJECT SCENARIOS

Q734. (Amazon – Scenario Based)

Production is down. What do you do?

Answer

1. Understand impact
 2. Rollback if required
 3. Check logs & monitoring
 4. Fix root cause
 5. Implement prevention steps
-

Q735. (Microsoft – Scenario)

Clients want a risky feature that affects system stability.

Answer

“I explain risks clearly, suggest a safer alternative, and help them understand long-term impact before proceeding.”



TIME & PRIORITY MANAGEMENT

Q736. (Infosys – HR Round)

How do you prioritize tasks?

Answer

“Based on business value, deadlines, dependencies, risk, and impact to users.”



WORK STYLE & CULTURE FIT

Q737. (Accenture – HR)

Do you prefer working alone or in a team?

Answer

“I can work independently when required, but I believe teams create better solutions and learning happens together.”



CAREER DECISIONS

Q738. (ALL MNCs)

Why are you leaving your current company?

Safe Answer

“I am looking for better learning opportunities, exposure to large systems, and a place where I can grow long-term.”

Golden Rule

✗ Never badmouth current employer.

ETHICAL SCENARIOS

Q739. (Deloitte – Ethics)

What if your teammate takes credit for your work?

Answer

“I will address it calmly, clarify contributions professionally, and involve the manager only if required. I believe in fairness without conflict.”

TECH + HR MIX

Q740. (Microsoft – Hybrid)

Explain a technical concept to a non-tech person.

Example

“An API is like a waiter. You request food, the waiter takes it to the kitchen, and brings the response back. API does the same between app and server.”

LEADERSHIP & MENTORSHIP

Q741. (Amazon – Leadership)

Have you mentored anyone?

Answer

“Yes, I’ve supported juniors in understanding system design, debugging issues, writing cleaner code, and learning best practices.”



FINAL HR QUESTIONS

Q742. (Accenture – Final HR)

What are your strengths and weaknesses?

Answer

Strength: Problem solving, ownership, learning mindset.

Weakness: Sometimes over-focus on perfection—working to balance speed + quality.

Q743. (Microsoft – Final Round)

What motivates you?

“Building meaningful systems, solving challenges, and seeing my work deliver real business value.”

Q744. (Deloitte – HR)

How do you handle feedback?

“I take feedback positively, analyze it, improve, and ensure mistakes do not repeat.”

Q745. (Amazon – Culture Fit)

Describe a time you went beyond your role.

Example: Stayed extra to stabilize production or helped another team deliver critical functionality.



COMMON HR TRAPS

Q746. (ALL MNCs)

Do you have any questions for us?

Always Ask

- What tech stack does the team use?
- How is success measured?
- What growth opportunities exist?

Never say:

✗ “No questions”



ADVANCED SCENARIO QUESTIONS (Q747–Q779)

Q747. (Amazon – Leadership)

How will you handle layoffs impacting your team productivity?

Answer

“I would first ensure team morale stays stable by communicating transparently, clarifying priorities, and ensuring no panic. I would help redistribute responsibilities, identify critical tasks, and avoid burnout through realistic planning.”

Key Thought Process

Stability → Communication → Prioritization → Execution

Q748. (Microsoft – Behavioral)

How do you work across different time zones?

Answer

“I plan communication windows, keep documentation strong, use async communication tools, and ensure clarity in tasks. I avoid dependency bottlenecks by planning ahead.”

Q749. (Accenture – Delivery Scenario)

You missed sprint goals twice; what will you do?

Answer

“I’d analyze why: underestimation, dependencies, or unknown complexity. Then I’d improve estimation accuracy, communicate early, break tasks better, and ensure realistic planning moving forward.”

Q750. (Deloitte – Client Facing)

Client escalated due to delay—how do you handle it?

Answer

“I would acknowledge the concern, explain the real reason without blaming anyone, present a revised clear plan, and ensure frequent updates. The goal is regaining trust, not arguments.”

Q751. (Amazon – Ownership Principle)

A teammate isn’t performing; how do you handle it?

Answer

“I support first: understand the challenge, guide them, and collaborate. If performance continues to affect delivery, I escalate professionally to ensure team success.”

Q752. (Microsoft – Communication Skill)

Explain a complex technical issue to a non-technical stakeholder

Answer Example

“Instead of saying ‘server latency due to thread blocking’, I’ll say ‘the system is taking time because requests are queuing; we’re optimizing to respond faster.’ Clear, simple, and meaningful.”

Q753. (Accenture – Reality Check)

What if product requirements keep changing?

Answer

“I would encourage stable requirement baselines, involve product owners, and ensure proper sprint planning. Continuous changes kill productivity, so structured change handling is key.”

Q754. (Infosys – Team Discipline)

The team is not following coding standards—what will you do?

Answer

“Introduce code reviews, automated linting, best practices sessions, and leadership alignment. Standards maintain quality.”

Q755. (Amazon – Pressure Handling)

Production down during the weekend—what do you do?

Answer

“Immediately respond, assess impact, fix or rollback, ensure communication, then follow with root-cause analysis. Stability first.”

Q756. (Microsoft – Time Management)

You have multiple high-priority tasks. What do you do?

Answer

“Prioritize by business impact, deadlines, dependencies, and risk. Communicate if timelines clash. Structured prioritization ensures the right outcomes.”

Q757. (Deloitte – Ownership)

You finished your work early; what next?

Answer

“I help my team, improve code, review system performance, or enhance documentation. Ownership continues beyond completion.”

Q758. (Amazon – Leadership Reflection)

Describe a time you took initiative without being asked

Answer

“Once I identified performance risk before release. I optimized queries proactively which prevented potential production issues.”

Q759. (Microsoft – Real Delivery Scenario)

How do you handle last-minute requirement changes?

Answer

“Evaluate impact, communicate feasibility and risks, provide alternatives, and ensure realistic delivery instead of rushing unstable code.”

Q760. (Accenture – Conflict Resolution)

Teammates disagree strongly; what do you do?

Answer

“Encourage data-based discussion or small POC. Let results decide, not ego.”

Q761. (Amazon – Bias for Action)

You found a bug that others ignored

Answer

“I validate impact, log it, communicate risk, and ensure it’s fixed before it becomes expensive.”

Q762. (Microsoft – Ownership Culture)

How do you ensure quality in your code?

Answer

“Reviews, unit tests, clean architecture, scalability thinking, and defensive coding.”

Q763. (Deloitte – Business Thinking)

How do you balance speed vs quality?

Answer

“I avoid shortcuts. I deliver fast but stable releases using automation and structured coding principles.”

Q764. (Infosys – Learning Mindset)

What is your biggest professional learning?

Answer

“Production failures teach responsibility, testing, and design thinking more than tutorials ever can.”

Q765. (Amazon – Ownership Reflection)

Describe a time you worked beyond your responsibility

Answer

“Handled deployment, helped support team, and ensured stability even though it wasn’t assigned to me. Ownership matters.”

Q766. (Accenture – Leadership Round)

How do you keep a team motivated?

Answer

“Clarity, appreciation, collaboration, and celebrating wins.”

Q767. (Microsoft – Accountability)

Have you ever strongly disagreed with the final decision?

Answer

“Yes, but once the decision was finalized, I supported it and ensured successful delivery.”

Q768. (Deloitte – Production Impact)

How do you prevent repeating mistakes?

Answer

“Root cause analysis, documentation, automation, and preventive checks.”

Q769. (Amazon – Production Priority)

Feature release vs stability issue — what do you pick?

Answer

“Stability first. A failing system kills business trust.”

Q770. (Accenture – Professional Integrity)

How do you handle unethical requests?

Answer

"I refuse professionally, explain risks, and ensure integrity is maintained."

Q771. (Microsoft – Team Culture)

How do you help new team members ramp up?

Answer

"Documentation, mentorship, explaining architecture, and guiding patiently."

Q772. (Amazon – Ownership Real Case)

Describe a tough leadership situation

Answer

"Took responsibility for production issues, coordinated the team, communicated clearly, and restored the system successfully."

Q773. (Infosys – Problem Solving)

How do you approach unknown technical issues?

Answer

"Break it down → analyze logs → replicate issue → research → implement fix → validate."

Q774. (Deloitte – Pressure Handling)

What if every task feels urgent?

Answer

"Everything can't be top priority. I align with stakeholders to define reality."

Q775. (Microsoft – Ownership & Growth)

Biggest professional achievement?

(Example format interviewer expects)

“Improving performance, reducing cost, saving downtime, or delivering major success.”

Q776. (Amazon – Team Contribution)

How do you ensure team success, not only personal success?

Answer

“By helping, sharing knowledge, contributing to solutions, and maintaining healthy collaboration.”

Q777. (Accenture – Leadership Reality)

What if you are assigned a boring task?

Answer

“I still deliver with the same quality. Consistency defines professionalism.”

Q778. (Microsoft – Resilience)

Describe a time you failed and bounced back

Answer

“Deployment mistakes → learned → automated checks → improved maturity.”

Q779. (Deloitte – Responsibility Under Pressure)

What if business wants an impossible timeline?

Answer

“I communicate realistic estimates, provide a phased delivery plan, and ensure the best feasible outcome without compromising quality.”

Q780. (ALL MNCs – FINAL)

Why should we trust you with critical systems?

Best Interview Answer

“Because I take complete ownership, communicate transparently, focus on security and stability, and always ensure quality over shortcuts. I treat production systems with responsibility and respect.”

Chapter 11 – System Design (MNC Level)

Q781 – Q840 (60 Questions)

- Low-level design (LLD)
- High-level design (HLD)
- API design
- Scalability
- Caching, load balancing
- Real .NET + Azure examples

👉 *Very important for 5+ years, Amazon, Microsoft, Deloitte*

CHAPTER 11 – SYSTEM DESIGN (MNC INTERVIEW LEVEL)

◆ Foundations of System Design (Q781–Q800)

(Low-Level + High-Level thinking with real examples)



SYSTEM DESIGN BASICS

Q781 – Q800 (Interview-Ready + Real-World Focused)

Q781. (ALL MNCs – FIRST DESIGN QUESTION)

What is System Design in simple words?

Easy explanation

System Design means planning how a system will work **before writing code**. It defines architecture, communication, scalability, storage, and reliability.

It answers

- How components interact
- How data flows
- How the system scales
- How failures are handled

Real-life example

Before building an e-commerce platform, you decide:

Frontend → React / Angular

Backend APIs → .NET Core / Node

Database → SQL

Cache → Redis

Deployment → Azure / AWS

Security → JWT / OAuth

Monitoring → App Insights / CloudWatch

This planning is system design.

Q782. (Microsoft – Interview Favorite)

Difference between Low-Level Design (LLD) and High-Level Design (HLD)

HLD

LLD

Big picture

Implementation detail

Architecture view Class / method view

Tech selection Coding structure

Scalability focus Code correctness
focus

Interview Tip

Say confidently:

“HLD is for architecture & scalability. LLD is for implementation and coding structure.”



CORE BUILDING BLOCKS

Q783. (Amazon – Architecture Round)

What are the core components of any large system?

Most enterprise systems include

- Client (Web / Mobile)
- Backend services
- Database
- Cache layer
- Load balancer
- Security (Auth)
- Logging & Monitoring

Real-world

Netflix
Amazon
Banking systems
—all follow the same blueprint.

Q784. (Accenture – Scenario Based)

What is scalability and why does it matter?

Explanation

Scalability means the ability of a system to handle **increased load without failure**.

Real example

Salary day → banking traffic spikes.
The system must handle increased load smoothly.



SCALING STRATEGIES

Q785. (Microsoft – Technical Round)

Vertical vs Horizontal Scaling

Vertical Scaling

Upgrade RAM / CPU

Simple

Limited

Horizontal Scaling

Add more servers

Complex

Highly scalable

Single point of failure More reliable

Interview Tip

Say:

“Modern clouds prefer horizontal scaling.”

Q786. (Amazon – Interview Favorite)

What is a Load Balancer and why is it needed?

Easy explanation

A load balancer distributes incoming traffic across multiple servers to:

- Avoid overload
- Improve availability
- Prevent downtime

Example

Azure Load Balancer
AWS ELB
NGINX



DATA & STORAGE DESIGN

Q787. (Microsoft – Design Round)

How do you choose between SQL and NoSQL?

Use SQL when

- Strong relationships
- ACID transactions
- Structured schema
- Banking / Orders / Inventory

Use NoSQL when

- Huge scale
 - Flexible schema
 - High write speed
 - Logs / Analytics / IoT
-

Q788. (Amazon – Scenario Based)

What is database sharding?

Sharding = Splitting database into multiple parts.

Example

User DB split by ID range

1–1,000,000 → DB1

1,000,001–2,000,000 → DB2

Helps handle massive users.

CACHING (VERY IMPORTANT)

Q789. (Microsoft – Interview Favorite)

What is caching and why do we need it?

Easy explanation

Caching stores frequently accessed data in fast in-memory storage to reduce database hits.

Real use

Product list cached in Redis

→ Faster response

→ Less DB load

Q790. (Amazon – Architecture Round)

Where should cache be used?

- Frequently accessed API data
- Database queries
- Configuration values

Interview Tip

Say:

“Cache data that is read often and changes rarely.”



CONSISTENCY & RELIABILITY

Q791. (Deloitte – Senior Round)

What if cache and DB values differ?

This is **cache inconsistency**.

Solutions

- Cache expiration

- Write-through cache
 - Write-behind cache
-



API DESIGN THINKING

Q792. (Accenture – Interview Favorite)

What makes a good API design?

Key points

Clear endpoints
Meaningful HTTP verbs
Versioning
Pagination
Security

Example:

```
GET /api/v1/orders?page=1
```



STATE MANAGEMENT

Q793. (Microsoft – Advanced Round)

Stateless vs Stateful systems

Stateless

Stateful

No session
stored

Stores
session

Easy to scale

Hard to scale

Cloud friendly

Limited

REST APIs

Legacy apps

Interview Tip

Say:

“Web APIs must be stateless.”



FAULT TOLERANCE

Q794. (Amazon – Reliability Round)

What if one service goes down?

A good system:

- Fails gracefully
- Retries
- Uses fallback
- Avoids cascade failure

Pattern

Circuit Breaker



SECURITY AT DESIGN LEVEL

Q795. (Microsoft – Security Design)

How do you design secure systems?

- Authentication (Who are you?)
- Authorization (What can you do?)
- HTTPS
- Secure secrets

Security is **designed**, not patched later.



OBSERVABILITY

Q796. (Accenture – Interview Favorite)

Why logging & monitoring are critical?

Because:

bugs happen → **in production**, not local.

Tools

Azure Monitor
Application Insights
CloudWatch



REAL DESIGN SCENARIOS

Q797. (Amazon – Scenario Based)

The system works for 1k users but fails at 100k – why?

Common reasons:

- No caching
 - Single DB
 - No load balancing
 - Bad queries
-



TRADE-OFF QUESTIONS

Q798. (Microsoft – Architecture Round)

Is more microservices always better?

Answer

No.

Microservices increase:

- Complexity
- DevOps overhead
- Communication challenges

Use only when scale demands.

COMMUNICATION IN DESIGN INTERVIEW

Q799. (ALL MNCs – IMPORTANT)

How should you talk in a system design interview?

Best strategy

Ask clarifying questions
Define assumptions
Explain thinking
Prefer simple, scalable designs

Interviewers judge **thinking**, not perfection.

FINAL FOUNDATION QUESTION

Q800. (ALL MNCs – FINAL)

What is the most important thing in system design?

Best Interview Answer

“Understanding requirements and making the right trade-offs between performance, cost, scalability, complexity, and reliability.”

◆ Real System Design Patterns & Scenarios

Q801 – Q820 (Amazon, Microsoft, Deloitte, Accenture Focus)



DESIGN APPROACH (INTERVIEW EXPECTATION)

Q801. (ALL MNCs – DESIGN STARTER)

How do you approach a system design problem in an interview?

Easy, step-by-step answer

- 1 Clarify requirements
- 2 Identify scale (users, TPS, growth)
- 3 Define high-level architecture
- 4 Choose database & storage strategy
- 5 Add cache layer
- 6 Plan for scalability & reliability
- 7 Discuss security
- 8 Explain trade-offs

Interview Tip

Never jump to DB immediately.
Speak like an architect.



CLASSIC DESIGN PROBLEMS

Q802. (Amazon – Interview Favorite)

Design a URL Shortener (like bit.ly)

High-Level Design

- API to create short URL
- Generate unique key
- Store mapping
- Redirect service

Real Components

Backend → .NET Core / Node

DB → SQL or NoSQL

Cache → Redis

Load Balancer → Nginx

Q803. (Microsoft – Follow-up)

How do you generate unique short URLs?

Options

Base62 encoding

Hashing

Auto-increment ID

Trade-off

Predictable vs secure randomness.



DATA & CACHE STRATEGY

Q804. (Amazon – Architecture Round)

Where do you use cache in URL shorteners?

Cache

shortURL → longURL mapping

Why?

Because reads >> writes

Redis improves performance massively.



SCALABILITY THINKING

Q805. (Accenture – Scenario Based)

How does URL shortener scale to millions of hits?

Answer

Stateless services
Horizontal scaling
Distributed cache
Load balancer
Partition DB



SECURITY & ABUSE PREVENTION

Q806. (Microsoft – Security Design)

How do you prevent abuse?

Rate limiting
CAPTCHA
Token-based API usage
Block harmful URLs



E-COMMERCE DESIGN

Q807. (Amazon – System Design)

Design an E-Commerce Platform

Core Components

Product service
Order service
Payment service

Inventory service
User service

This is how Amazon, Flipkart work.



DATABASE DESIGN

Q808. (Microsoft – Data Design)

Which DB for Orders and why?

SQL

Why?

Transactions
Consistency
Referential integrity

Orders cannot be eventually consistent.



INVENTORY CHALLENGE

Q809. (Amazon – Hard Scenario)

How do you prevent overselling?

Locking
Optimistic concurrency
Queue based processing



ASYNC PROCESSING

Q810. (Accenture – Architecture Round)

Why use message queues?

Decouple services
Handle spikes
Retry mechanism
Reliable delivery

Tools

Azure Service Bus
RabbitMQ
Kafka



NOTIFICATION SYSTEM

Q811. (Microsoft – Interview Favorite)

Design Notification System (Email / SMS / Push)

Design

Event → Queue → Worker → Provider

Benefits

Scalable
Retry safe
Resilient



REAL-TIME SYSTEMS

Q812. (Amazon – Advanced)

Design Real-Time Chat System

WebSockets
Message store
Online status tracking
Delivery confirmation

CONSISTENCY & EVENTUAL CONSISTENCY

Q813. (Microsoft – Architecture Round)

Strong vs Eventual Consistency

Strong	Eventual
---------------	-----------------

Immediate	Slight delay
-----------	--------------

Secure	Highly scalable
--------	-----------------

Real World

Banking → Strong
Social Feeds → Eventual

FAILURE HANDLING

Q814. (Deloitte – Senior Round)

What if the message queue goes down?

Retry
Dead-letter queue

Alerting
Fallback handling

IDEMPOTENCY

Q815. (Amazon – Interview Favorite)

What is Idempotency? Why is it important?

Same request multiple times
→ Same result

Real Example

Payment APIs
Order submission

API GATEWAY

Q816. (Microsoft – Architecture)

Why use API Gateway?

Authentication
Rate limiting
Logging
Routing
Central entry point

SERVICE COMMUNICATION

Q817. (Accenture – Technical Round)

Sync vs Async communication

Sync	Async
Immediate	Queue based
Tight coupling	Loose coupling
Faster	More resilient



DESIGN TRADE-OFFS

Q818. (Amazon – Senior Round)

Monolith vs Microservices – How to choose?

Consider

Team size
System complexity
Deployment needs
Scalability needs

Microservices = power + complexity.



INTERVIEW BEHAVIOR

Q819. (ALL MNCs)

What if the interviewer disagrees with your design?

Say:

"I'll validate assumptions, ask constraints, and refine the solution based on feedback."

Shows maturity.

FINAL QUESTION

Q820. (ALL MNCs – FINAL)

What matters more – perfect design or trade-offs?

Best Answer

"Trade-offs matter more. Real systems are about balancing performance, cost, complexity, security, and scalability."

◆ Advanced System Design Scenarios

Q821 – Q840 (Amazon | Microsoft | Deloitte | Accenture)

RATE LIMITING & TRAFFIC CONTROL

Q821. (Amazon – Interview Favorite)

Design a Rate Limiter

Easy Explanation

A **rate limiter** controls how many requests a client can send in a specific time window.

Why is it needed?

- Prevent abuse
- Protect backend
- Ensure fair usage

Real example

A user cannot hit API **10,000 times/sec.**

Common Techniques

- Fixed Window
 - Sliding Window
 - Token Bucket
 - Leaky Bucket
-

Q822. (Microsoft – Architecture Round)

Where should rate limiting be implemented?

Options

API Gateway
Load Balancer
Application Layer

Best Practice

Implement **at API Gateway level**

- Stops bad traffic early
 - Saves backend cost
-

Q823. (Accenture – Technical Round)

Token Bucket vs Leaky Bucket

Token Bucket	Leaky Bucket
Allows burst traffic	Smooth fixed rate
Flexible	Strict throttling
Common in APIs	Common in networking



FILE STORAGE DESIGN

Q824. (Amazon – Design Round)

Design a file storage system (like Google Drive)

Core Components

Upload service
Metadata database
Blob/Object storage
CDN for fast delivery

Key Considerations

Security
Sharing links
Versioning
Scalability

Q825. (Microsoft – Follow-up)

How do you handle large file uploads?

Chunk uploads
Resume uploads
Parallel uploading

Useful in real world:
Google Drive, OneDrive.



AUTHENTICATION SYSTEM DESIGN

Q826. (Accenture – Interview Favorite)

Design an authentication system

Core Components

Login API
User database
Token service
Token validation middleware

Supports:
JWT
OAuth
SSO

Q827. (Microsoft – Security Round)

JWT vs Session-based auth — which scales better?

JWT scales better

Why?

Stateless
No server-side session storage required

TOKEN MANAGEMENT

Q828. (Amazon – Advanced)

How do you handle token expiration & refresh?

Short-lived Access Token
Long-lived Refresh Token

Flow:
Access token expires → Client uses refresh token → New access token issued.

Security benefit.

LOGGING & ANALYTICS

Q829. (Microsoft – Architecture Round)

Design a centralized logging system

Core Components

Log producers
Log ingestion pipeline
Storage
Search & visualization

Real Tools

ELK Stack
Azure Monitor
Splunk

EVENT-DRIVEN SYSTEMS

Q830. (Amazon – Senior Round)

Why event-driven architecture scales better?

Loose coupling
Async processing
Handles spikes
Improves reliability

DATA STREAMING

Q831. (Microsoft – Advanced)

Design a real-time analytics system

Producers
Stream processor (Kafka / Spark)
Storage
Dashboard

Used in:
Live dashboards

Financial trading
Metrics monitoring

CONSISTENCY PATTERNS

Q832. (Accenture – Architecture Round)

What is eventual consistency?

Data is not updated instantly everywhere
But becomes consistent after some time.

Example

Likes count on social media.

CACHE STRATEGIES

Q833. (Amazon – Interview Favorite)

Write-through vs Write-back Cache

Write-through	Write-back
Data written to DB immediately	Writes delayed
Safer	Faster
Slower	Risk of data loss

FAILURE RECOVERY

Q834. (Microsoft – Reliability)

What is a Dead Letter Queue (DLQ)?

Stores failed messages
Helps analyze issues
Prevents message loss

SEARCH SYSTEM DESIGN

Q835. (Amazon – Design Round)

Design a search system

Indexer Service
Search Engine (ElasticSearch)
Ranking logic
Autosuggest

Used in:
E-commerce
Job portals
Streaming platforms

API VERSIONING AT SCALE

Q836. (Microsoft – Advanced)

How do you version APIs in large systems?

URL versioning
Header-based
Backward compatibility

Most common:
`/api/v1/users`

SECURITY AT SCALE

Q837. (Accenture – Security)

How do you protect APIs from DDoS?

Rate limiting
WAF
CDN protection
Autoscaling

COST OPTIMIZATION

Q838. (Amazon – Architecture)

How do you reduce cloud cost at scale?

Auto scaling
Right-size VMs
Cache heavily
Use reserved instances

DESIGN VALIDATION

Q839. (ALL MNCs – IMPORTANT)

How do you validate your design?

Load testing
Chaos testing
Monitoring
Alerts
Capacity planning

Shows you are production minded.

FINAL SYSTEM DESIGN QUESTION

Q840. (ALL MNCs – FINAL)

What makes a system design interview successful?

Best Answer

“Clear communication, right assumptions, scalable thinking, secure design, and strong trade-off explanation.”

CHAPTER 12 – MICROSERVICES & ARCHITECTURE

◆ Microservices Fundamentals & Core Patterns (Q841–Q860)

(Very important for 4–10+ years – Senior / Architect rounds)

Microservices Basics (Q841–Q860)

Q841. (ALL MNCs – Entry Question)

What are microservices in simple words?

Easy explanation

Microservices mean breaking one big application into multiple **small, independent services**, where each service:

- Handles one responsibility
- Can be deployed independently
- Can scale independently

Real-world example

Instead of one huge e-commerce app:

- User Service
- Order Service
- Payment Service

Each works independently.

Q842. (Accenture – Interview Favorite)

Monolith vs Microservices

Monolith	Microservices
Single deployment	Independent deployments
Simple to start	Complex to manage
Tight coupling	Loose coupling
Hard to scale	Easy to scale

Interview Tip

Say:

“Start with a monolith, move to microservices when scale demands.”



WHEN TO USE MICROSOOERVICES

Q843. (Microsoft – Architecture Round)

When should you NOT use microservices?

Avoid microservices when:

- Team is small
- Application is simple

- Traffic is low
- No scaling problems

Because microservices **increase operational complexity**.

SERVICE BOUNDARIES

Q844. (Amazon – Design Round)

How do you decide microservice boundaries?

Use **business domains**, not technical layers.

Correct thinking

 One “Database Service”
User Service, Order Service, Payment Service

This follows **Domain-Driven Design**.

DATABASE PER SERVICE

Q845. (Microsoft – Interview Favorite)

Why should each microservice have its own database?

Because it ensures:

- Independent scaling
- Independent deployments
- Loose coupling

Keyword interviewers love

“Database per service”

SERVICE COMMUNICATION

Q846. (Accenture – Technical Round)

Synchronous vs Asynchronous communication

Sync (HTTP)	Async (Messaging)
Immediate response	Event-driven
Simple	Highly scalable
Tight coupling	Loose coupling

Q847. (Amazon – Architecture Round)

When do you choose async communication?

Use async when:

- High traffic
- Background processing
- Resilience required

Common Tools

Kafka
RabbitMQ
Azure Service Bus

API GATEWAY

Q848. (Microsoft – Interview Favorite)

What is an API Gateway and why is it needed?

Single entry point for all microservices.

Handles

Authentication

Routing

Rate limiting

Logging

Real example

Azure API Management



SERVICE DISCOVERY

Q849. (Accenture – Senior Round)

What is service discovery in microservices?

When instances change dynamically, service discovery helps services find each other.

Real tools

Kubernetes

Consul



RESILIENCE PATTERNS

Q850. (Amazon – Interview Favorite)

What is the Circuit Breaker pattern?

Stops calling a failing service to protect the system.

Example

Payment service down → Temporarily stop requests.

RETRY & TIMEOUT

Q851. (Microsoft – Advanced Round)

Why retries can be dangerous?

Because too many retries can:

- Overload the system
- Increase latency

Best practice

Retry with exponential backoff.

DATA CONSISTENCY

Q852. (Accenture – Architecture Round)

How do microservices maintain data consistency?

Using:

- Eventual consistency
 - Saga pattern
-

SAGA PATTERN

Q853. (Amazon – Senior Round)

Explain Saga Pattern with example

Manages distributed transactions with events.

Example

Order workflow:

Order → Payment → Inventory → Shipping

If payment fails:

Order is rolled back.



EVENT-DRIVEN ARCHITECTURE

Q854. (Microsoft – Interview Favorite)

Why is event-driven architecture popular?

Because it offers:

- Loose coupling
- Better scalability
- Fault isolation



DEPLOYMENT

Q855. (Accenture – DevOps Round)

How do you deploy microservices safely?

Independent CI/CD pipelines

Blue-Green deployment

Feature flags



OBSERVABILITY

Q856. (Microsoft – Senior Round)

Why is observability critical?

Because failures are distributed.

3 pillars

Logs

Metrics

Traces



SECURITY

Q857. (Amazon – Security Round)

How do you secure service-to-service communication?

mTLS

OAuth tokens

Private networking



REAL SCENARIO

Q858. (Deloitte – Scenario Based)

One microservice is slow — how do you debug?

Check latency metrics

Trace request path

Check dependent services



TRADE-OFF

Q859. (Microsoft – Architecture Round)

Biggest drawback of microservices?

Operational complexity



FINAL QUESTION

Q860. (ALL MNCs – FINAL)

When are microservices the right choice?

Best Interview Answer

“When the system is large, teams are independent, traffic is high, and scalability + resilience are genuine needs — not just buzzwords.”

◆ Advanced Patterns, Data & Platform Concerns (Q861–Q890)



DATA & CACHING IN MICROSERVICES

Q861. (Microsoft – Architecture Round)

How do you use caching in microservices without breaking consistency?

Easy explanation

Cache only what makes sense – mainly **read-heavy and slow-changing data**.
Keep cache close to the service that owns the data.

Best practices

- Use **TTL (expiry)**
- Prefer **cache-aside** pattern
- Invalidate on write where possible
- Avoid caching highly volatile data

Tools

Redis

Azure Cache for Redis

Q862. (Amazon – Interview Favorite)

Cache-aside vs Write-through cache

Cache-aside	Write-through
App controls cache	Cache auto-updates
More flexible	Safer
Can become stale	Slower writes



DATA DUPLICATION & OWNERSHIP

Q863. (Accenture – Senior Round)

Is data duplication allowed in microservices?

Answer

Yes — **controlled duplication is allowed**.

Why?

To reduce:

- cross-service dependency
- network calls
- latency

Golden rule

There must still be **one source of truth**.

CQRS PATTERN

Q864. (Microsoft – Interview Favorite)

What is CQRS and when should you use it?

Explanation

CQRS = **Command Query Responsibility Segregation**
Separate:

- Write model (commands)
- Read model (queries)

When to use

- Heavy read traffic
 - Complex business logic
 - Different read/write performance needs
-

Q865. (Amazon – Real Scenario)

CQRS real example

Orders:

- Writes stored in SQL
 - Read model copied to NoSQL for fast dashboards
-

EVENT SOURCING

Q866. (Microsoft – Advanced Round)

What is Event Sourcing?

Instead of storing the latest state, store **every event** that changed it.

Example

OrderCreated
PaymentReceived
OrderShipped

State = replay of events.

Q867. (Accenture – Trade-off Question)

Event Sourcing pros and cons

Pros

Complete audit history
Easy rollback

Cons

Complex
Harder querying

SCHEMA & VERSIONING

Q868. (Amazon – Architecture Round)

How do you handle schema changes in microservices?

Backward compatible first:

- Do not break consumers
 - Version APIs
 - Version events
 - Use consumer-driven contracts
-

CONTRACT TESTING

Q869. (Microsoft – Senior Round)

What is contract testing and why is it important?

Ensures provider changes do not break consumers.

Tools

Pact

Postman Contract Testing

SERVICE MESH

Q870. (Accenture – Advanced)

What is a Service Mesh?

Manages:

- Traffic
- Security
- Observability

Without code changes.

Tools

Istio

Linkerd

KUBERNETES BASICS

Q871. (Microsoft – Interview Favorite)

Why Kubernetes with microservices?

Because it offers:

- Auto-scaling
- Self-healing
- Rolling deployments
- Traffic management

Q872. (Amazon – Technical Round)

Pod vs Deployment

Pod

Deployment

Smallest unit Manages pods

Can die Ensures availability

CONFIGURATION MANAGEMENT

Q873. (Accenture – Senior Round)

How do you manage configuration?

Use:

- Environment variables
- Centralized config service
- Azure App Configuration

 Never hardcode config.

SECURITY & ZERO TRUST

Q874. (Microsoft – Security Round)

What is Zero Trust architecture?

Never assume trust — always verify.
Even inside the network.

OBSERVABILITY

Q875. (Amazon – Interview Favorite)

How do you trace a request across microservices?

Use **distributed tracing + correlation IDs**.

Tools

OpenTelemetry
Application Insights

FAILURE MODES

Q876. (Deloitte – Scenario)

One service is healthy but the system is slow — why?

Because:

- Downstream dependency slow
 - Thread exhaustion
 - Retry storms
-

DEPLOYMENT & RELEASE

Q877. (Accenture – DevOps Round)

What are feature flags and why use them?

Deploy feature but disable it until safe.
Helps in safe rollout.

DATA MIGRATION

Q878. (Microsoft – Architecture)

How do you migrate monoliths to microservices safely?

- Strangler pattern
 - Incremental extraction
 - Parallel run
-

STRANGLER PATTERN

Q879. (Amazon – Interview Favorite)

Explain Strangler Fig pattern

Gradually replace old system pieces with new microservices.

Zero big-bang migration.

CROSS-CUTTING CONCERNS

Q880. (Accenture – Technical Round)

How do you handle logging, auth, rate limiting centrally?

API Gateway + Service Mesh

TEAM & ORG DESIGN

Q881. (Microsoft – Senior Round)

Conway's Law — what is it?

System design reflects team structure.

Design teams thoughtfully.

PERFORMANCE OPTIMIZATION

Q882. (Amazon – Advanced)

How to reduce latency?

- Co-locate services
 - Async where possible
 - Reduce chattiness
-

BACKPRESSURE

Q883. (Microsoft – Advanced)

What is backpressure?

Prevents fast producers from overwhelming slow consumers.

COST & GOVERNANCE

Q884. (Accenture – Architecture Round)

How to control cloud cost?

- Auto-scale wisely
 - Right-size resources
 - Resource limits
-

SECURITY INCIDENTS

Q885. (Deloitte – Scenario)

Compromised service — actions?

Revoke tokens
Isolate service
Rotate secrets

ANTI-PATTERNS

Q886. (Microsoft – Interview Favorite)

Common microservices anti-patterns

- Shared database
 - Over-microservicing
 - Chatty services
-

DESIGN VALIDATION

Q887. (Amazon – Senior Round)

How to test microservices architecture?

Unit tests
Contract tests
Chaos testing

CHAOS ENGINEERING

Q888. (Microsoft – Advanced)

What is Chaos Engineering?

Intentionally breaking components to ensure resilience.

REAL INTERVIEW BEHAVIOR

Q889. (ALL MNCs)

How do you explain complex architecture to non-tech people?

Use:
Diagrams
Simple analogies
Business outcomes

FINAL MICROSOFTSERVICES QUESTION

Q890. (ALL MNCs – FINAL)

What decides success in microservices?

Best Answer

“Clear boundaries, disciplined DevOps, strong observability, resilience, and teams that understand responsibility and ownership.”

CHAPTER 13 – TESTING & QUALITY ENGINEERING

◆ Testing Fundamentals & Unit Testing (Q891–Q910)

(Asked in Accenture, Deloitte, Microsoft, Amazon, Infosys – 3+ years)



TESTING BASICS (INTERVIEW FOUNDATION)

Q891 – Q910

Q891. (ALL MNCs – BASIC QUESTION)

What is software testing and why is it important?

Easy Explanation

Software testing ensures that the application:

- Works as expected
- Does not break existing functionality
- Performs correctly under different conditions
- Is safe to release to production

Real-world example

A payment bug in production can cause financial loss and customer trust issues, so testing is critical.

Q892. (Accenture – Interview Favorite)

Difference between Manual Testing and Automation Testing

Manual Testing	Automation Testing
Done by Easys	Done using scripts
Slower	Faster
Good for UI/UX testing	Best for repetitive tasks
Not reusable	Highly reusable

Interview Tip

Always say:

“Automation supports CI/CD and enables faster, safer releases.”

TESTING TYPES

Q893. (Microsoft – Technical Round)

Types of testing in enterprise applications

- Unit Testing
- Integration Testing
- System Testing
- Regression Testing

Each level catches different bugs and improves confidence.

TEST PYRAMID (VERY IMPORTANT)

Q894. (Amazon – Interview Favorite)

What is the Test Pyramid?

The Test Pyramid suggests:

- Many unit tests
- Fewer integration tests
- Very few UI tests

Why?

Unit tests are fast and cheap.

UI tests are slow and flaky.

UNIT TESTING IN .NET

Q895. (Microsoft – Technical Round)

What is unit testing in C#?

Unit testing tests **one small piece of code** in isolation.

Real C# Example (xUnit)

```
[Fact]
public void Add_ShouldReturnSum()
{
    var calc = new Calculator();
    var result = calc.Add(2, 3);
    Assert.Equal(5, result);
}
```

Q896. (Accenture – Developer Round)

Which unit testing frameworks are used in .NET?

- xUnit (most popular)
- NUnit
- MSTest

Interview Tip

Say:

“xUnit is preferred in modern .NET projects.”

MOCKING (VERY IMPORTANT)

Q897. (Amazon – Interview Favorite)

What is mocking and why is it needed?

Easy Explanation

Mocking means creating fake dependencies so you test **only your code**, not external systems.

Real example

Testing service logic without hitting a real database.

Q898. (Microsoft – Technical Round)

Mocking example using Moq

```
var mockRepo = new Mock<IUserRepository>();  
mockRepo.Setup(r => r.GetUser(1))  
    .Returns(new User { Name = "John" });
```

Why important

- Tests remain fast
- No external failures
- Behavior controlled



DEPENDENCY INJECTION & TESTABILITY

Q899. (Accenture – Interview Favorite)

How Dependency Injection helps in testing?

Explanation

Dependency Injection allows replacing real dependencies with mocks during testing.

Keyword

“Loose coupling improves testability.”



INTEGRATION TESTING

Q900. (Microsoft – Advanced Round)

What is integration testing?

Integration testing ensures modules work together correctly.

Real example

API + Database + Business Logic.

Q901. (Amazon – Scenario Based)

When do you prefer integration tests over unit tests?

When verifying:

- Database behavior
 - API endpoints
 - Security
 - Configurations
-

API TESTING

Q902. (Accenture – Technical Round)

How do you test Web APIs?

- Unit tests for controllers/services
 - Integration tests using TestServer
 - Manual/Postman testing
 - Newman in CI pipelines
-

Q903. (Microsoft – Advanced)

API Integration Test Example Approach

Use:

- `WebApplicationFactory`
- `HttpClient`
- In-memory hosting

This validates real endpoints.



FRONTEND TESTING (REACT & ANGULAR)

Q904. (Amazon – Interview Favorite)

Types of frontend tests

- Component Tests
 - Integration Tests
 - End-to-End (E2E) Tests
-

Q905. (Microsoft – React/Angular Round)

Why do we keep fewer E2E tests?

Because they are:

- Slow
- Flaky
- Expensive to maintain

Used only for:

- Login flows
 - Checkout flow
 - Critical scenarios
-



TEST DATA & ENVIRONMENTS

Q906. (Accenture – Real Project)

How do you manage test data?

- Seed test database
 - Use fake data
 - Separate test environment
 - Never use production data
-

TESTING IN CI/CD

Q907. (Microsoft – DevOps Round)

How does testing fit in the CI/CD pipeline?

Typical pipeline:

- 1 Build
- 2 Run Unit Tests
- 3 Run Integration Tests
- 4 Deploy

If tests fail → **deployment stops automatically.**

QUALITY MINDSET

Q908. (Amazon – Behavioral + Tech)

Who is responsible for quality – QA or Developer?

Best Answer

“Quality is everyone’s responsibility. Developers, QA, DevOps — all play a role.”



COMMON TESTING MISTAKES

Q909. (Deloitte – Senior Round)

Common mistakes in testing

- Too many UI tests
 - No mocking
 - Ignoring edge cases
 - Skipping automation
 - Not testing performance
-



FINAL TESTING QUESTION

Q910. (ALL MNCs – FINAL TESTING QUESTION)

How do you ensure high-quality releases?

Best Interview Answer

“By combining good design, automated unit + integration tests, proper code reviews, continuous testing in CI/CD, and strong monitoring in production.”

◆ Automation, Performance & Quality Strategy (Q911–Q930)



TEST AUTOMATION STRATEGY

Q911. (Microsoft – Senior Round)

What is a test automation strategy?

Easy Explanation

A **test automation strategy** defines how an organization plans and manages automated testing. It answers:

- What should be automated
- What should not be automated
- Which tools and frameworks to use
- When tests should run
- Where automation fits in CI/CD

Key Idea

Automation is about **business value**, not about writing maximum tests.

Real-world:

Banks automate login, payments, and core flows — not cosmetic UI.

Q912. (Accenture – Interview Favorite)

What should NOT be automated?

Automation should NOT be used for:

- Frequently changing UI
- One-time testing scenarios
- Visual design testing
- Highly exploratory testing

Best Practice

Automate **stable, repetitive, business-critical flows**.



TEST COVERAGE & METRICS

Q913. (Amazon – Interview Favorite)

What is test coverage and why does it matter?

Explanation

Test coverage measures how much of your code is executed by tests.

Why it matters

- Helps identify untested areas
- Improves confidence
- Supports risk assessment

Interview Tip

Say:

“Coverage is a guide, not a goal. Quality of tests is more important than percentage.”

Q914. (Microsoft – Advanced)

Is 100% test coverage required?

Best Answer

No.

Focus should be on:

- Critical flows
- High-risk areas
- Business impact paths

100% coverage with weak tests = false confidence.

PERFORMANCE TESTING

Q915. (Accenture – Performance Round)

What is performance testing?

Explanation

Performance testing checks how the system behaves under expected and heavy workload.

It measures:

- Response time
- Throughput
- CPU/Memory usage
- Stability over time

Helps ensure users get a smooth experience.

Q916. (Amazon – Scenario Based)

Performance testing tools you used

Examples

- JMeter
- k6
- Azure Load Testing
- Locust

Mention real usage like:

“Tested API throughput and optimized response time before production rollout.”

LOAD vs STRESS TESTING

Q917. (Microsoft – Interview Favorite)

Load testing vs Stress testing

Load Testing	Stress Testing
Test under expected user load	Push system beyond limits
Ensures system works normally	Finds breaking point
Predict performance	Tests resilience & recovery

SECURITY TESTING

Q918. (Accenture – Security Round)

What is security testing?

Security testing identifies vulnerabilities like:

- SQL Injection
- XSS
- Broken Authentication
- Insecure APIs

Goal:

Protect data, prevent hacking, ensure compliance.

Q919. (Microsoft – Advanced)

Tools for security testing

- OWASP ZAP
- Burp Suite
- Snyk
- Dependency Check

Mention:

“Automated security scans as part of CI/CD improve safety.”

STATIC CODE ANALYSIS

Q920. (Amazon – Interview Favorite)

What is static code analysis?

Analyzes code **without running it** to detect:

- Code smells
- Potential bugs
- Security vulnerabilities

Tools

- SonarQube
 - ESLint
 - StyleCop
-



TESTING MICROSERVICES

Q921. (Accenture – Senior Round)

Challenges testing microservices

- Distributed failures
 - Data consistency
 - Multiple moving parts
 - Network dependency issues
-

Q922. (Microsoft – Architecture Round)

How do you test microservices?

Approach

- Unit tests
 - Contract tests
 - Integration tests
 - End-to-end flows
-



TEST ENVIRONMENTS

Q923. (Amazon – Scenario)

Why separate test environments are required?

To avoid:

- Data conflicts
- Interference with real users
- Environment-specific bugs

Helps replicate production safely.

TEST DATA MANAGEMENT

Q924. (Accenture – Real Project)

How do you handle sensitive test data?

- Mask production data
- Use synthetic/fake data
- Follow compliance rules
- Secure storage

Never expose real user data.

SHIFT-LEFT TESTING

Q925. (Microsoft – Interview Favorite)

What is shift-left testing?

Test early in the development lifecycle.

Benefits

- Bugs found earlier = cheaper to fix

- Better quality
 - Faster releases
-

BUG TRIAGE & RCA

Q926. (Amazon – Senior Round)

How do you handle critical production bugs?

Steps:

- 1 Stabilize the system (fix or rollback)
 - 2 Analyze logs
 - 3 Perform RCA (Root Cause Analysis)
 - 4 Add tests/prevention measures
-

QUALITY CULTURE

Q927. (Accenture – HR + Tech)

How do you promote quality in a team?

- Code reviews
- Automated testing
- Pair programming
- Knowledge sharing

Quality is a culture.

QUALITY vs SPEED

Q928. (Microsoft – Architecture Round)

How do you balance quality and delivery speed?

- Automate as much as possible
- Focus testing on high-risk features
- CI/CD + good engineering discipline

Deliver fast without breaking stability.

COMMON PITFALLS

Q929. (Deloitte – Senior Round)

Why does test automation fail in projects?

- No strategy
- Wrong tools selected
- Hard-to-maintain scripts
- Lack of CI integration

Automation must be treated like real development.

FINAL QUALITY QUESTION

Q930. (ALL MNCs – FINAL QUALITY QUESTION)

What defines a high-quality software product?

Best Interview Answer

“A high-quality product is reliable, secure, fast, user-friendly, well-tested, and easy to maintain in the long run.”

CHAPTER 14 – SECURITY (FULL-STACK & CLOUD LEVEL)

♦ Core Security Concepts & OWASP Fundamentals (Q931–Q950)

(Asked in Amazon, Microsoft, Deloitte, Accenture, Infosys)



SECURITY BASICS (INTERVIEW FOUNDATION)

Q931. (ALL MNCs – BASIC SECURITY QUESTION)

What is application security?

Easy Explanation

Application security means **protecting software from unauthorized access, data leaks, and attacks** throughout its lifecycle:

- During development
- During deployment
- During runtime

It ensures that only the right users access the right data in a safe way.

Key Point

Security is **not optional**. It must be part of design, coding, testing, and deployment.

Q932. (Accenture – Interview Favorite)

Authentication vs Authorization

Authentication	Authorization
Verifies identity	Verifies permissions
“Who are you?”	“What can you access?”
Login step	After login step

Real Example

Login to bank → that proves identity (Authentication).
Seeing only your account → that is Authorization.



OWASP TOP 10 (VERY IMPORTANT)

Q933. (Microsoft – Interview Favorite)

What is OWASP Top 10?

OWASP Top 10 is a globally recognized list of the **most critical web application security risks**.

Examples include:

- SQL Injection
- XSS
- Broken authentication
- Sensitive data exposure

Interviewers expect awareness, not memorization.

Q934. (Amazon – Security Round)

What is SQL Injection and how do you prevent it?

SQL Injection happens when an attacker **injects SQL code** through user inputs to manipulate the database.

Example

Login input:

```
' OR 1=1 --
```

This can bypass login and expose data.

Prevention

- Parameterized queries
 - ORM (Entity Framework)
 - Input validation
-

Q935. (Microsoft – Interview Favorite)

What is Cross-Site Scripting (XSS)?

XSS occurs when attackers inject malicious scripts into webpages which execute in user browsers.

Example

A comment box accepting:

```
<script>alert("Hacked")</script>
```

Prevention

- Input sanitization
- Output encoding

- Avoid unsafe HTML rendering
-

Q936. (Accenture – Security Round)

What is CSRF and how do you prevent it?

CSRF (Cross-Site Request Forgery) tricks a logged-in user into performing unintended actions.

Prevention

- Anti-forgery tokens
 - SameSite cookies
 - CSRF protection middleware
-



API SECURITY (VERY IMPORTANT)

Q937. (Microsoft – Interview Favorite)

How do you secure REST APIs?

- Authentication (JWT / OAuth2)
- Authorization
- HTTPS
- Rate limiting
- Input validation

Security should be **layered**, not dependent on a single control.

Q938. (Amazon – Advanced)

Why is HTTPS mandatory?

HTTPS encrypts data in transit and prevents:

- Man-in-the-middle attacks
- Credential theft
- Sensitive data leaks

Simply put: **HTTP = insecure**, **HTTPS = secure**.



JWT & TOKEN SECURITY

Q939. (Accenture – Interview Favorite)

What is JWT and why is it secure?

JWT (JSON Web Token) is a token containing user identity and claims, digitally **signed** by the server.

Important Note

- JWT is **tamper-proof**
 - JWT is **not encrypted** by default
-

Q940. (Microsoft – Security Round)

Common JWT security mistakes

- Very long expiry tokens
- Storing JWT in localStorage (XSS risk)

- No token revocation strategy
-



SECURE CODING PRACTICES

Q941. (Amazon – Interview Favorite)

What is secure coding?

Secure coding means writing code that:

- Validates input
- Handles errors safely
- Avoids exposing internal system details

The goal is to prevent vulnerabilities before attackers find them.

Q942. (Microsoft – C# Security)

How do you store secrets securely in .NET apps?

- Azure Key Vault
- Environment variables
- User Secrets in development

✗ Never hardcode secrets in code or config files.



CLOUD SECURITY

Q943. (Accenture – Cloud Security)

Shared Responsibility Model

Cloud provider secures:

- Infrastructure
- Datacenters
- Hardware

Customer secures:

- Application
- Data
- Identity & Access

IDENTITY & ACCESS

Q944. (Microsoft – Security Round)

What is Role-Based Access Control (RBAC)?

RBAC grants access based on user roles.

Example

Admin → Full permissions

User → Limited access

SECURITY HEADERS

Q945. (Amazon – Advanced)

What are security headers and why important?

Security headers protect browsers from attacks.

Examples:

- Content-Security-Policy
- X-Frame-Options
- Strict-Transport-Security

They reduce XSS, clickjacking, and injection risks.



ERROR HANDLING & LOGGING

Q946. (Accenture – Interview Favorite)

Why should error messages not expose details?

Detailed errors reveal:

- Technology stack
- Database names
- Internal structure

Hackers love detailed errors.

Provide **user-friendly** but **non-revealing** messages.



SECURITY TESTING

Q947. (Microsoft – Security Testing)

How do you test the security of applications?

- Code reviews
- Automated security scans
- Penetration testing
- OWASP checklist

Security must be tested continuously.



REAL INCIDENT SCENARIOS

Q948. (Amazon – Scenario Based)

User reports suspicious activity – what steps?

- 1 Lock or secure the account
 - 2 Investigate logs
 - 3 Reset credentials
 - 4 Assess impact
 - 5 Fix root cause
-



COMPLIANCE & GOVERNANCE

Q949. (Deloitte – Compliance Round)

What is GDPR in simple words?

GDPR is a European law that protects user personal data and privacy.

Key idea:
Users own their data, not companies.

FINAL SECURITY QUESTION

Q950. (ALL MNCs – FINAL SECURITY QUESTION)

What is the biggest security mistake developers make?

Best Interview Answer

“The biggest mistake is treating security as an afterthought instead of designing it from the beginning.”

◆ Advanced Security, Architecture & Real Incidents (Q951–Q970)

Q951. (Microsoft – Advanced Security)

What is Broken Authentication?

Easy Explanation

Broken authentication happens when **login, session, or token handling is implemented incorrectly**, allowing attackers to impersonate real users.

Real-World Examples

- Weak password policies
- Session IDs predictable or stolen
- No session timeout

This directly impacts user accounts and financial systems.

Q952. (Amazon – Security Round)

What is Broken Access Control?

Explanation

Users can access resources or actions they **should not be allowed to**.

Example

A normal user accessing:

`/admin/dashboard`

or modifying someone else's order.

This is one of the most critical OWASP risks.



INPUT & DATA SECURITY

Q953. (Accenture – Interview Favorite)

Why is input validation alone not enough?

Explanation

Attackers **bypass frontend validation easily** using tools like Postman, CURL, or proxy tools.

So backend validation is mandatory.

The backend is the real gatekeeper.



SENSITIVE DATA PROTECTION

Q954. (Microsoft – Data Security)

How do you protect sensitive data at rest?

Explanation

- Encrypt database storage
- Secure where data is stored
- Control who can access it

Azure Examples

- Azure SQL Transparent Data Encryption (TDE)
 - Encrypted Blob storage
-

Q955. (Amazon – Security)

Encryption at rest vs in transit

At Rest	In Transit
Stored data	Moving data
Disk or DB encryption	HTTPS / TLS

Both are mandatory in modern systems.



SECURE API DESIGN

Q956. (Accenture – Interview Favorite)

How do you prevent API abuse?

Explanation

- Rate limiting
- API keys / authentication
- Monitoring & alerts

Stops bots, scraping, and brute-force attacks.



AUTHORIZATION MODELS

Q957. (Microsoft – Advanced)

RBAC vs ABAC

RBAC	ABAC
Role-based access	Attribute-based
Simple	Very flexible
Used widely	Used in complex policies

Example of ABAC:

Allow access **if the user is manager AND department = finance.**



DEVOPS & SECURITY (DEVSECOPS)

Q958. (Amazon – DevSecOps)

What is DevSecOps?

Explanation

DevSecOps integrates **security into CI/CD pipelines**, not just at the end.

Example

- Static code scan in build
- Dependency scan before deployment

Security becomes continuous.



SECRETS MANAGEMENT

Q959. (Microsoft – Security Round)

Why should secrets never be stored in code or repos?

Explanation

If repos leak, secrets leak. That means instant compromise.

Best Practice

- Azure Key Vault
 - Managed identity
 - Environment variables
-



CLOUD SECURITY

Q960. (Accenture – Cloud Security)

How do you secure cloud resources?

Explanation

- IAM (role-based permissions)
- Private networks / firewalls
- Logging and monitoring

Security = Access + Network + Visibility.



NETWORK SECURITY

Q961. (Microsoft – Advanced)

What is WAF and why is it important?

Web Application Firewall protects apps from common attacks like:

- SQL Injection
- XSS
- Bot attacks

It is an important security shield in production.



LOGGING & AUDITING

Q962. (Amazon – Security Ops)

Why are audit logs critical?

Because they answer:

Who did what, when, and from where?

Critical for:

- Forensics
- Compliance
- Legal investigations



INCIDENT RESPONSE

Q963. (Deloitte – Scenario Based)

Steps after a security breach

- 1 Contain
- 2 Investigate
- 3 Fix
- 4 Inform stakeholders
- 5 Prevent recurrence

Handling a breach is as important as preventing one.



SUPPLY CHAIN SECURITY

Q964. (Microsoft – Advanced)

What is dependency vulnerability?

Third-party libraries may contain security flaws.

Example

Old npm package or outdated NuGet exposing your app.

So always update and scan dependencies.



SECURITY HARDENING

Q965. (Accenture – Interview Favorite)

How do you harden an application?

- Disable unused features
- Secure configurations
- Least privilege access

Reduce attack surface = reduce risk.



FRONTEND SECURITY

Q966. (Amazon – Frontend Security)

How do you secure frontend applications?

- Input sanitization
- Secure storage (avoid localStorage for sensitive tokens)
- HTTPS everywhere

Frontend is not security — but it shouldn't be weak either.



SECURITY MISCONFIGURATION

Q967. (Microsoft – OWASP)

What is security misconfiguration?

Leaving insecure default settings.

Example

- Default passwords
- Open blob storage
- Debug enabled in production

This is one of the most common real-world vulnerabilities.



COMPLIANCE & STANDARDS

Q968. (Deloitte – Compliance)

ISO 27001 in simple words

A global standard for **information security management**.

It ensures organizations handle data securely.



SECURITY MINDSET

Q969. (ALL MNCs – IMPORTANT)

How do you build a security mindset in a team?

- Regular security training
- Code reviews with security checks
- Automated scans

- Culture of awareness

Security is a habit.



FINAL SECURITY QUESTION

Q970. (ALL MNCs – FINAL)

What is the best security advice for developers?

Best Interview Answer

“Assume your application will be attacked and design accordingly.”

CHAPTER 15 – LEADERSHIP, ARCHITECTURE & FINAL MNC ROUNDS

◆ Tech Lead Thinking, Decision Making & Senior Scenarios (Q971–Q990)

(Amazon, Microsoft, Deloitte, Accenture – Final / Bar Raiser rounds)



LEADERSHIP FUNDAMENTALS (Q971–Q990)

Q971. (ALL MNCs – FINAL ROUND FAVORITE)

What does leadership mean for a software engineer?

Easy Explanation

Leadership is not about position. It means:

- Taking ownership
- Supporting teammates
- Making responsible decisions
- Ensuring delivery without compromising quality

Interview-ready line

“A leader ensures both the team and the system succeed — technically and professionally.”

Q972. (Microsoft – Senior Round)

How do you lead without authority?

Explanation

- Influence through data and reasoning
- Lead by example
- Help others grow
- Earn trust instead of demanding it

Real leaders don't need titles to inspire action.



ARCHITECTURE DECISION MAKING

Q973. (Amazon – Architecture Round)

How do you make architecture decisions?

Step-by-step

- 1 Understand requirements
- 2 Identify constraints and risks
- 3 Compare multiple approaches
- 4 Evaluate trade-offs
- 5 Take decision
- 6 Document and communicate

Clear thinking + clarity = strong design leadership.

Q974. (Accenture – Interview Favorite)

Tell me about a design decision you regret

Strong Answer

“I optimized too early once, which increased complexity. Since then, I measure first, optimize later. Simplicity first, scalability when required.”



OWNERSHIP & ACCOUNTABILITY

Q975. (Amazon – Leadership Principle)

Describe a time you took ownership beyond your role

STAR Approach

- Situation: Production incident
- Action: Took charge, coordinated teams, communicated transparently
- Result: System restored, RCA completed, preventive actions implemented

Ownership means acting when needed, not when asked.



TECH LEAD SCENARIOS

Q976. (Microsoft – Tech Lead Round)

How do you handle junior developers making mistakes?

Best Answer

“I treat mistakes as learning opportunities. I guide them calmly, explain impact, and help improve their approach instead of blaming.”

This builds strong engineers.

COMMUNICATION & STAKEHOLDERS

Q977. (Deloitte – Client Round)

How do you explain technical risks to non-technical stakeholders?

Explanation

- Use simple language
- Explain business impact
- Present safer alternatives
- Avoid technical jargon

Communication is leadership.

CONFLICT RESOLUTION

Q978. (Accenture – Behavioral)

How do you handle conflict between team members?

Answer

“I listen to both sides, align everyone to business goals, focus on facts rather than ego, and help reach a win-win solution.”



QUALITY VS DELIVERY PRESSURE

Q979. (Amazon – Scenario Based)

Business wants fast delivery; quality may suffer. What do you do?

Best Answer

“I explain risks and propose a phased delivery or safer approach. Speed is important, but stability and user trust are more important.”



ARCHITECTURE TRADE-OFFS

Q980. (Microsoft – Architecture Round)

How do you balance simplicity and scalability?

Explanation

- Start simple
- Keep architecture modular
- Design extension points

- Scale when real demand appears

Smart scaling is timing + design.



FAILURE & LEARNING

Q981. (ALL MNCs – IMPORTANT)

Tell me about a project that failed

Focus on:

- What went wrong
- What you learned
- What you changed next time

Mature engineers learn, not blame.



DECISION UNDER UNCERTAINTY

Q982. (Amazon – Bar Raiser)

How do you make decisions with incomplete information?

Best Answer

“I make logical assumptions, validate quickly, monitor impact, and adjust based on feedback.”



MENTORSHIP & GROWTH

Q983. (Microsoft – Senior Round)

How do you mentor team members?

Explanation

- Code reviews
- Pair programming
- Sharing design thinking
- Encouraging independent problem solving

Mentorship = multiplying team capability.



ETHICS & RESPONSIBILITY

Q984. (Deloitte – Ethics Round)

What if a client asks for something unethical?

Answer

“I would respectfully deny, explain risks, involve leadership if needed, and always prioritize integrity and user trust.”



LONG-TERM THINKING

Q985. (Amazon – Leadership)

How do you design systems for the future?

Explanation

- Modular architecture
- Observability
- Avoid premature complexity

Future-ready ≠ over-engineered.



CROSS-TEAM COLLABORATION

Q986. (Accenture – Program Level)

How do you work with multiple teams?

Explanation

- Clear communication
- Well-defined APIs
- Proper documentation
- Regular sync meetings

Collaboration reduces friction.



CAREER MATURITY

Q987. (ALL MNCs)

What differentiates a senior engineer from a junior?

Best Answer

“A junior focuses only on coding. A senior focuses on impact, reliability, trade-offs, and long-term success.”



FINAL SELF-REFLECTION

Q988. (Microsoft – Final Round)

What is your biggest professional strength?

Safe answer:

Problem-solving + ownership + calm under pressure.



VISION & TRUST

Q989. (Accenture – Final HR)

Why should we trust you with critical systems?

Best Answer

“Because I value stability, security, communication, and responsibility — above shortcuts.”



FINAL MNC QUESTION

Q990. (ALL TOP MNCs – FINAL QUESTION)

If selected, what value will you add to our organization?

Perfect Closing Answer

“I will bring strong technical expertise, ownership mindset, collaborative attitude, and a focus on building reliable, scalable systems that deliver real business value.”

FINAL CHAPTER – MASTER QUESTIONS (Q991–Q1000)

◆ Ultimate Interview, Career & Mindset Questions

(Final HR + Architecture + Leadership Blend)

FINAL TECH + LEADERSHIP REFLECTION (Q991–Q1000)

Q991. (ALL TOP MNCs – FINAL TECH REFLECTION)

What is the most important lesson you learned as a software engineer?

Best Easy Answer

“Technology keeps changing, but fundamentals, clean design, ownership, and a responsible mindset never go out of style. Long-term success comes from building systems thoughtfully and taking accountability for them.”

Q992. (Amazon – Bar Raiser)

Tell me about a time you had to make a tough technical decision

Strong Answer Structure

- **Context** – What was at stake?

- Options considered
- Trade-offs discussed with team
- Decision
- Outcome

Perfect Summary Line

“I chose stability over speed. It helped avoid tech debt and protected production reliability.”

Q993. (Microsoft – Final Architecture Round)

How do you balance innovation with stability?

Perfect Answer

“I encourage innovation in safe environments like POCs and staging, but I protect production using proven, reliable, and well-tested solutions. This way, the business benefits from both innovation and stability.”

Q994. (Accenture – Leadership Round)

How do you handle responsibility when something goes wrong?

Best Answer

“I take ownership, fix the issue quickly, communicate transparently, and ensure we put strong preventive measures in place so the problem never repeats.”

Q995. (Deloitte – Ethics & Trust)

What would you do if you strongly disagree with a decision that affects users?

Best Answer

“I would raise my concerns respectfully with data, explain risks and alternatives, and ensure user impact is clearly understood. Once a final decision is made, I support it and help implement it responsibly.”

Q996. (ALL MNCs – CAREER VISION)

What kind of engineer do you want to be remembered as?

Easy Answer

“Someone reliable, ethical, collaborative, and known for building meaningful, stable systems that genuinely solve real user problems.”

Q997. (Amazon – Ownership Principle)

Describe a situation where you prevented a big issue before it happened

STAR Approach

- Identified risk early
- Analyzed impact
- Took proactive action
- Prevented future failure

Final Line

“Preventing failures is as valuable as fixing them — sometimes even more.”

Q998. (Microsoft – Culture Fit)

How do you adapt to change in technology and teams?

Best Answer

“By staying curious, continuously learning, embracing change positively, and helping teammates adapt so the whole team grows together.”

Q999. (ALL TOP MNCs – FINAL HR QUESTION)

Do you have anything you want to add before we conclude?

Perfect Closing Line

“I truly appreciate the discussion today. I’m excited about the opportunity and confident I can bring strong value while continuing to grow with the team.”

Q1000. (ALL MNCs – THE LAST QUESTION)

Why should we hire you over other candidates?

BEST POSSIBLE FINAL ANSWER

“Because I combine strong technical expertise with ownership, clear communication, business understanding, and a mindset focused on building scalable, secure, and reliable systems that create real value for users and the organization.”



30-60-90 Day Study Plan

A Practical Roadmap to Crack Top MNC Interviews

This study plan is designed to help you prepare **systematically** for interviews at **Amazon, Microsoft, Accenture, Deloitte, Capgemini, Cognizant, Infosys, and other top MNCs.**

You can adapt the pace based on your availability, but **consistency is more important than speed.**



Recommended Daily Time

- Weekdays: **2–3 hours/day**
 - Weekends: **4–6 hours/day**
-

DAYS 1–30: FOUNDATION & CONFIDENCE

Goal: Clear screening rounds and junior–mid technical interviews

Week 1 – DSA Fundamentals

Focus

- Arrays, Strings
- Hashing & Dictionary
- Basic problem-solving mindset

Use Chapters

- Chapter 1 – DSA (Q1–Q40)

Outcome

- Solve problems confidently
 - Explain logic clearly (not just code)
-

Week 2 – Core OOP & C#

Focus

- OOP principles with real examples
- C# basics, LINQ
- Clean coding habits

Use Chapters

- Chapter 2 – OOP (Q201–Q240)
- Chapter 3 – C# Basics

Outcome

- Explain OOP without memorization
 - Understand real-world usage
-

♦ Week 3 – ASP.NET Core & SQL

Focus

- Web API fundamentals
- REST principles
- SQL joins, indexes

Use Chapters

- Chapter 4 – ASP.NET Core
- Chapter 5 – Database & SQL (Basics)

Outcome

- Build and explain a simple API
 - Understand how backend works in production
-

♦ Week 4 – Frontend Basics + HR

Focus

- React or Angular (choose one)

- Basic UI + API integration
- HR fundamentals

Use Chapters

- Chapter 6 (React) OR Chapter 7 (Angular)
- Chapter 10 – HR & Scenario-Based

Outcome

- Explain your project clearly
- Deliver a confident self-introduction

DAYS 31–60: SENIOR-LEVEL SKILLS

Goal: Clear senior technical and architecture rounds

◆ Week 5 – Advanced Backend & Security

Focus

- Async/await
- JWT authentication
- Error handling & logging

Use Chapters

- Chapter 3 – Advanced C#
- Chapter 4 – Advanced Web API
- Chapter 14 – Security (Basics)

Outcome

- Secure APIs confidently
 - Explain backend decisions clearly
-

♦ Week 6 – DevOps, Docker & CI/CD

Focus

- CI/CD concepts
- Docker basics
- Azure DevOps pipelines

Use Chapters

- Chapter 8 – CI/CD, Docker & Azure

Outcome

- Explain how code moves from dev to production
-

♦ Week 7 – Cloud & Azure

Focus

- Azure App Service
- Storage, monitoring
- Scalability basics

Use Chapters

- Chapter 9 – Cloud & Azure

Outcome

- Explain cloud hosting and scaling with confidence

♦ Week 8 – Testing & Quality

Focus

- Unit testing
- Mocking
- Test pyramid

Use Chapters

- Chapter 13 – Testing & Quality Engineering

Outcome

- Understand quality mindset expected in MNCs

DAYS 61–90: FINAL ROUND & ARCHITECT MINDSET

Goal: Clear system design, leadership, and final HR rounds

♦ Week 9 – System Design Fundamentals

Focus

- HLD vs LLD
- Scalability
- Caching & load balancing

Use Chapters

- Chapter 11 – System Design (Q781–Q820)

Outcome

- Design systems verbally and clearly
-

♦ Week 10 – Microservices & Architecture

Focus

- Microservices patterns
- Communication strategies
- Trade-offs

Use Chapters

- Chapter 12 – Microservices & Architecture

Outcome

- Think like a senior/architect
-

♦ Week 11 – Security & Leadership

Focus

- OWASP risks
- Secure design
- Ownership & leadership mindset

Use Chapters

- Chapter 14 – Security
- Chapter 15 – Leadership & Final Rounds

Outcome

- Handle bar-raiser and final interviews confidently

◆ Week 12 – Mock Interviews & Revision

Focus

- Company-wise mock interviews
- Weak area revision
- HR answer practice

📖 Use Chapters

- Company-Wise Mock Interviews
- Final Questions (Q991–Q1000)

🎯 Outcome

- Interview-ready mindset
- Calm, confident performance

🧠 How to Use This Plan Effectively

- ✓ Speak answers aloud
- ✓ Practice explaining, not memorizing
- ✓ Revise weak areas every weekend
- ✓ Focus on clarity over complexity

🏆 After 90 Days, You Will Be Able To

- Answer MNC interview questions confidently
- Design scalable systems clearly
- Handle HR and leadership rounds professionally
- Think and speak like a **Top-MNC engineer**

🌟 Final Note

This plan is not about rushing.
It's about **building confidence step by step**.

Consistency beats intensity. Always.

🏁 Final Interview-Day Checklist

What to Do Before, During, and After Your Interview

This checklist is designed to help you **perform at your best on interview day**, avoid common mistakes, and present yourself as a **calm, confident, and professional engineer**.

Many candidates fail interviews not because of lack of knowledge, but because of **poor interview-day execution**.

This checklist eliminates that risk.

PART 1 – THE DAY BEFORE THE INTERVIEW

🧠 Mental Preparation

- ✓ Stop learning new topics
- ✓ Revise only your **strong areas**
- ✓ Review your projects and resume
- ✓ Sleep for **7–8 hours**

- ✗ Do not panic
- ✗ Do not over-study late night

Remember: *You are revising, not learning from scratch.*

📄 Document Preparation

- ✓ Resume (latest version)
 - ✓ Job description reviewed
 - ✓ Company research done
 - ✓ Interview link / address verified
-

Technical Setup (Online Interviews)

- ✓ Laptop fully charged
 - ✓ Stable internet connection
 - ✓ Camera and microphone tested
 - ✓ Backup internet (if possible)
-

Appearance & Environment

- ✓ Wear clean, professional clothing
- ✓ Neutral background
- ✓ Well-lit room
- ✓ Comfortable seating

First impressions matter in the first 30 seconds.

PART 2 – 15 MINUTES BEFORE INTERVIEW

- ✓ Join the call early
- ✓ Close all notifications
- ✓ Keep water nearby
- ✓ Take 3 deep breaths

Say to yourself:

“I am prepared. I belong here.”

PART 3 – DURING THE INTERVIEW (MOST IMPORTANT)

How to Answer Questions

- ✓ Listen carefully
- ✓ Pause 1–2 seconds before answering
- ✓ Think aloud (especially in coding & system design)
- ✓ Keep answers structured
- ✗ Don't rush
- ✗ Don't interrupt the interviewer

Technical / Coding Rounds

- ✓ Explain your approach first
- ✓ Ask clarifying questions
- ✓ Mention edge cases
- ✓ Write clean, readable code

If stuck, say:

“Let me think through this step by step.”

✗ Never say: *“I don’t know anything about this.”*

System Design Rounds

- ✓ Clarify requirements first
- ✓ Start with high-level design
- ✓ Discuss scalability & failures
- ✓ Explain trade-offs clearly

- ✗ Don’t jump directly into tools
 - ✗ Don’t over-engineer
-

HR & Behavioral Rounds

- ✓ Be honest and calm
- ✓ Use STAR method (Situation, Task, Action, Result)
- ✓ Focus on learning and ownership

- ✗ Never blame team members
 - ✗ Never criticize previous company or manager
-

PART 4 – COMMON INTERVIEW MISTAKES TO AVOID

- ⊘ Speaking too fast
 - ⊘ Using buzzwords without explanation
 - ⊘ Arguing with interviewer
 - ⊘ Over-explaining simple answers
 - ⊘ Saying “I memorized this”
-

PART 5 – QUESTIONS YOU SHOULD ASK INTERVIEWERS

Always ask **at least one** question:

- ✓ What does success look like in this role?
- ✓ What are the current challenges of the team?
- ✓ What learning opportunities are available?

Asking questions shows **interest and maturity**, not weakness.

PART 6 – AFTER THE INTERVIEW

- ✓ Thank the interviewer (email or verbally)
- ✓ Write down questions you struggled with
- ✓ Revise those topics later

- ✗ Don't overthink
 - ✗ Don't lose confidence
-



POWER MINDSET (VERY IMPORTANT)

Interviewers are not looking for perfection.

They are looking for **clear thinking, honesty, and reliability**.



FINAL REMINDER

You are not there to prove you know everything.

You are there to show that you:

- Can think clearly
- Can explain confidently
- Can be trusted with real systems

If you've prepared using this book, **you are ready**.

This Book + This Checklist = Interview Confidence

Use this checklist **before every interview**, even after you get selected once.

Author's Note

This book is the result of **real-world experience**, not just theoretical knowledge.

Over the past **10+ years**, I have worked as a **Senior Full Stack Developer**, building and maintaining **enterprise-grade applications** using technologies such as **.NET Core, C#, React.js, Angular, Microservices, Azure, SQL, MongoDB, Docker, EF Core, and ML.NET**.

During my career, I have:

- Attended **multiple MNC interviews**
- Mentored developers preparing for **Amazon, Microsoft, Accenture, Deloitte, Capgemini, Cognizant, Infosys, and other top companies**
- Observed a common gap — *candidates often know concepts but struggle to explain them confidently in interviews*

This book was created to **bridge that gap**.

Every question and answer in this book is written to help you:

- Think like an **MNC engineer**
- Explain concepts in **simple, Easy language**
- Handle **technical, system design, and HR rounds** with confidence
- Grow not just as an interview candidate, but as a **professional software engineer**

My goal is simple:

To help you succeed in interviews and build a strong, long-term career in software engineering.

About the Author

Sandeep Pal

Senior Full Stack Developer

10+ Years of Industry Experience

Expertise:

.NET Core | C# | Node.js | React.js | Angular | Microservices
Azure | SQL | MongoDB | Docker | EF Core | ML.NET

 **Connect with me on LinkedIn:**
 <https://www.linkedin.com/in/sandeepal/>

I regularly share:

- Interview preparation tips
 - Real-world development insights
 - Career guidance for developers
-

A Final Word

If this book helps you gain confidence, crack an interview, or grow in your career, then it has achieved its purpose.

Keep learning.

Keep building.

And most importantly — **believe in yourself.**